Proof of Concept Technical Solution for the Marconi Law Firm, LLC. (WordPress Website)

Orlando, Florida

Abraham Hammond Jr

Marvel IT Services LLC

Table of Contents

Inventory 6

Custom Network 6

IDs and Passwords 6

Preface 7

Network Topology Diagram 8

SSH Install & Access 8

OpenSSH Install (Ubuntu) 9

Node.js Application (Ghost) on Docker 9

Update Rocky8-Docker Host Name 9

Update Rocky8-Docker 11

Install EPEL Packages 10

Install Nano Editor 10

Docker CE 11

Set up stable repository 11

Install Docker CE 12

Verify Docker version 12

Initialize Docker 13

Start Docker 13

Enable Docker 13

Test Docker (hello-world) 14

Disable SELinux (/etc/selinux.config) 14

Reboot VM 15

Confirm SELinux Status 15

Install Ghost Docker Container 16

Check for Ghost Container (docker ps) 16

Ghost Container ID 16

NginX Reverse Proxy 16

Update Rocky8-Nginx Host Name 17

Update Rocky8-Nginx 18

Install EPEL Packages 17

Install Nano Editor 18

Disable SELinux 19

Reboot VM **19**

Confirm SELinux Status **20**

Rocky Firewall **20**

Stop Firewall 20

Disable Firewall 21

NginX **21**

Install NginX 21

Start NginX 22

Enable NginX 22

Confirm NginX Status 23

Reverse Proxy for Ghost Site **23**

Edit NginX configuration file 23

Reload NginX service 24

Terminate Docker 24

Delete First Ghost Container 25

Create New Ghost Container 25

Browse to Ghost from Firefox on Ubuntu (10.10.229.10/blog) 26

WordPress on Ubuntu - LAMP Stack ***28***

Update Ubuntu Host Name **28**

Update Ubuntu **28**

Upgrade Ubuntu **29**

Install Nano Editor **29**

Install Git **29**

Install Apache2 ..... **30**

Open Firewall Ports 80 and 443 30

Browse to Apache2 Ubuntu Default Page 30

Install MySQL **31**

Alter root user password (root@localhost) 31

Flush Privileges 32

Exit MySQL 32

Install PHP **33**

Install Required PHP Libraries 33

Install Required MySQL Libraries 33

Enable URL Rewrites (clean URLs) 34

Restart Apache Service 34

Create a test.php Web Page 35

Test the test.php Web Page 35

Database Configuration in MySQL Log into MySQL **36**

Create MySQL WordPress Database (WordPressDB) 36

Create MySQL WordPress User (WordPressUser) 37

Grant Privileges to the WordPress Database (WordPressDB) to WordPress User (WordPressUser) 37

Flush Privileges....38
Exit MySQL....38
Install WordPress....39
Grant Permission to /var/www/html/ Directory to WordPress User....39
Delete Files from /var/www/html/ Directory....39
Verify /var/www/html/ Directory is Empty....40
Clone WordPress to /var/www/html/ Directory....40
Verify /var/www/html/ Directory Contains WordPress Files....41
Verify Permissions on /var/www/html/ Directory....41
Edit Ownership....42
Edit Ownership of and the contents of /var/www/html/ Directory....42
Edit the apache2.conf File....42
Override All Default Apache Directives....43
Create a .htaccess File in the /var/www/html/.git/ Directory....43
Restart the Apache Service....44
WordPress Configuration....44
Configure WordPress....44
WordPress Configuration Selections....44
Run Installation....45
Create an Admin WordPress User....45
WordPress Site Selections....46
Test WordPress Website....46
WordPress Security Settings and Configurations....48
File Permissions....48
Vulnerability....48
Configuration....48
Validation....49
Securing wp-config.php....49
Vulnerability....49
Configuration....50
Validation....50
Firewall (Shield)....50
Vulnerability....51
Configuration....51
Validation....52
Conclusion....52
Appendix A....53
NginX Config File....53
Appendix B....54

NginX Access Log File **54**

NginX Error Log File **54**

Inventory

EQUIPMENT	OPERATIING SYSTEM	ADDITIONAL INFO	IP ADDRESS
Router/Custom Network	- (Firewall VM)	-Firewall VM	10.10.229.1
Docker	Rocky 8 (-Docker)	Ghost Container	10.10.229.11
NginX Reverse Proxy	Rocky 8 (-Nginx)	Reverse Proxy	10.10.229.10
WordPress	Ubuntu	LAMP Stack running WordPress	10.10.229.12

Custom Network

NETWORK NAME	SUBNET IP	SUBNET MASK	DNS	GATEWAY
ITE229	10.10.229.0	255.255.255.0	10.10.229.1	10.10.229.1

IDs and Passwords

ACCOUNT	USER ID	PASSWORD
Rocky8-Docker Root User	root	Fullsail1!
Rocky8-Nginx Root User	root	Fullsail1!
Ubuntu Root User	Root	Fullsail1!
MySQL Root User	root@localhost	Fullsail1!
MySQL WordPress User	WordPressUser	Fullsail1!
WordPress Admin	admin	Fullsail1!

Preface

This document will serve as proof of concept to Mr. Marconi for creating his WordPress website for his law firm and as audit documentation.

The purpose of audit documentation is to provide a comprehensive record of the organization's information technology infrastructure and security controls and processes. It plays a crucial role in providing transparency, accountability, and QA/QC regarding an organization's cybersecurity controls and practices. It enables organizations to demonstrate compliance, identify areas for improvement, and make informed decisions to strengthen their overall organizational cybersecurity.

Audit documentation serves several important purposes:

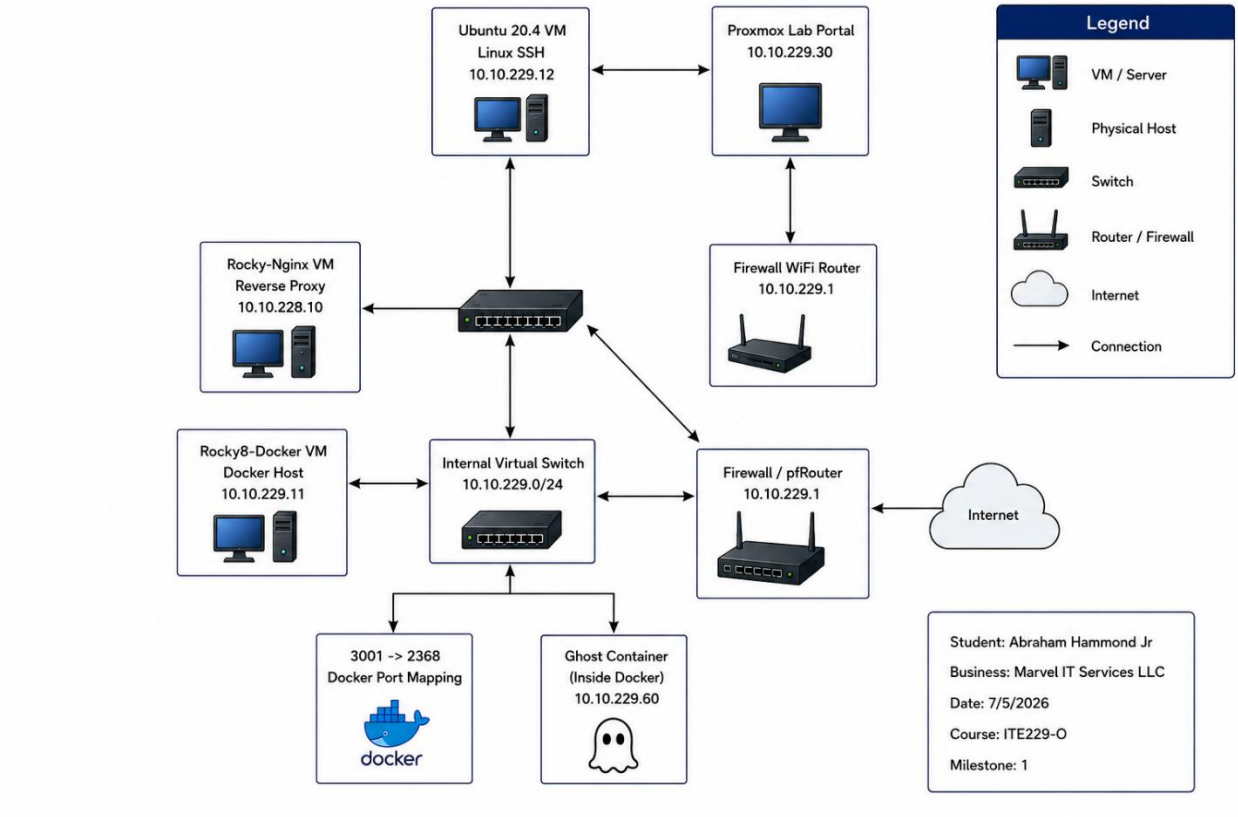
- Compliance: Evidence that an organization has undergone a thorough examination of its systems. It helps validate that the organization has implemented appropriate controls to protect its information systems and sensitive data.

- Validation: Verification of the effectiveness and adequacy of cybersecurity controls. It provides detailed information about the design, implementation, and operation of these controls, enabling reviewers to assess their reliability and identify any gaps or weaknesses.

- Records Maintenance: Historical record of cybersecurity audits conducted over time. It enables organizations to track their progress, identify trends, and evaluate the effectiveness actions taken. It also serves as reference for future audits and allows auditors to understand the current cybersecurity implemented and facilitates a more targeted approach to future cybersecurity updates and audits.

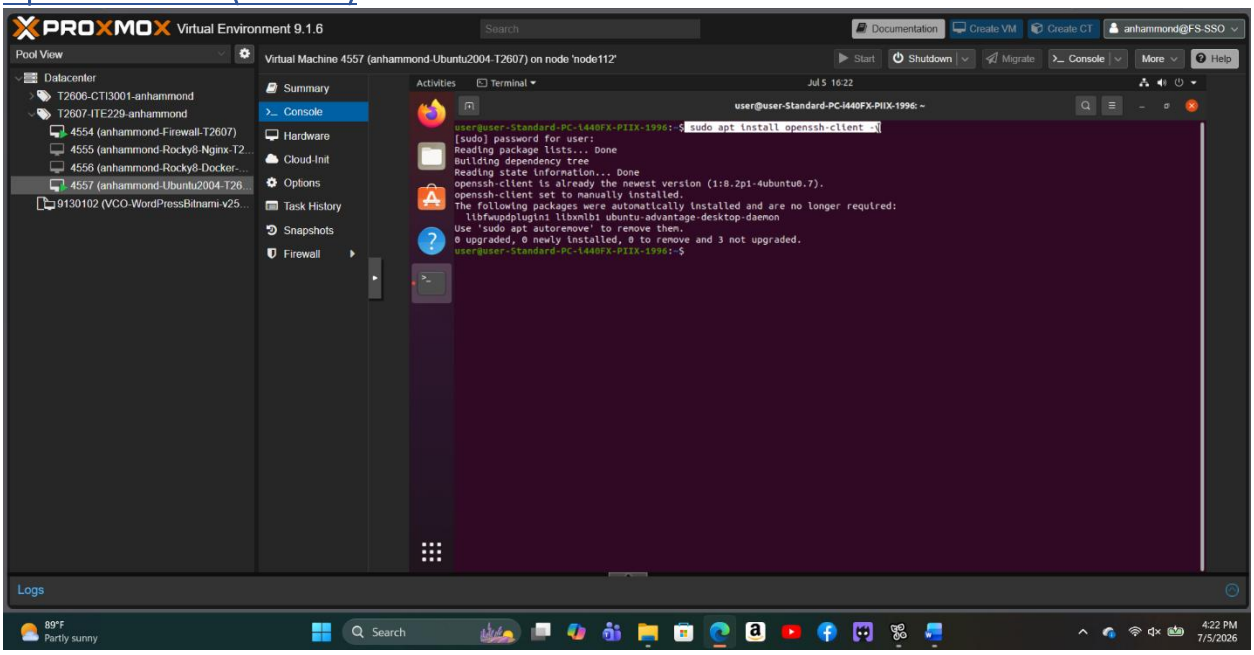
- Decision-making Support: Valuable insights and information that can support decision-making processes. It allows management to make informed decisions about allocating resources, prioritizing cybersecurity investments, and addressing identified risks and vulnerabilities.

Network Topology Diagram



SSH Install & Access

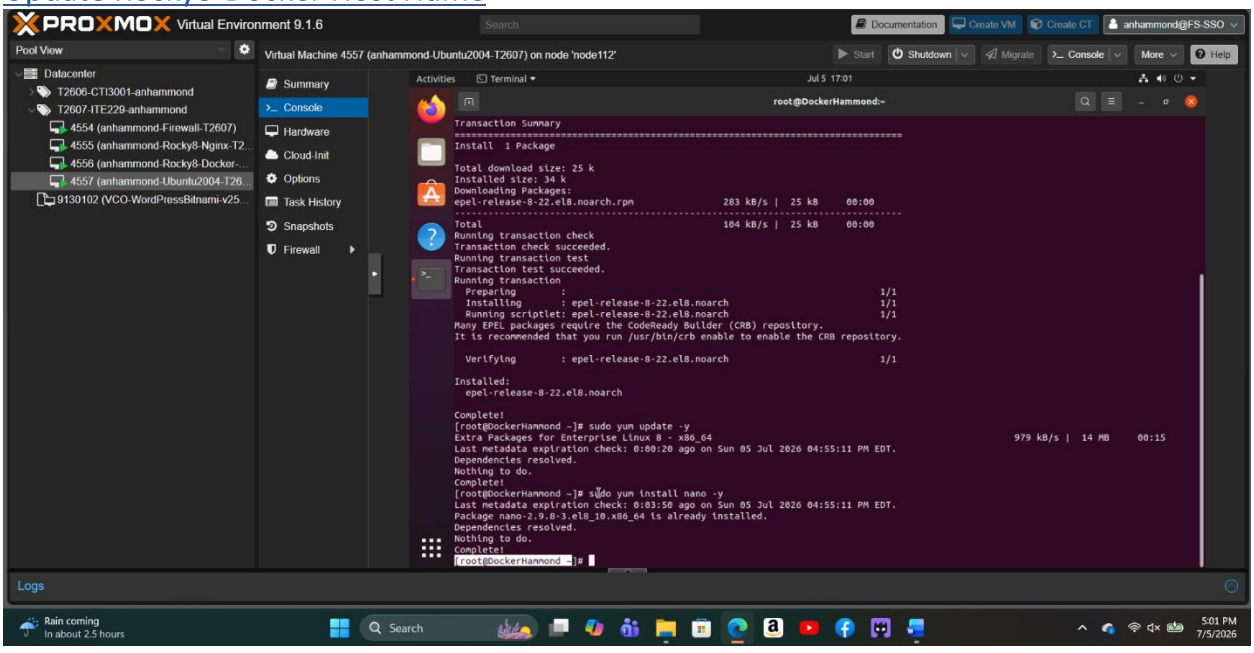
OpenSSH Install (Ubuntu)



I installed SSH using the command “sudo apt install openssh-client -y”, which downloaded all packages.

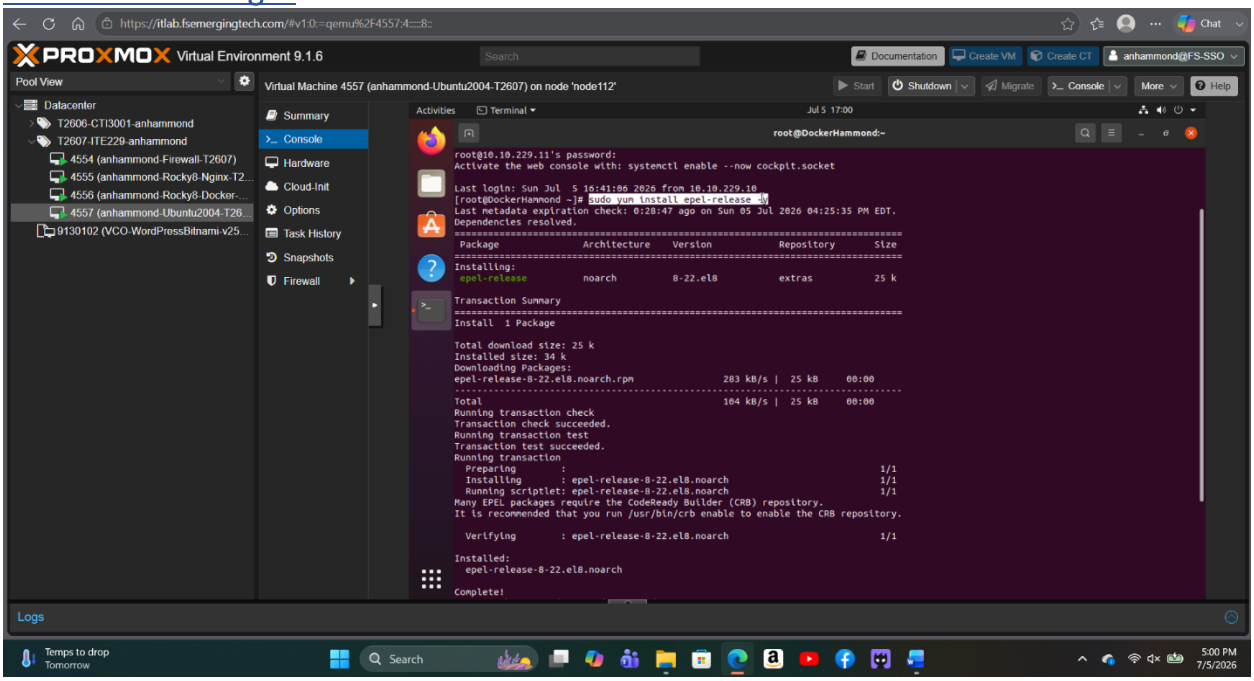
Node.js Application (Ghost) on Docker

Update Rocky8-Docker Host Name



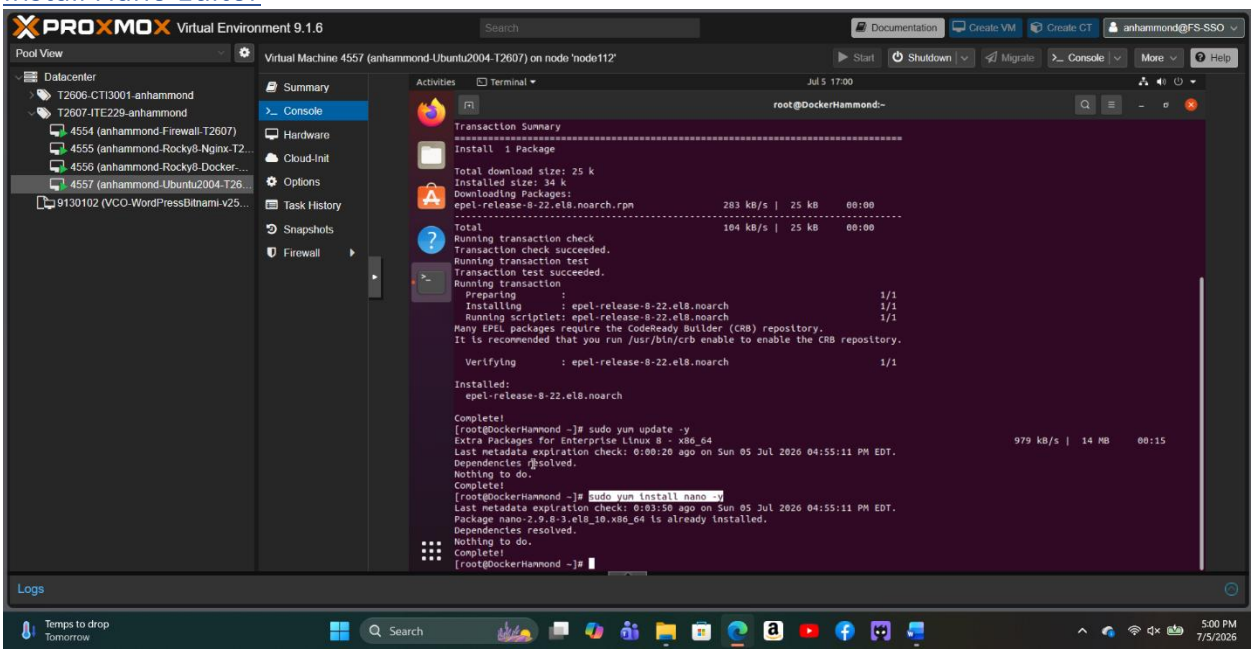
I changed the docker name to “DockerHammond” by typing in “nmtui”, and changed the hostname.

Install EPEL Packages



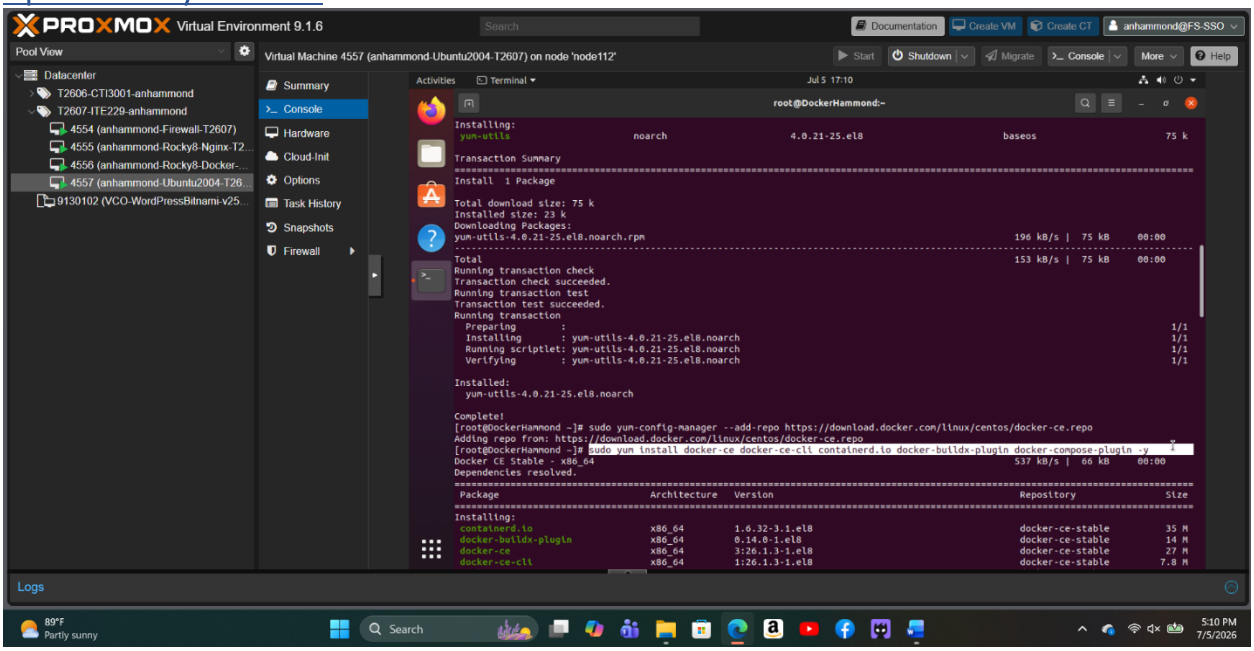
I downloaded more packages using “sudo yum install epel-release -y”.

Install Nano Editor



I then used “sudo install nano -y” to install the Nano Editor.

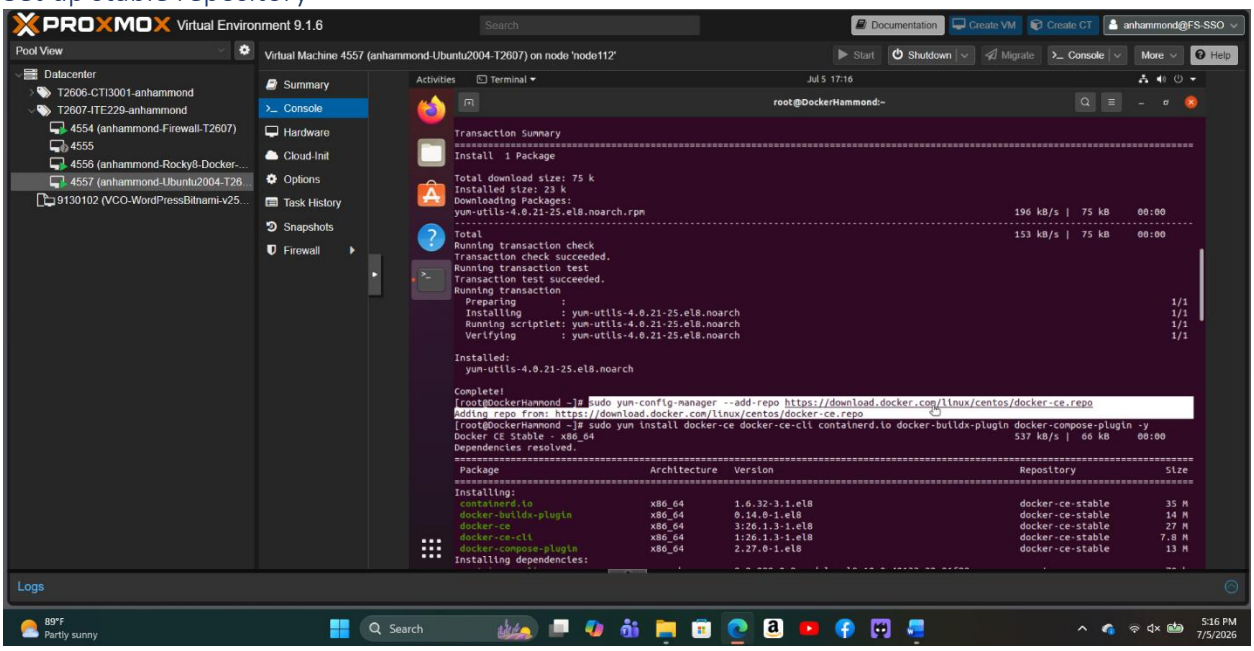
Update Rocky8-Docker



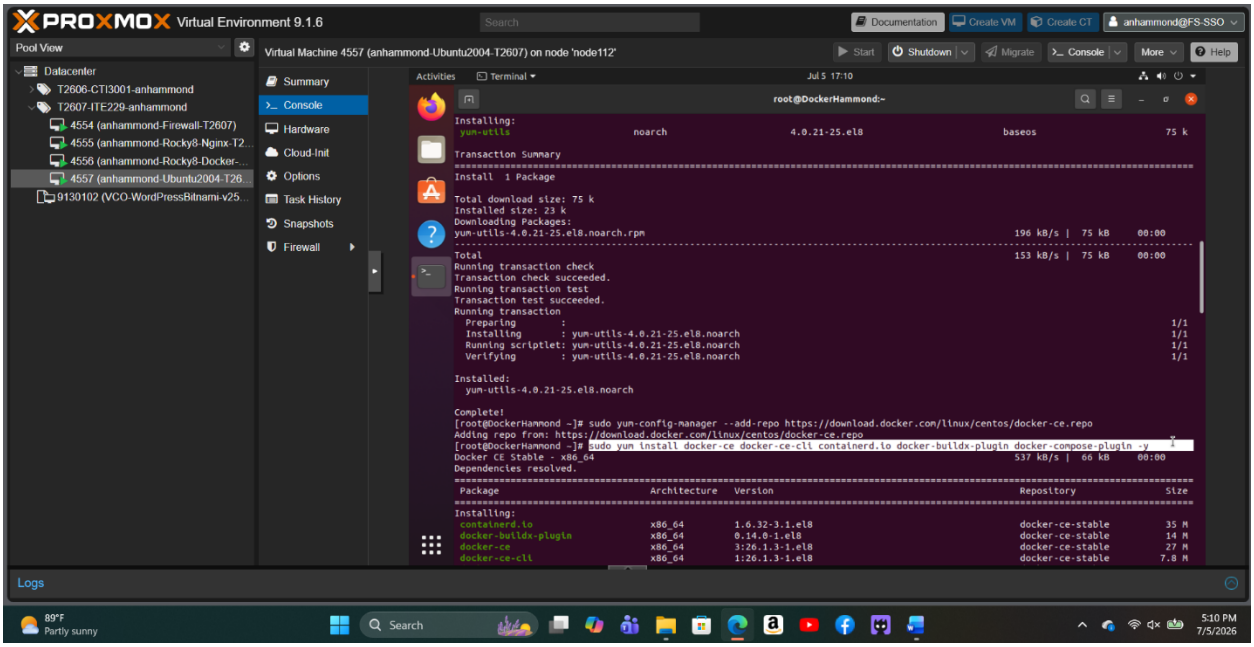
Using the command provided in the screenshot, I installed and updated Rocky.

Docker CE

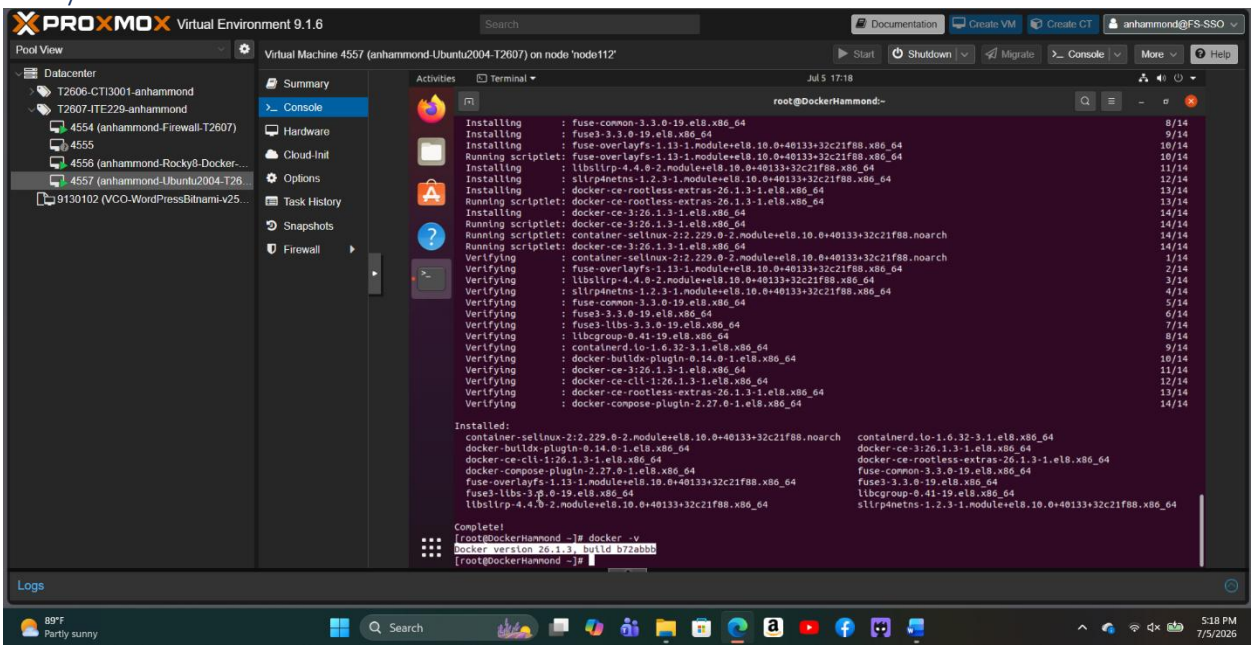
Set up stable repository



Install Docker CE

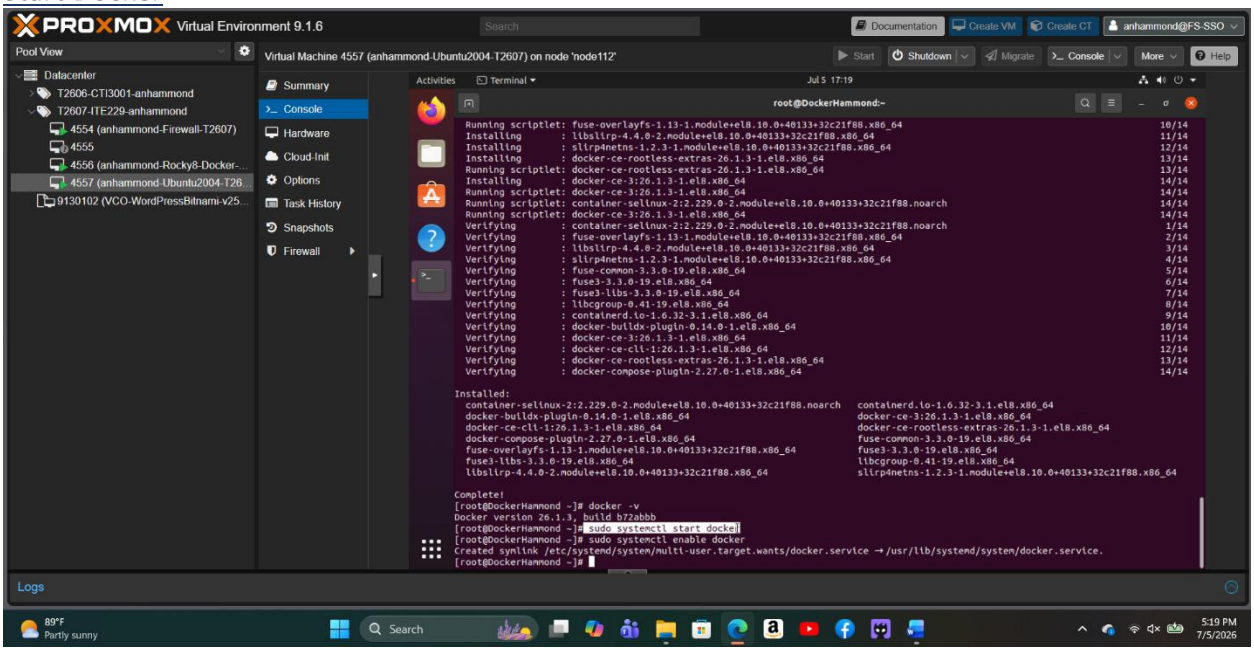


Verify Docker version

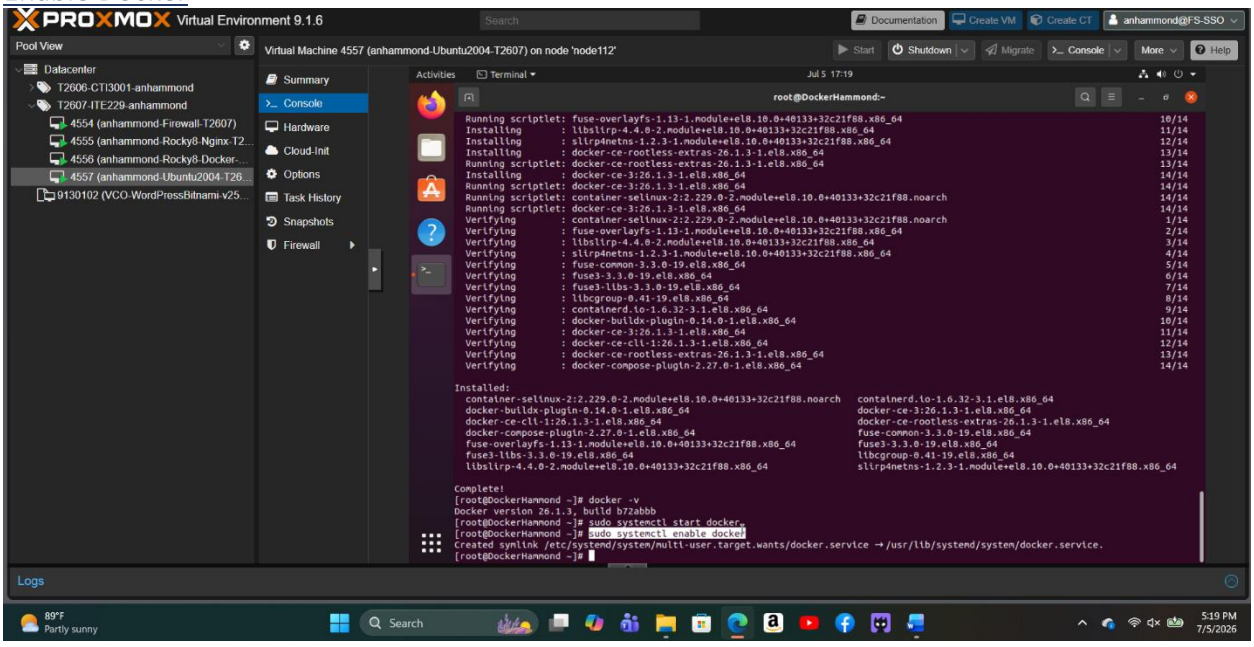


Initialize Docker

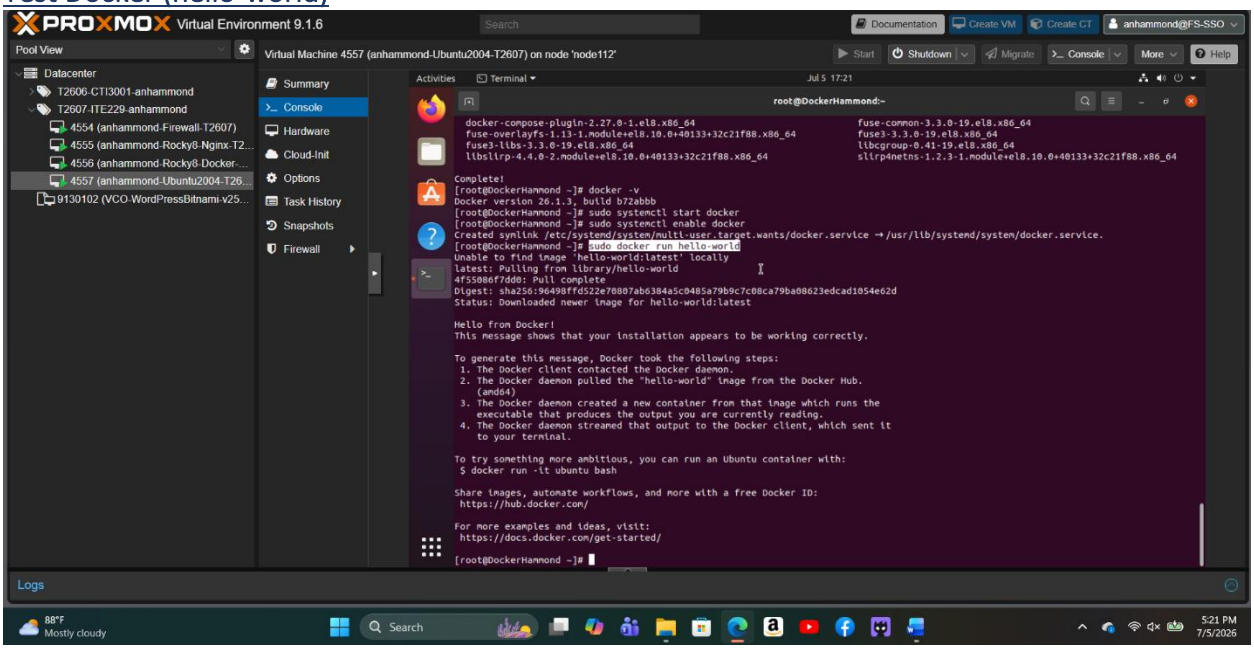
Start Docker



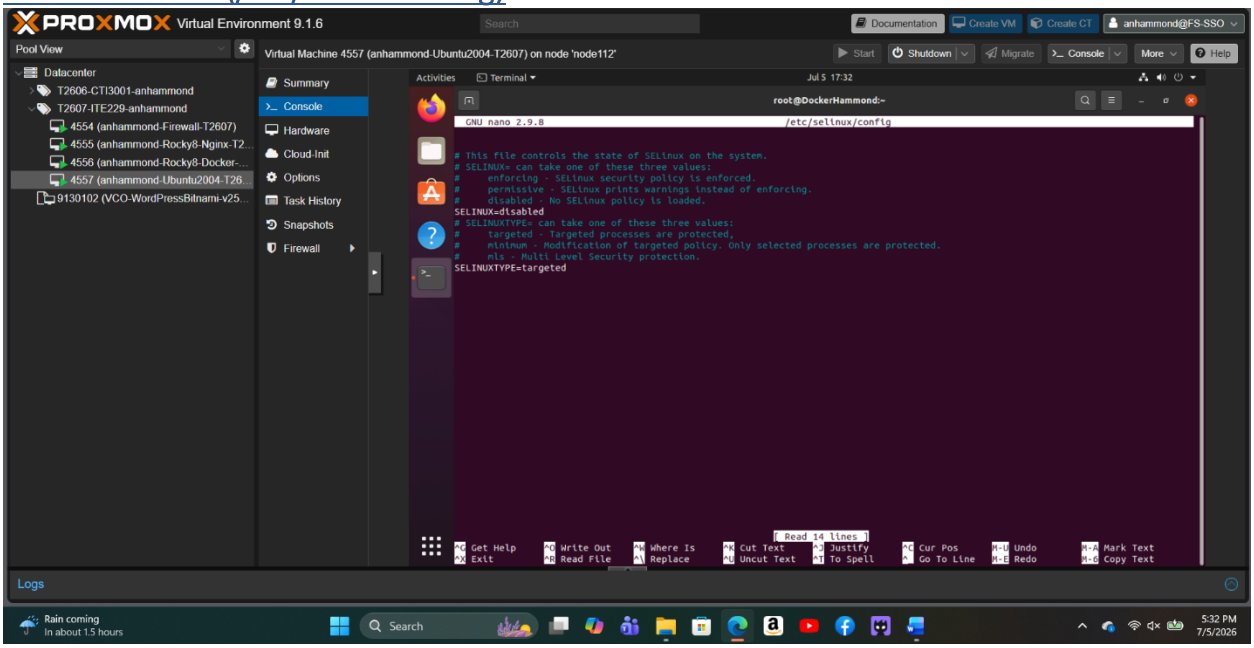
Enable Docker



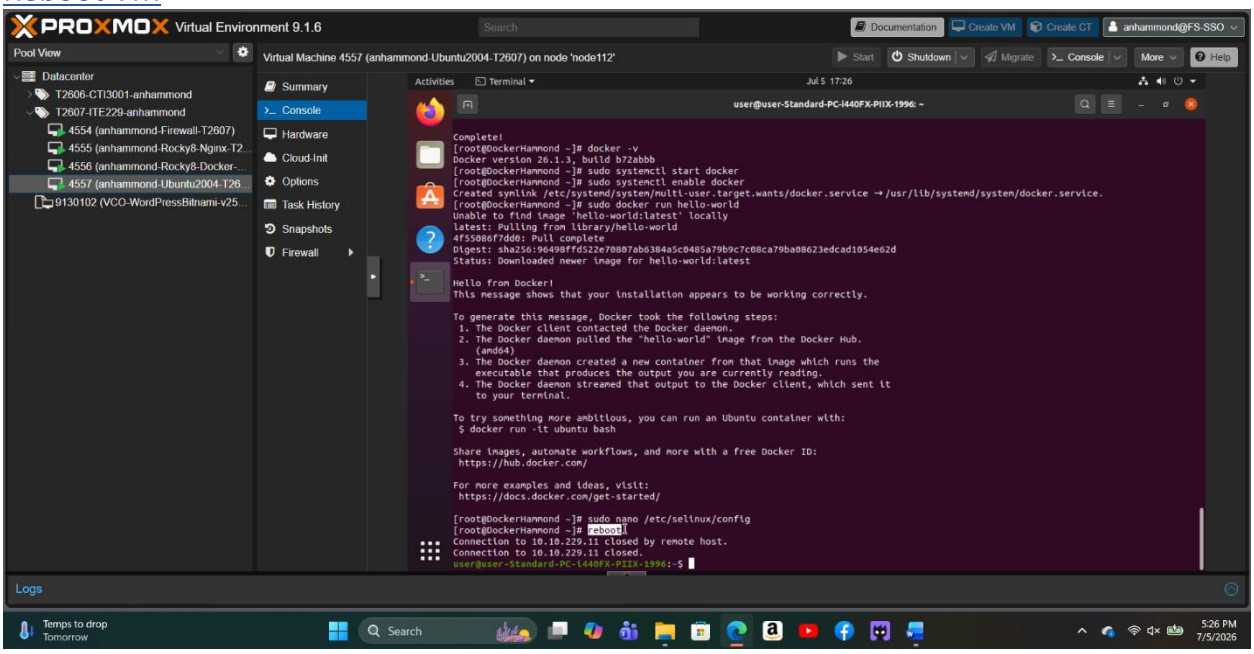
Test Docker (hello-world)



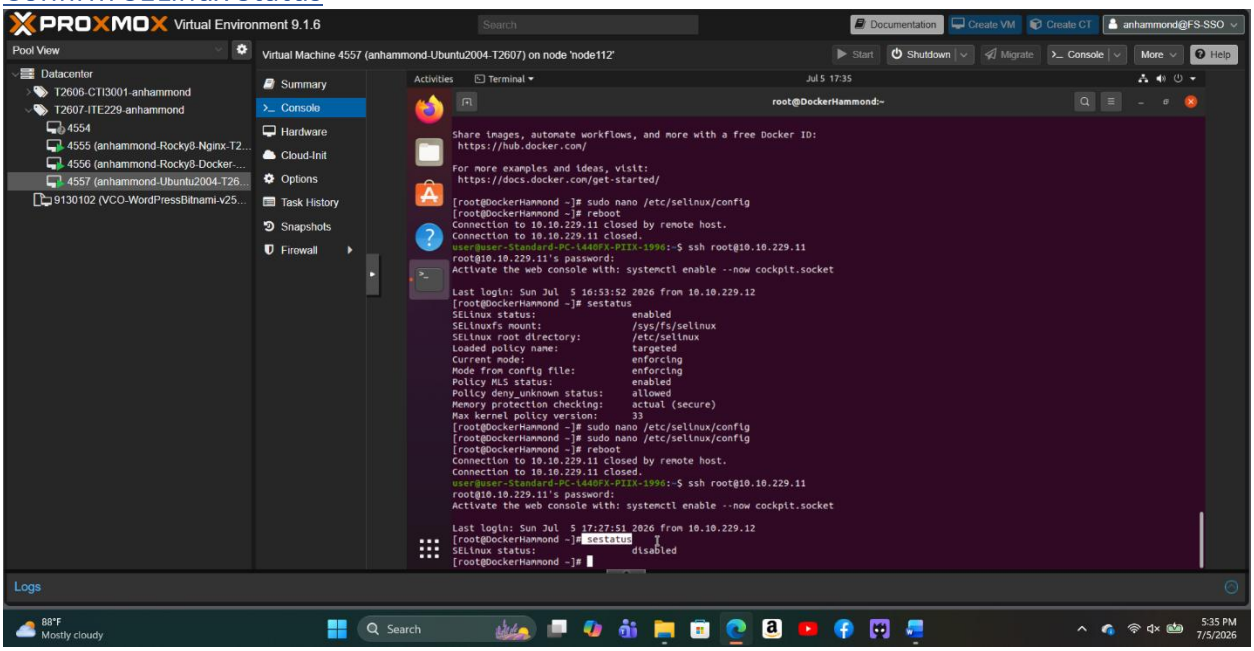
Disable SELinux (/etc/selinux.config)



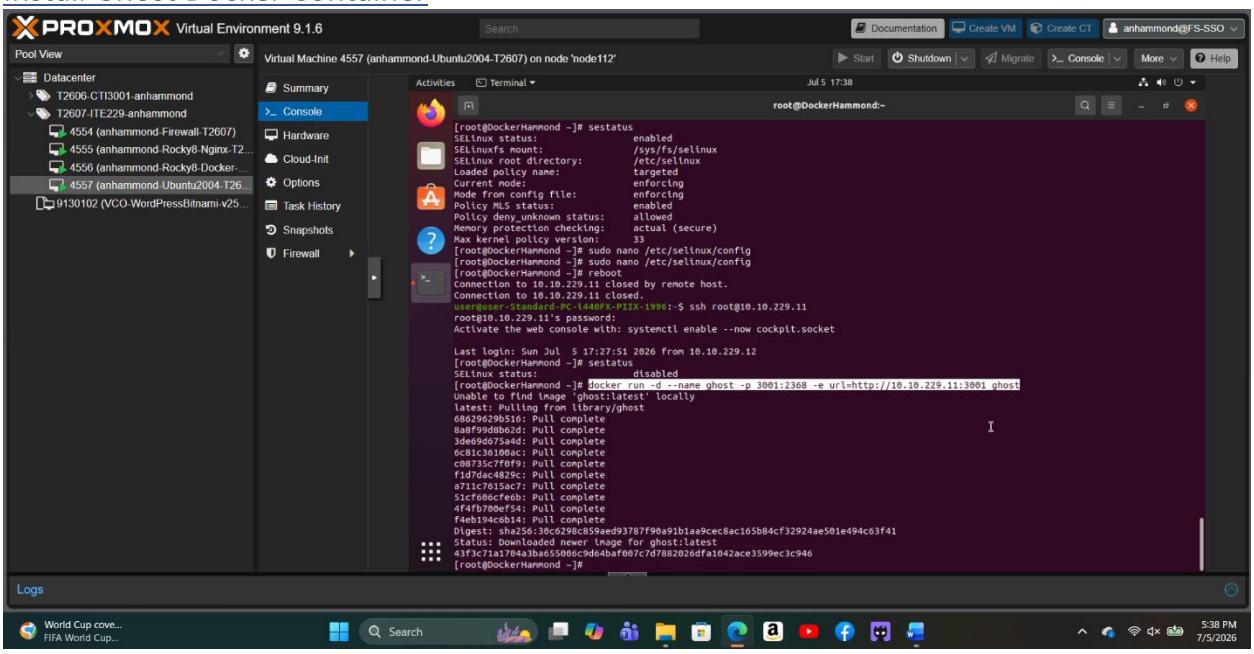
Reboot VM



Confirm SELinux Status

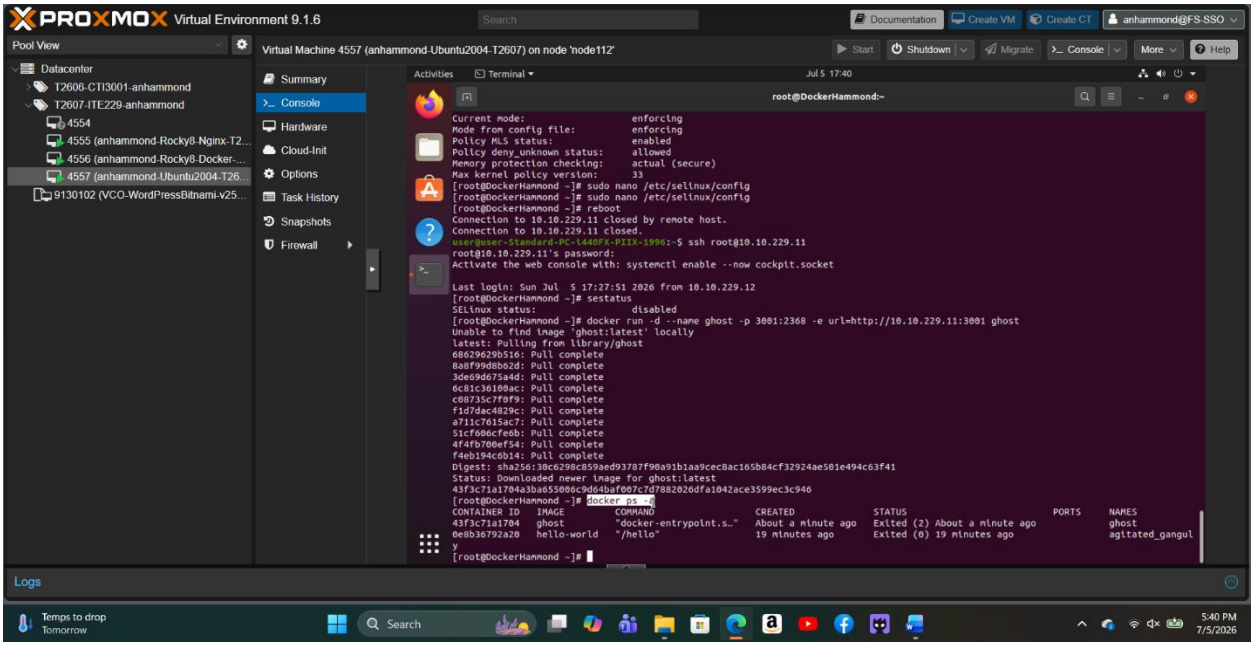


Install Ghost Docker Container



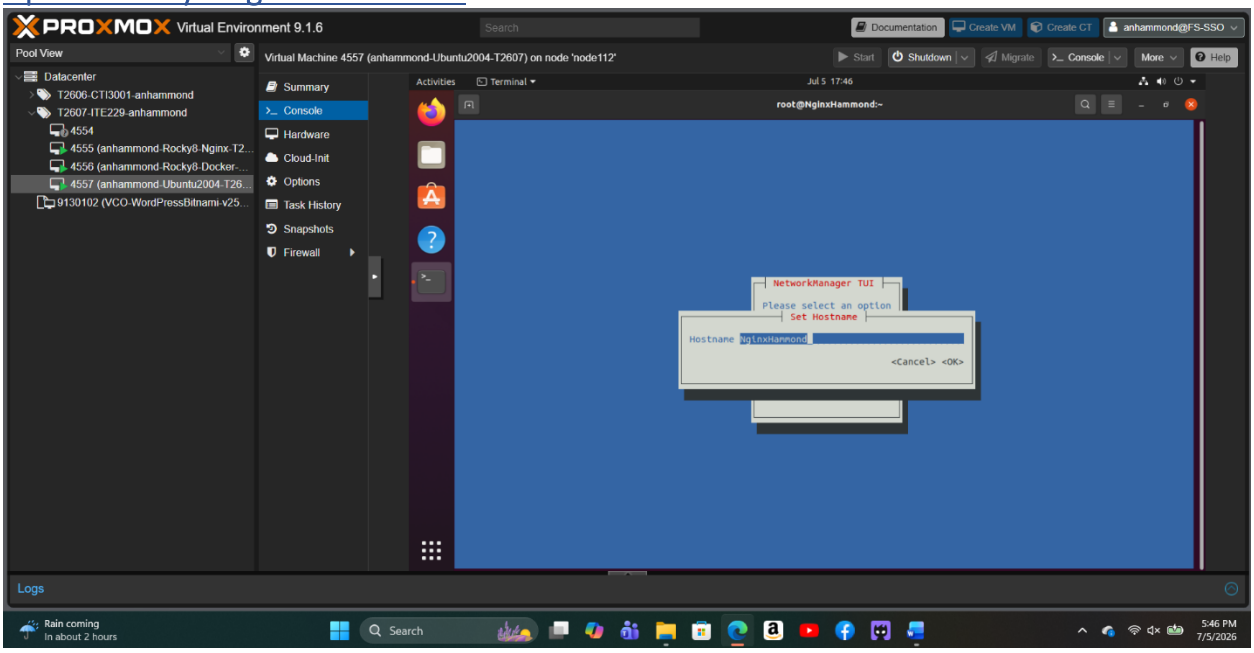
Check for Ghost Container (docker ps)

Ghost Container ID

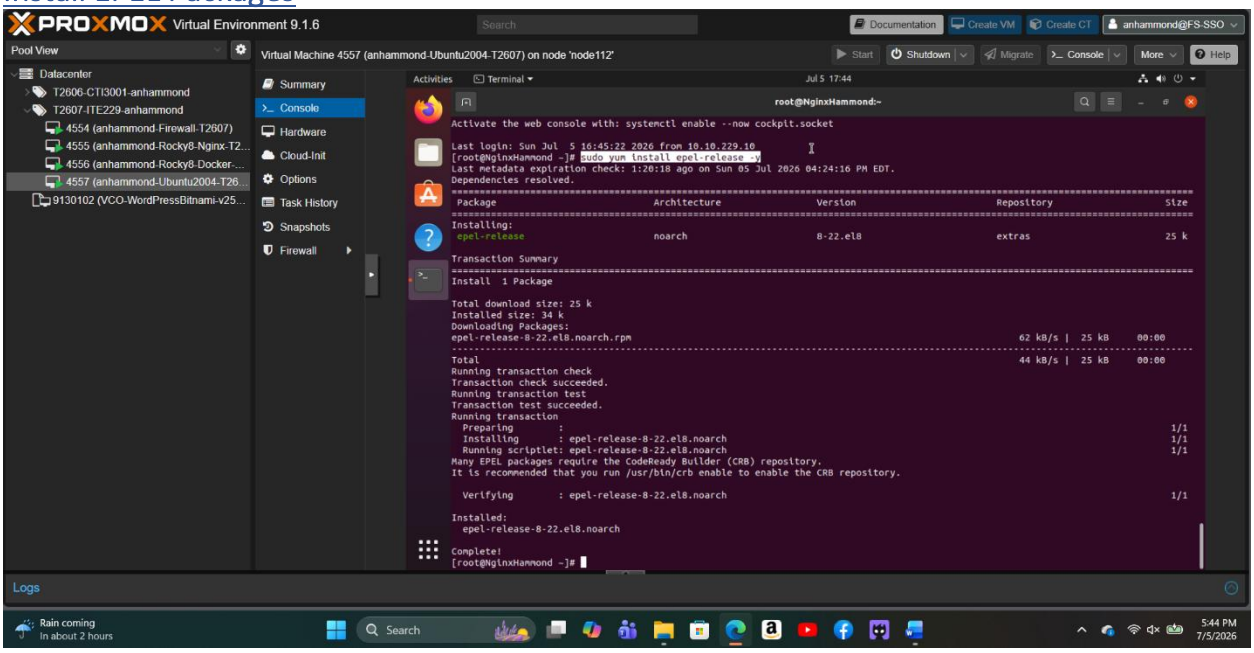


NginX Reverse Proxy

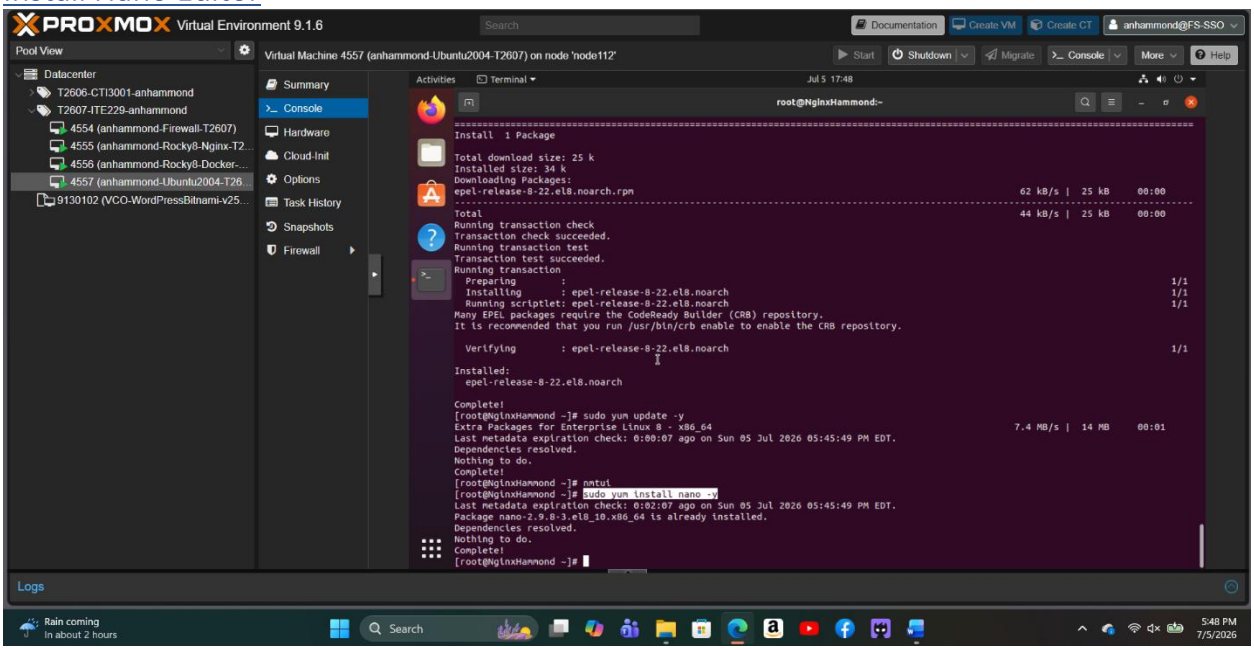
Update Rocky8-Nginx Host Name



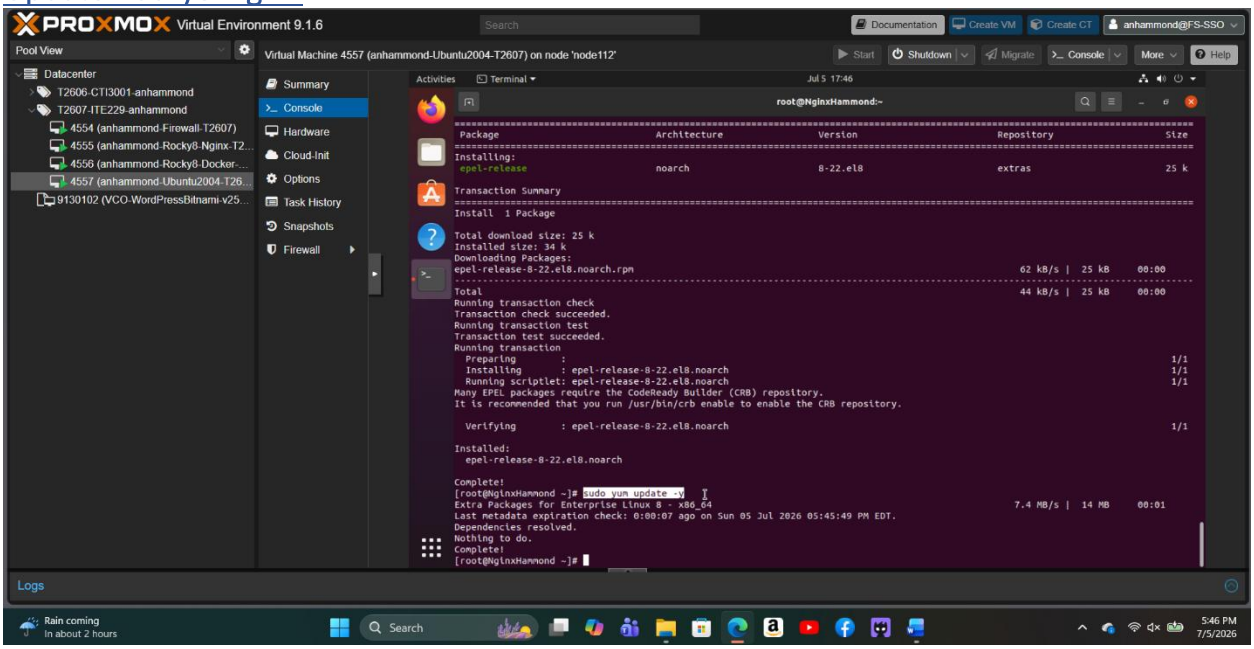
Install EPEL Packages



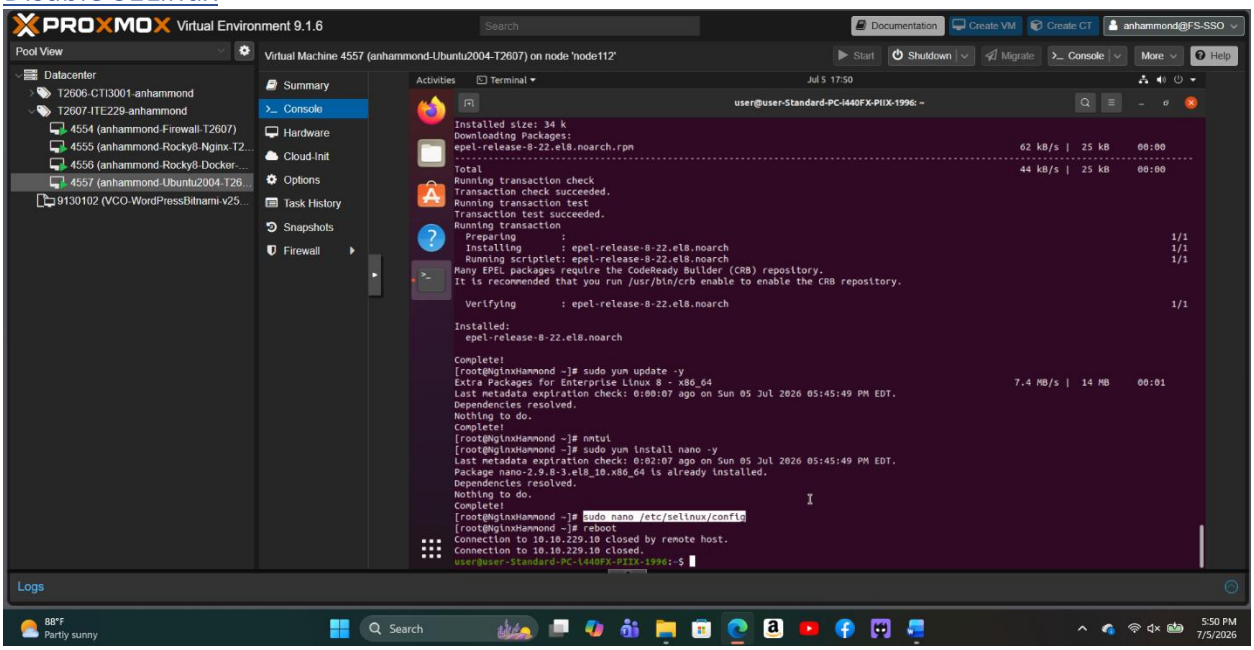
Install Nano Editor



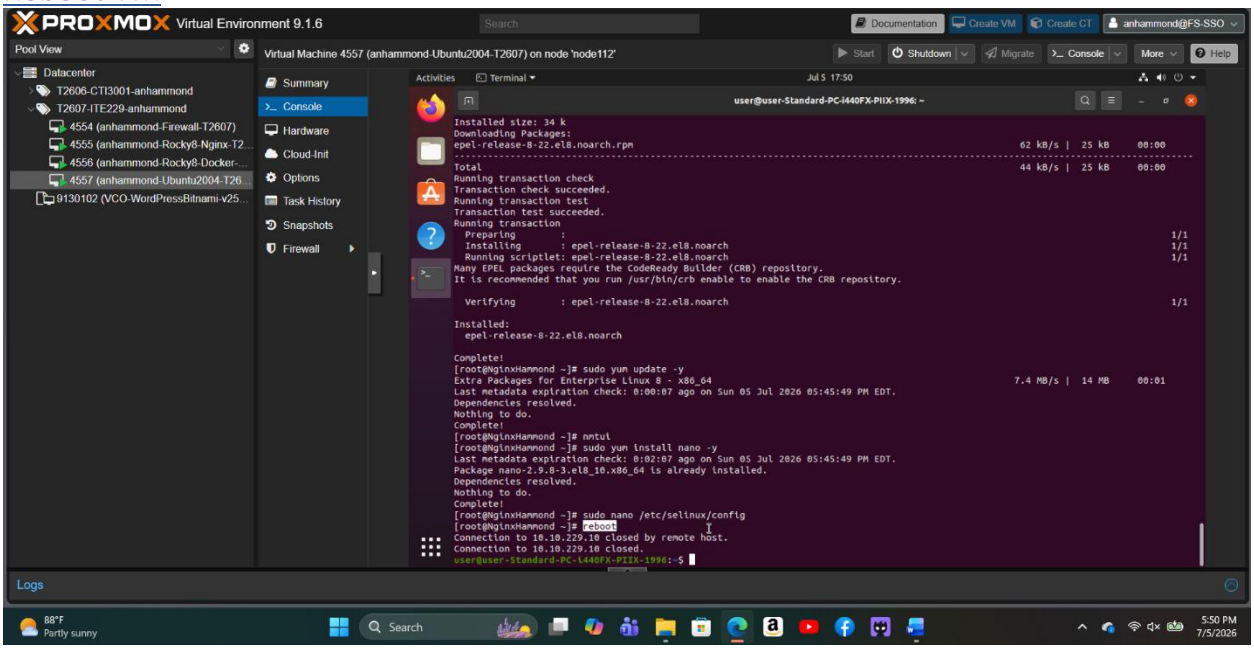
Update Rocky8-Nginx



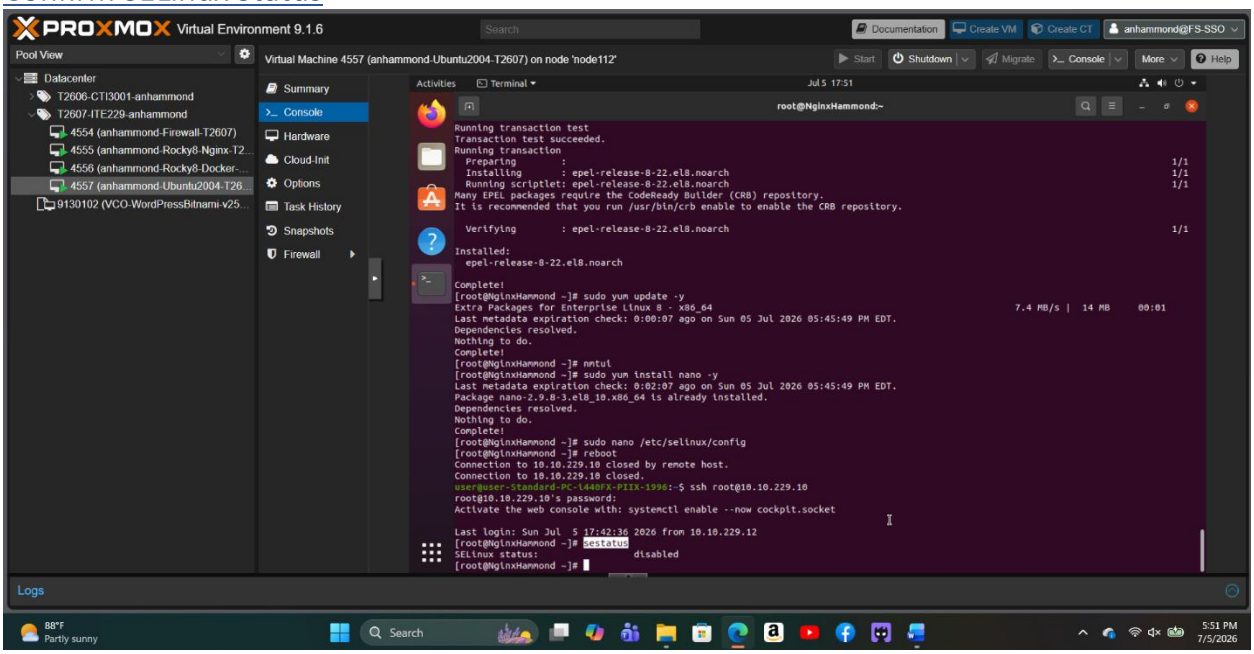
Disable SELinux



Reboot VM

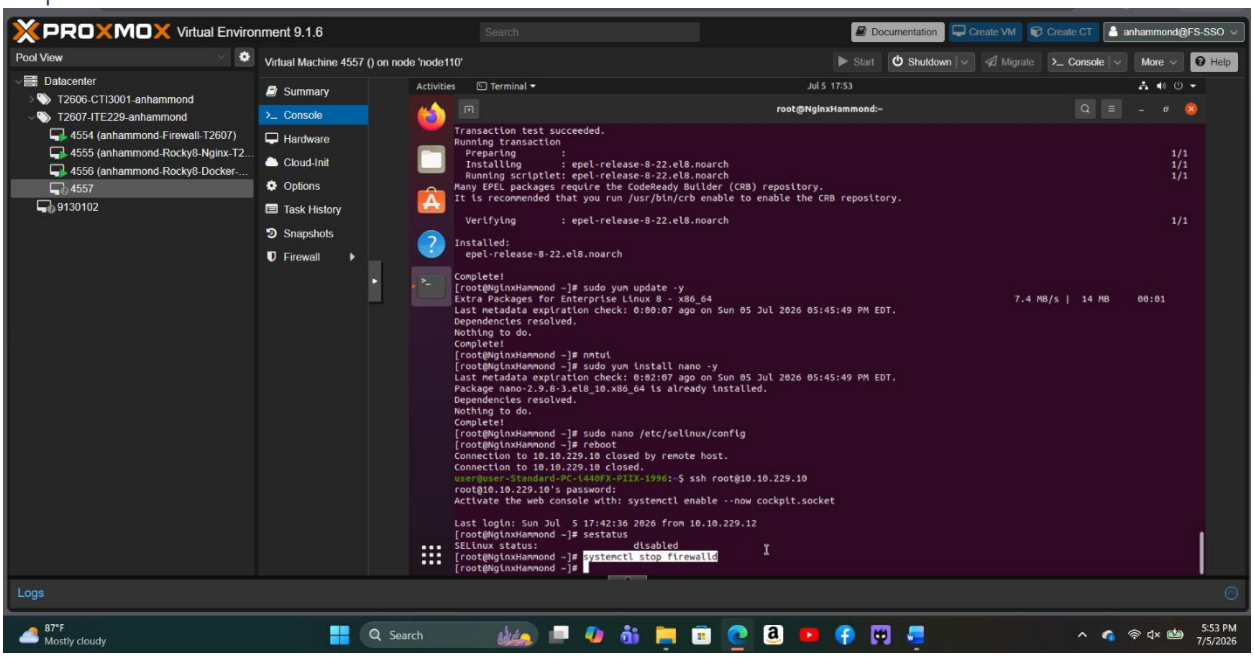


Confirm SELinux Status

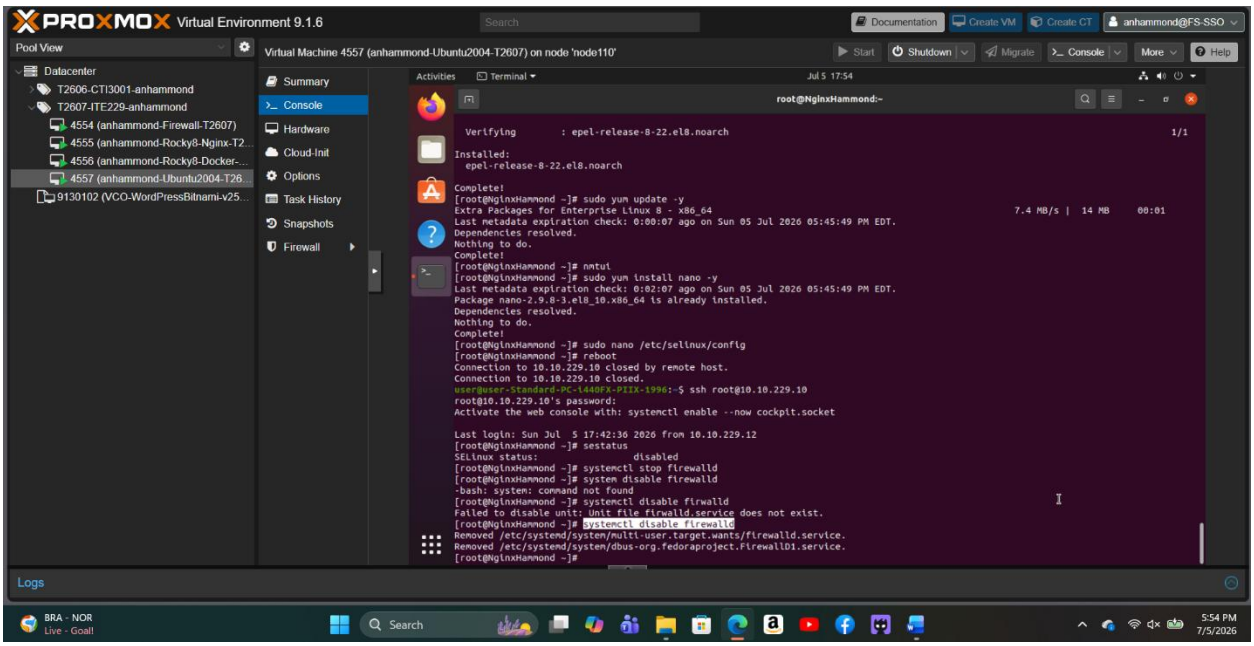


Rocky Firewall

Stop Firewall

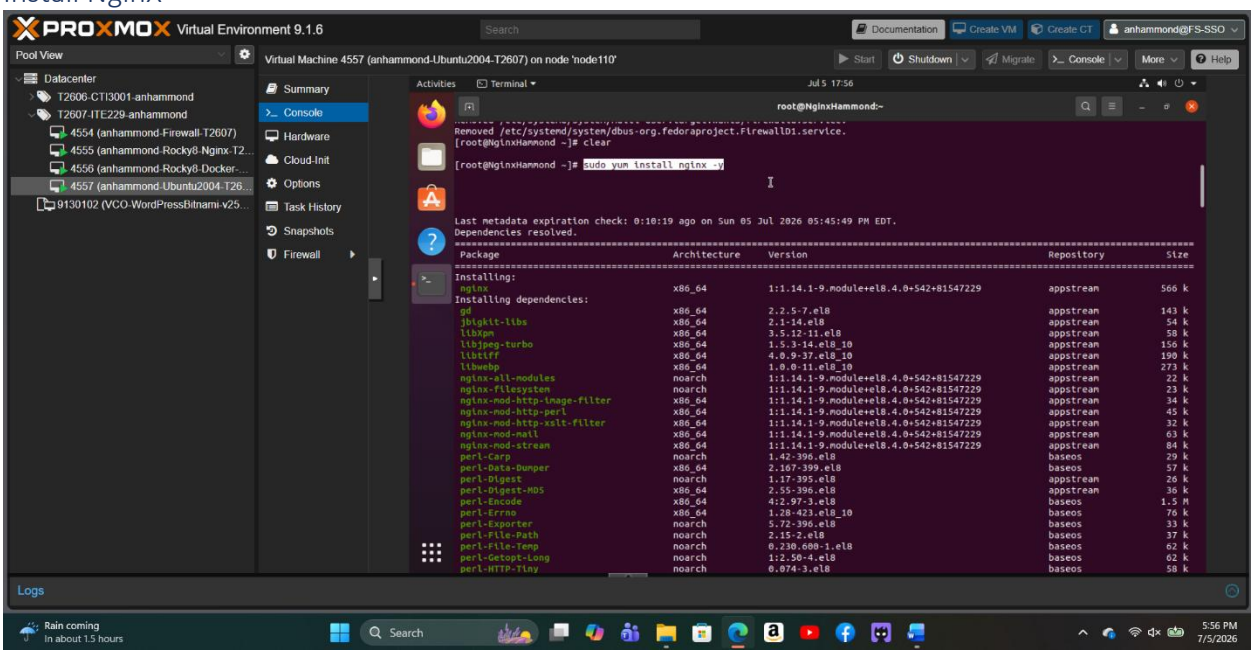


Disable Firewall

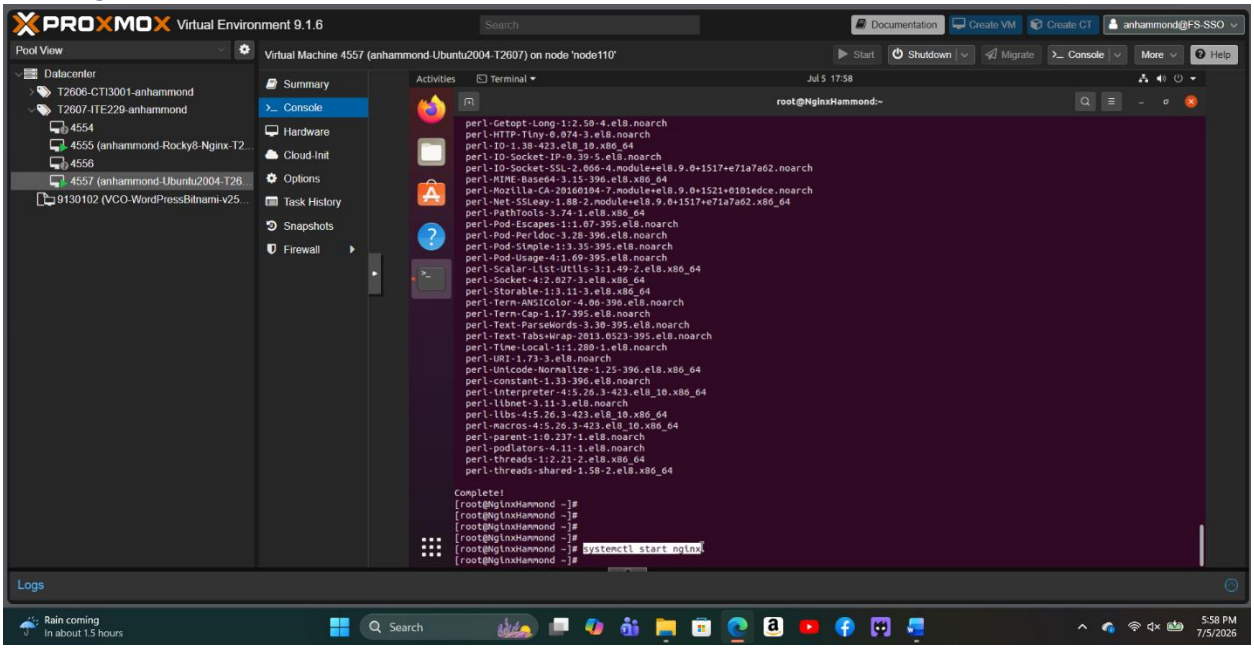


NginX

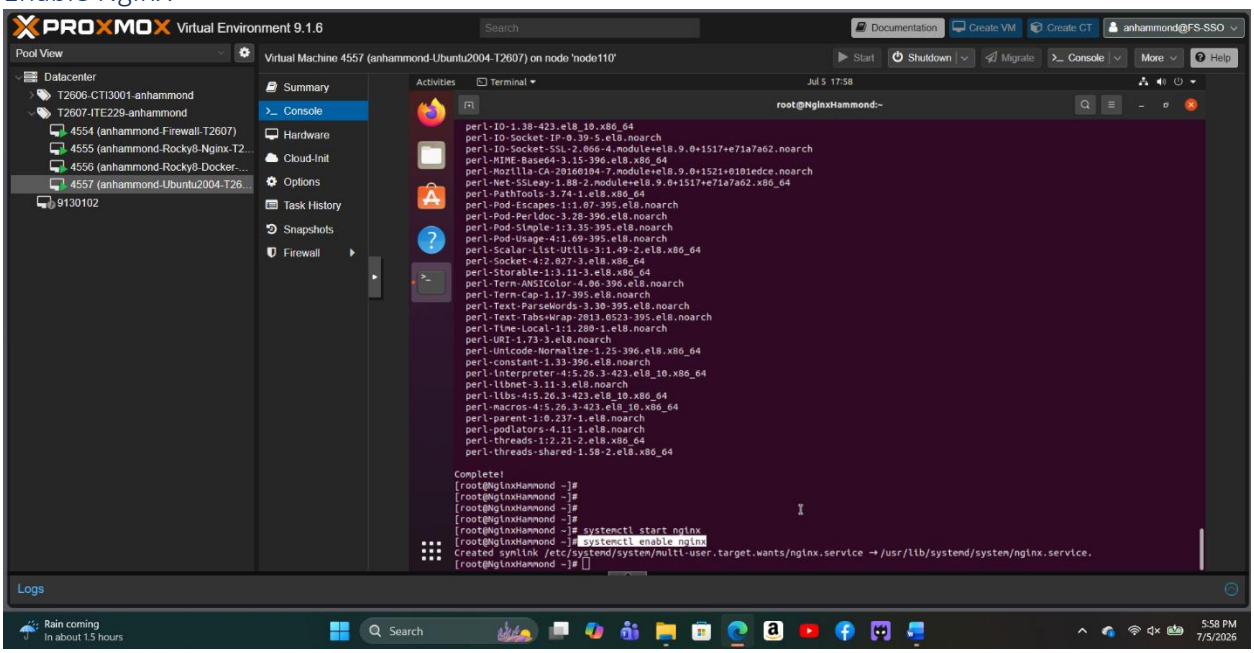
Install NginX



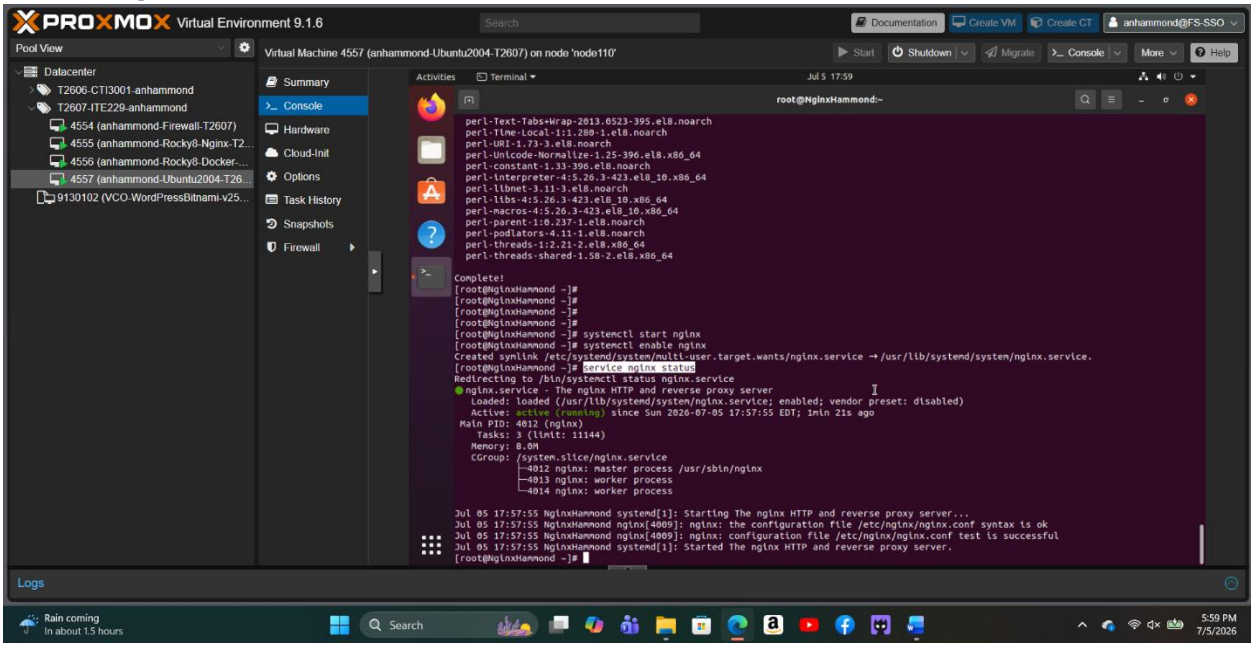
Start NginX



Enable NginX

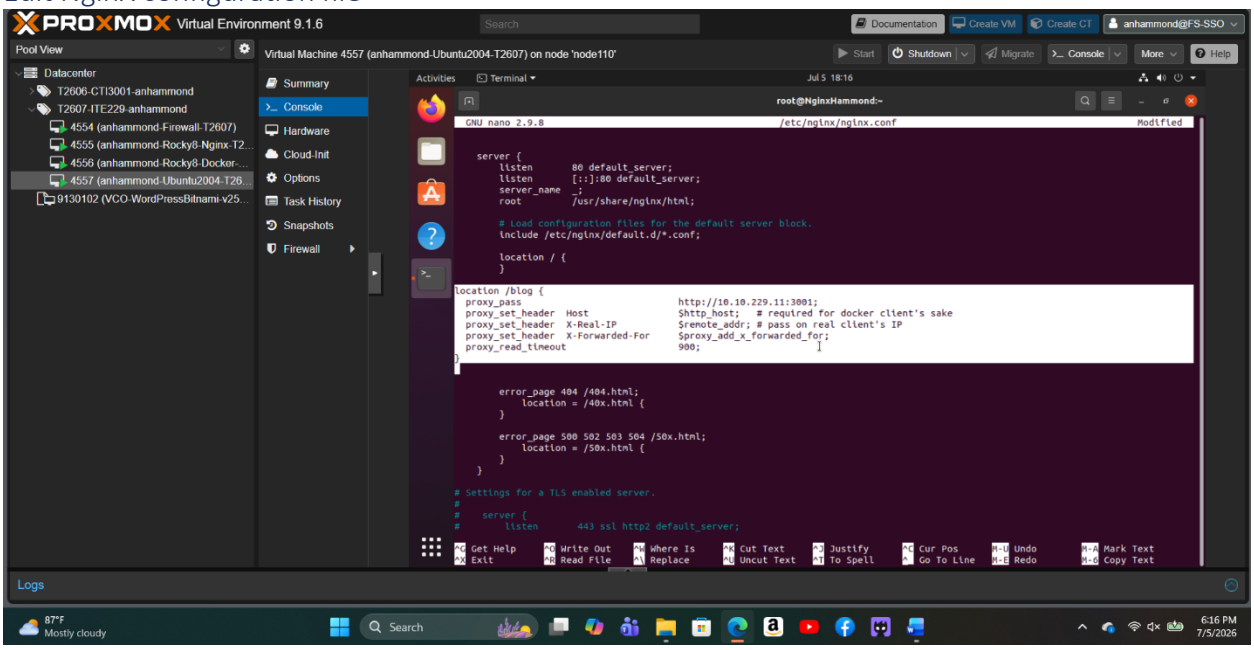


Confirm NginX Status

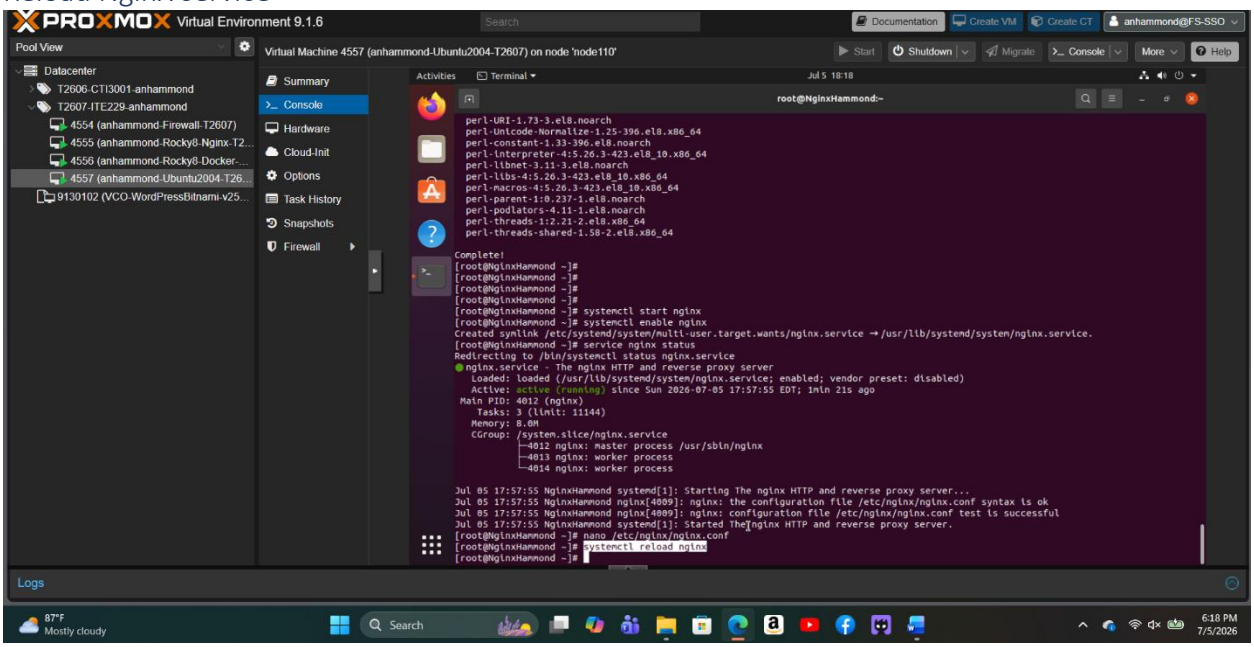


Reverse Proxy for Ghost Site

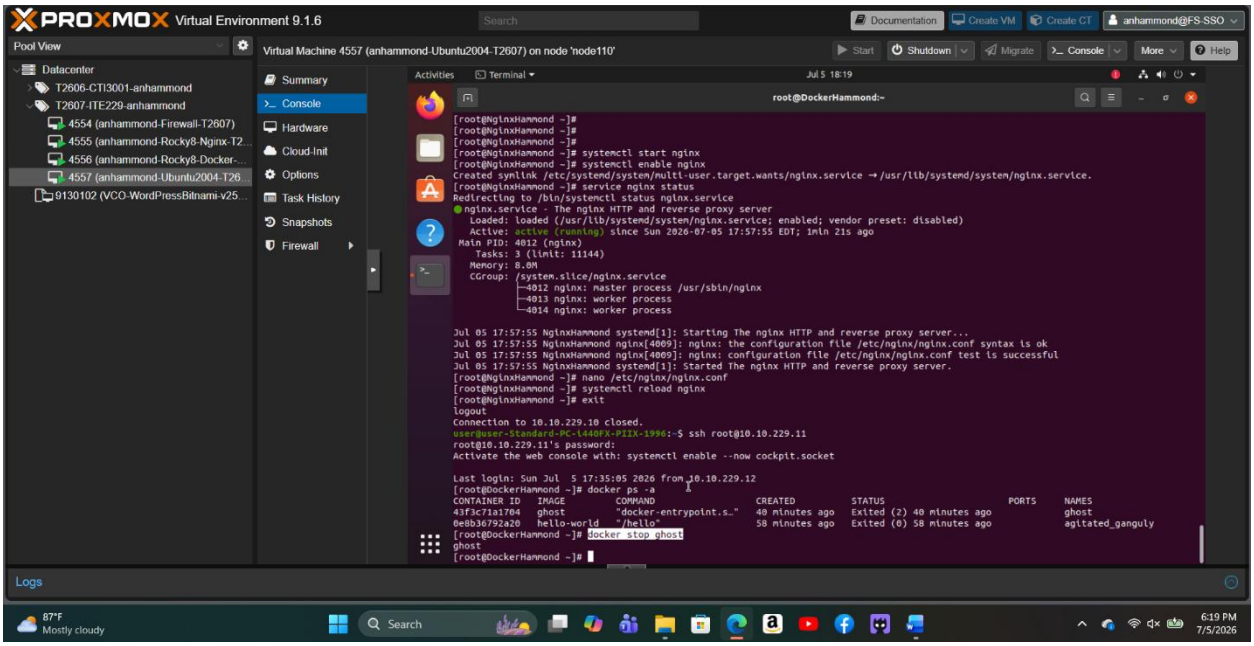
Edit NginX configuration file



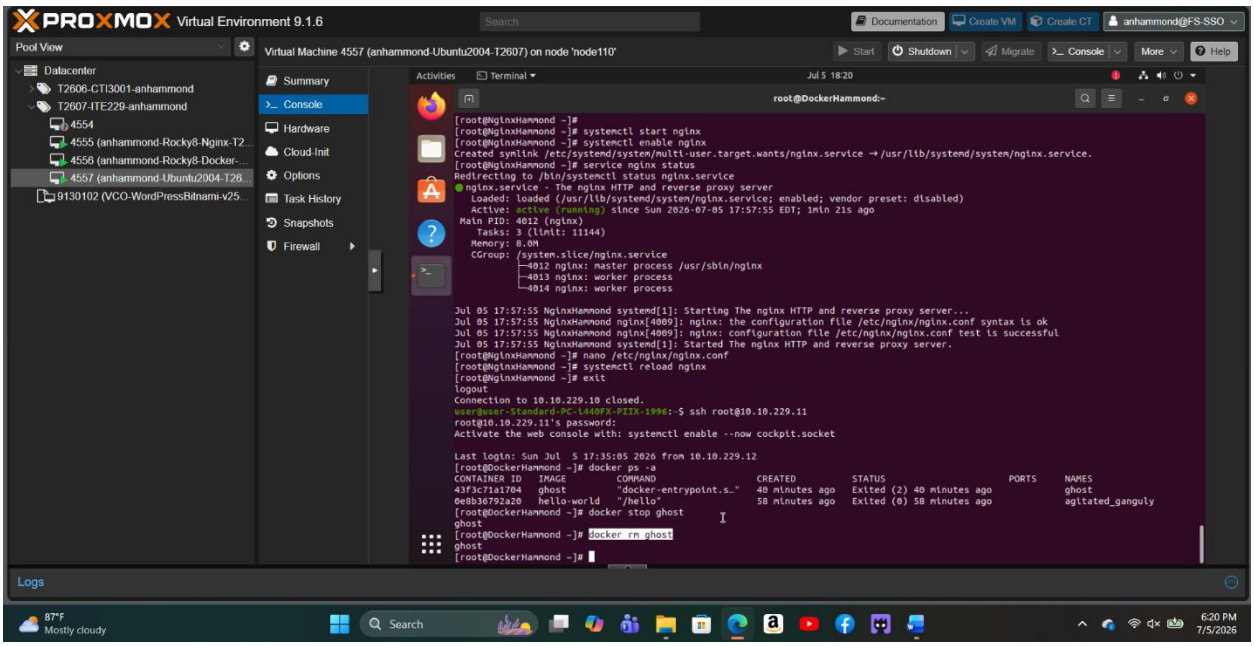
Reload NginX service



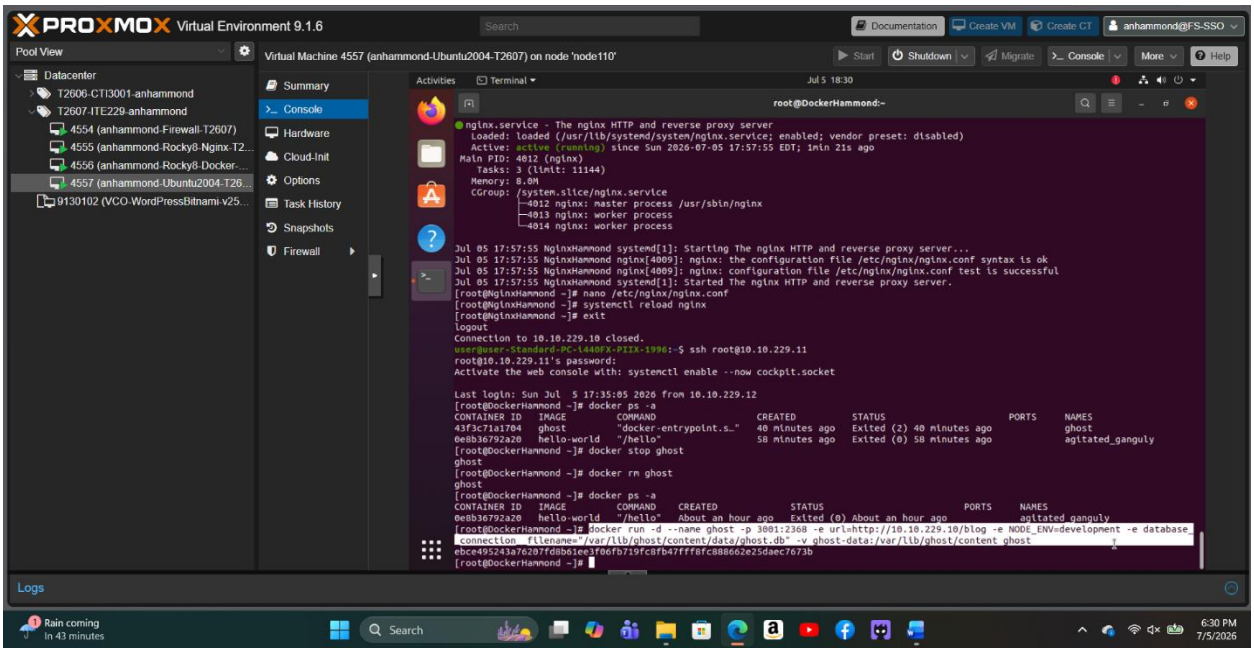
Terminate Docker



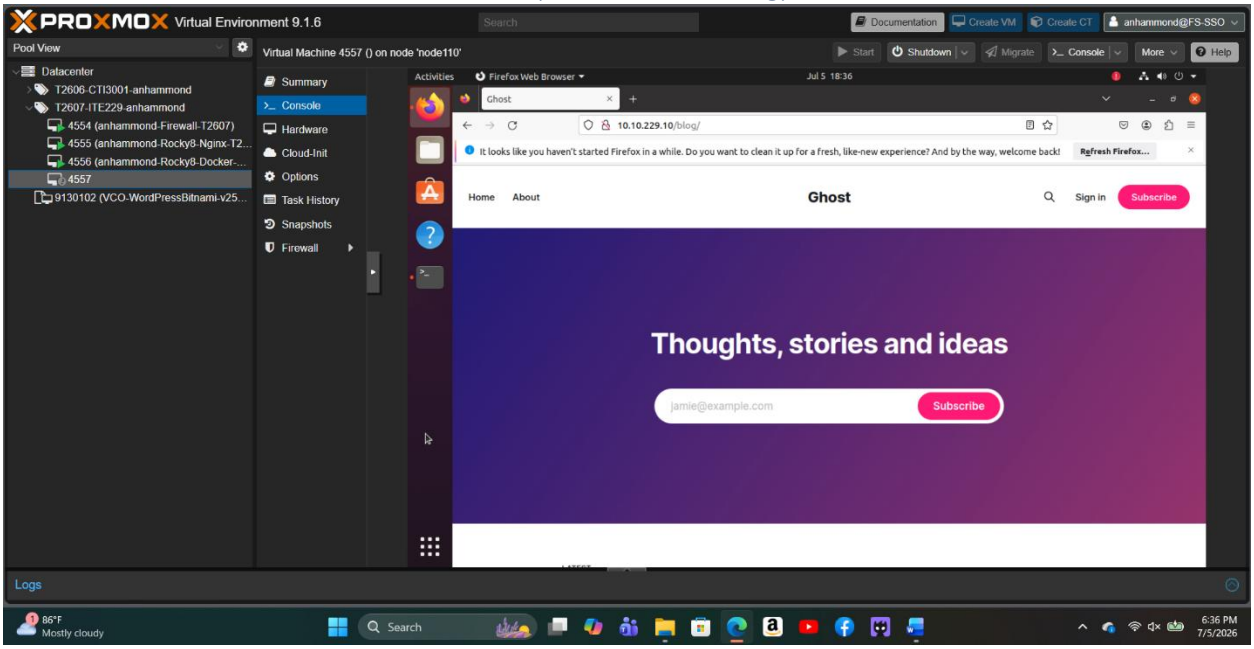
Delete First Ghost Container



Create New Ghost Container



Browse to Ghost from Firefox on Ubuntu (10.10.229.10/blog)

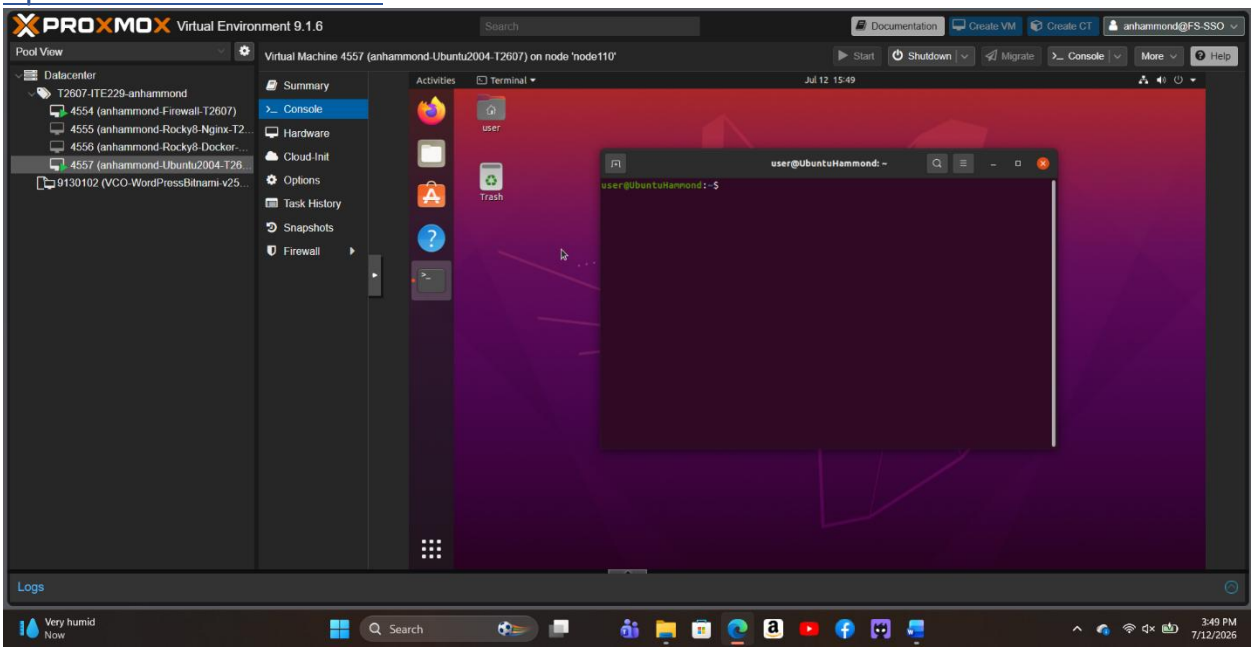


Please remember to complete Appendicies A and B at the end of this document for milestone 1 after you have completed the above steps. Use the "tail" command on your reverse proxy VM. You may need to research this command.

END OF MILESTONE 1

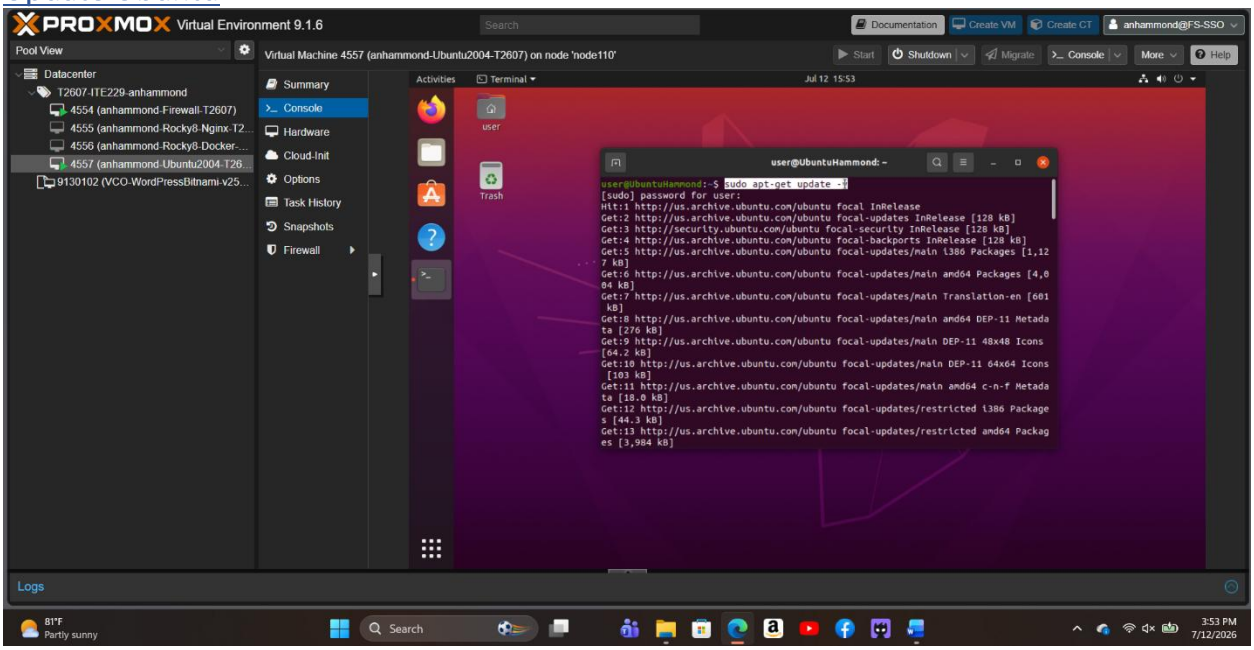
WordPress on Ubuntu - LAMP Stack

Update Ubuntu Host Name



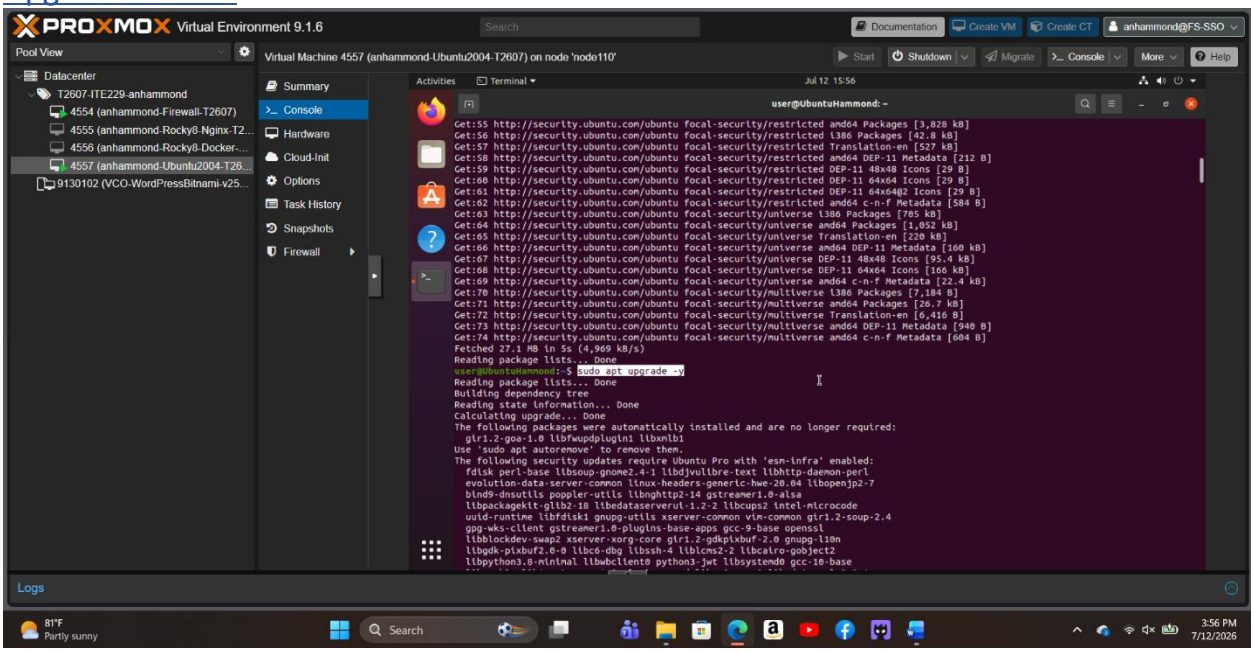
Used the “nmtui” command to change the host name. It’s unique and recognizable on a network.

Update Ubuntu



This step updates the Ubuntu OS and installed software packages to the latest available version.

Upgrade Ubuntu

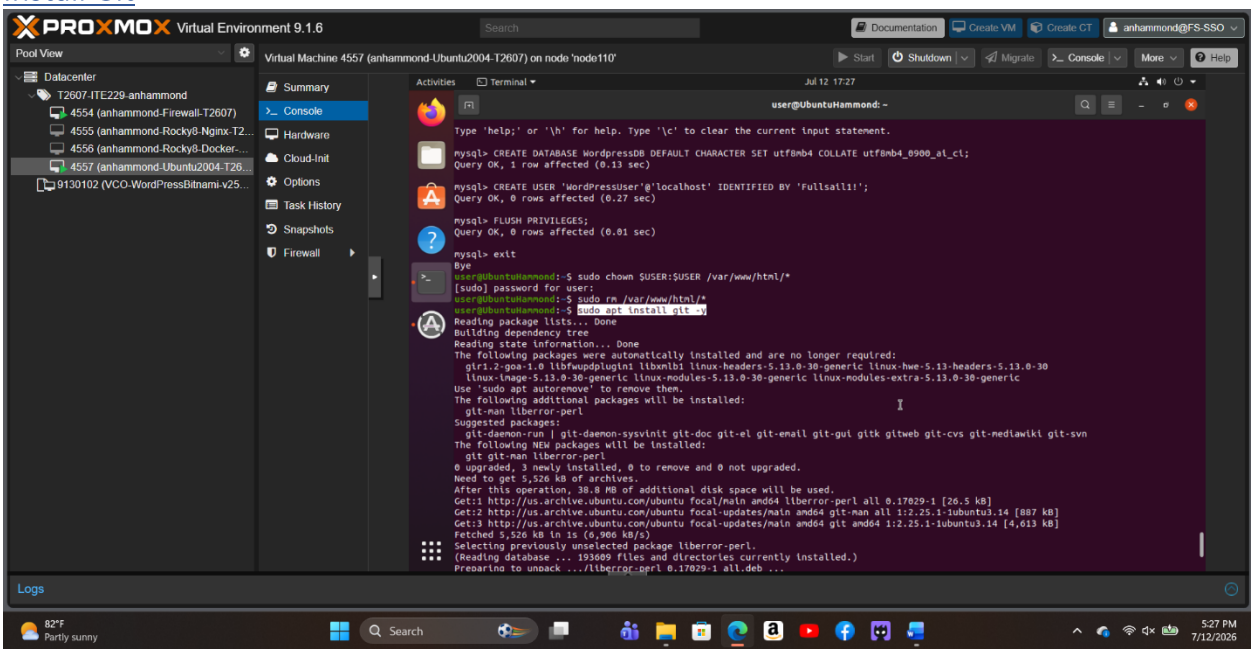


This step upgrades all installed Ubuntu packages to their latest available version.

Install Nano Editor

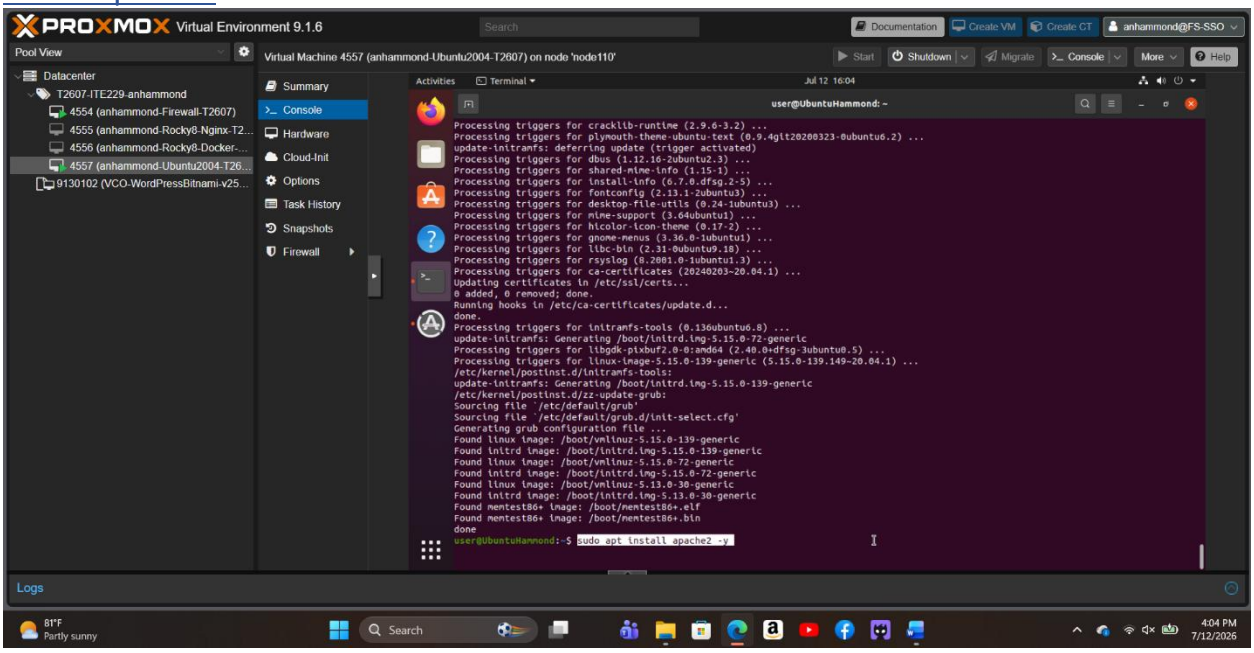
Legible, annotated screenshots AND written instructions/commands required

Install Git



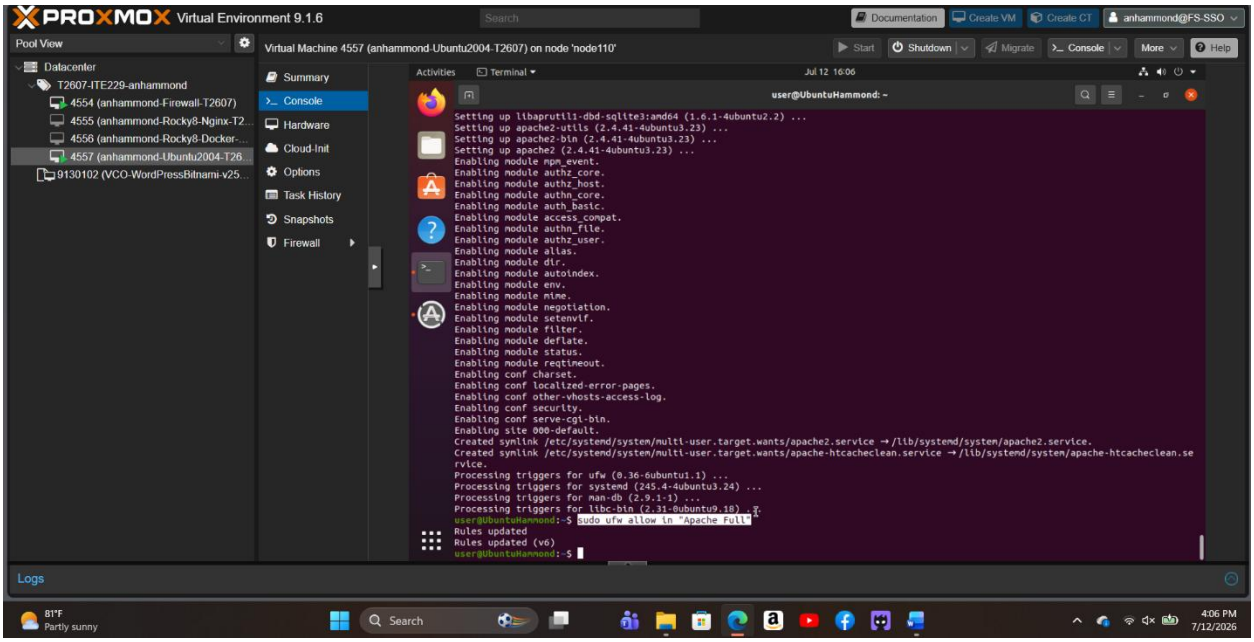
This step installs Git, a version contrl system used to track changes in files and manage software development projects.

Install Apache2



This step installs Apache2, a popular web server used to host websites and web applications.

Open Firewall Ports 80 and 443

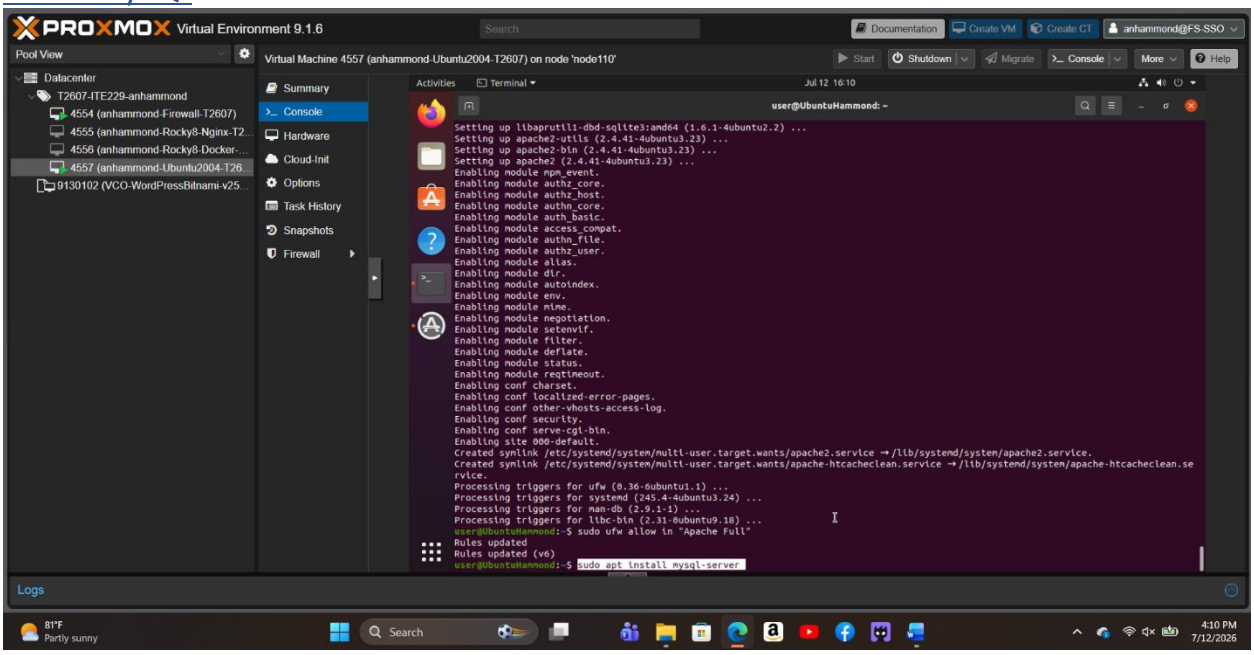


This step configures the Ubuntu firewall to allow incoming web traffic on port 80 and 443.

Browse to Apache2 Ubuntu Default Page

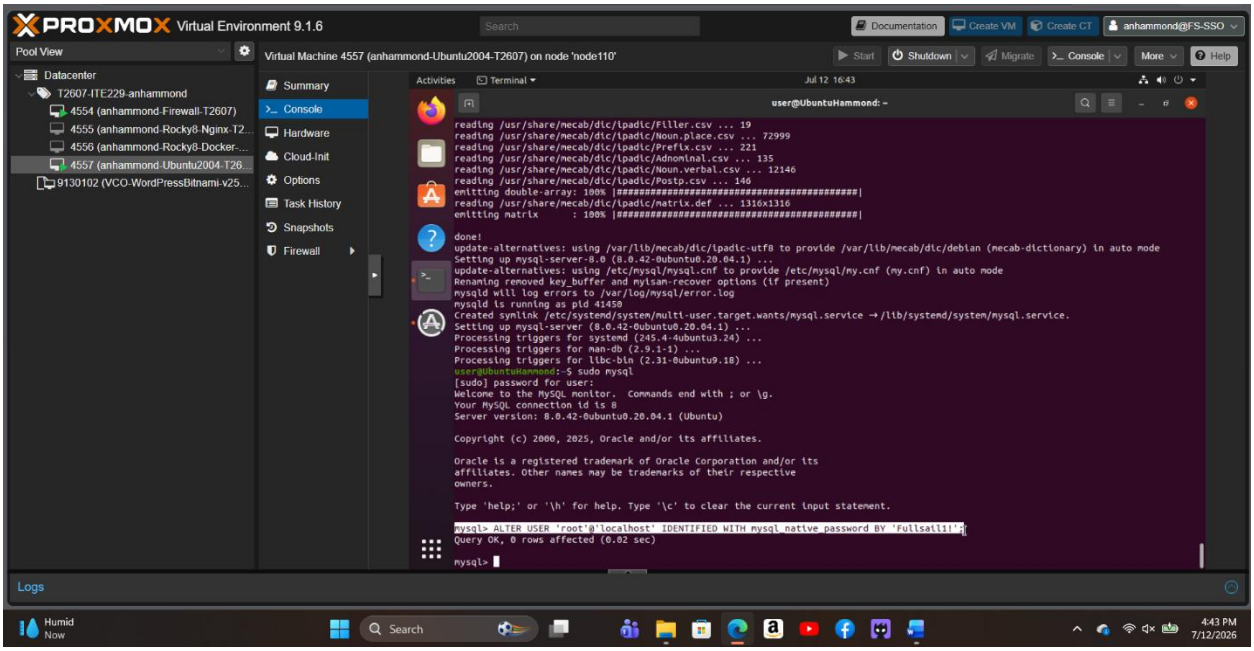
Legible, annotated screenshots AND written instructions/commands required

Install MySQL



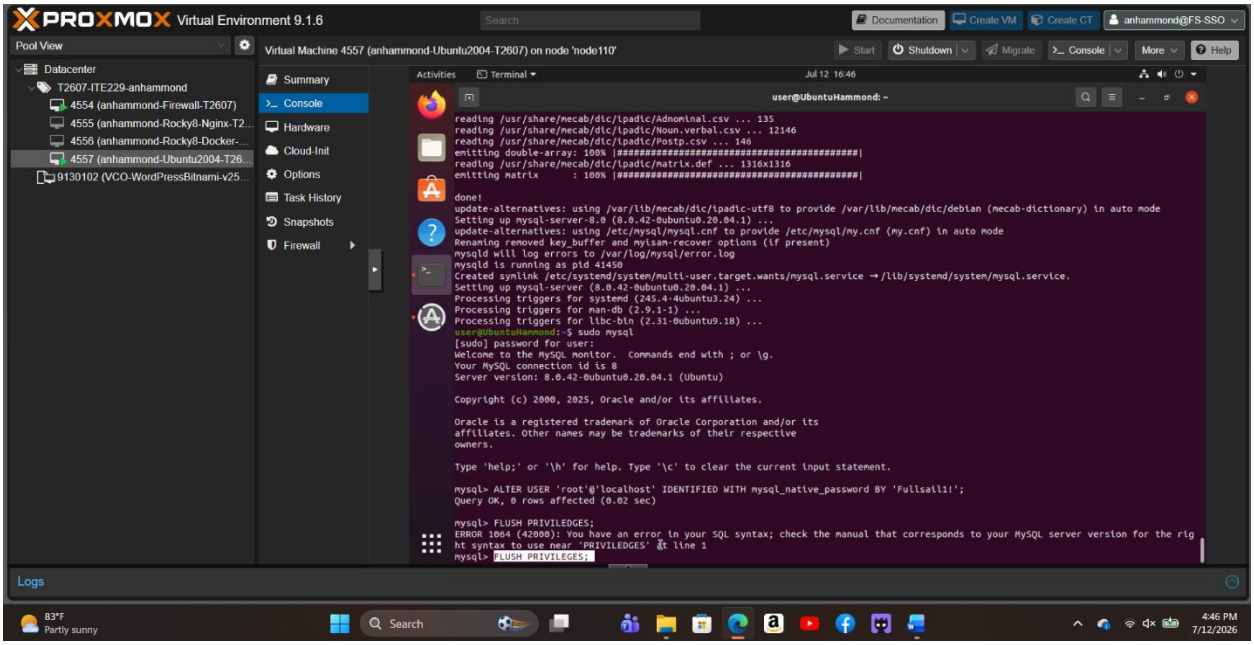
This step installs MySQL, a relational database management system used to store and manage websites and application data.

Alter root user password (root@localhost)



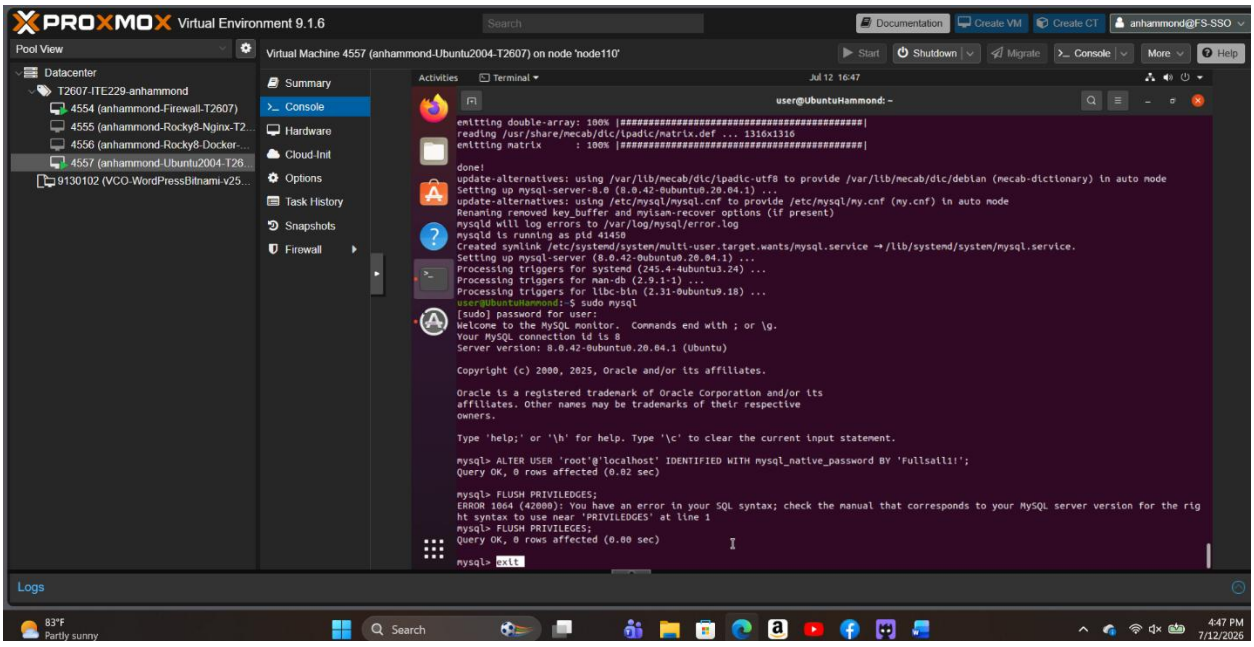
This step secures the MySQL database server by configuring a password for the root user account.

Flush Privileges



This step refreshes MySQL's privilege tables so that any changes made to user accounts, passwords, or permissions take effect immediately.

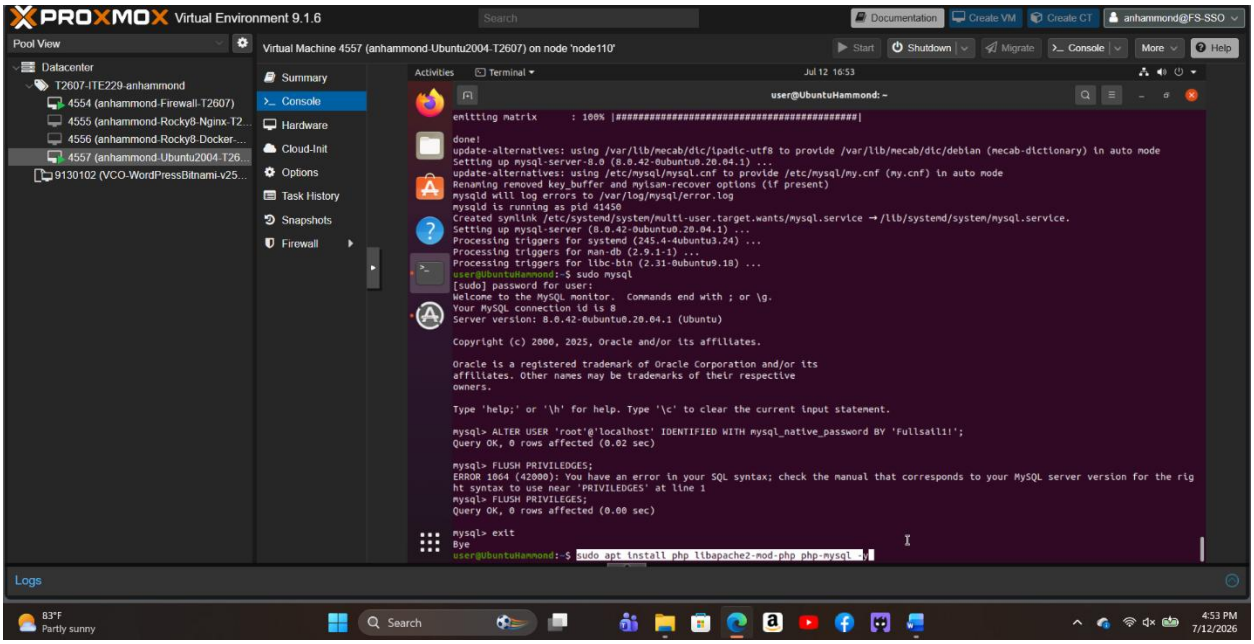
Exit MySQL



This step exits the MySQL command-line interface and returns the user to the Ubuntu terminal.

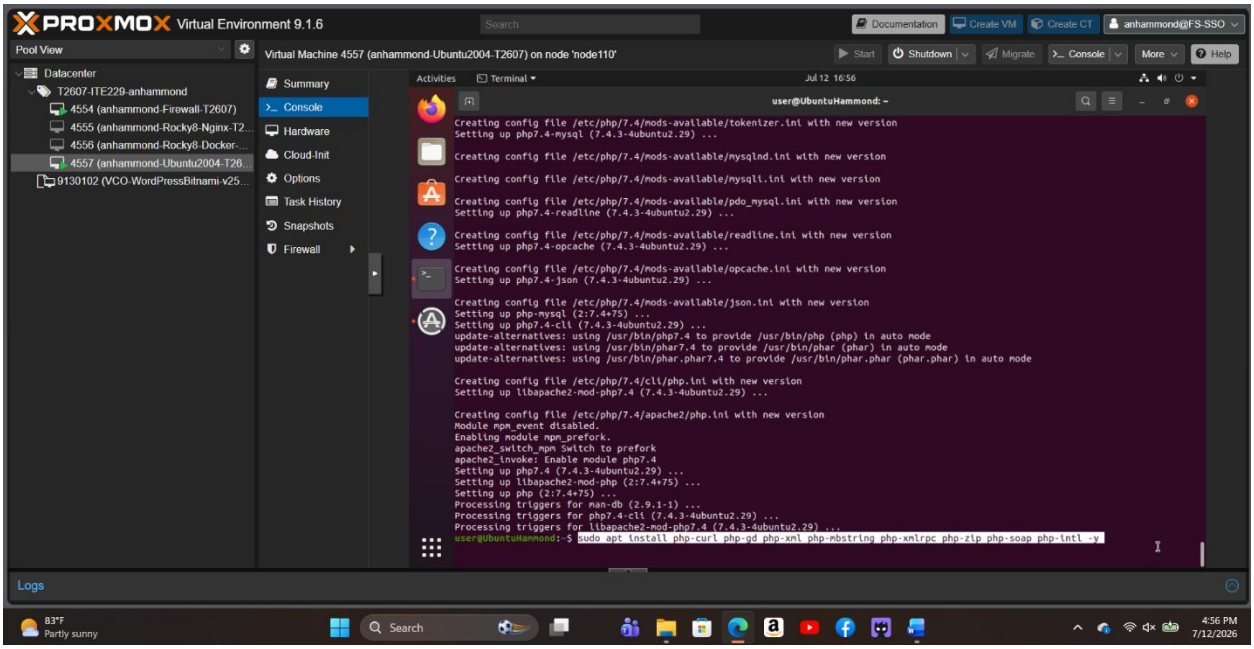
Install PHP

Install Required PHP Libraries



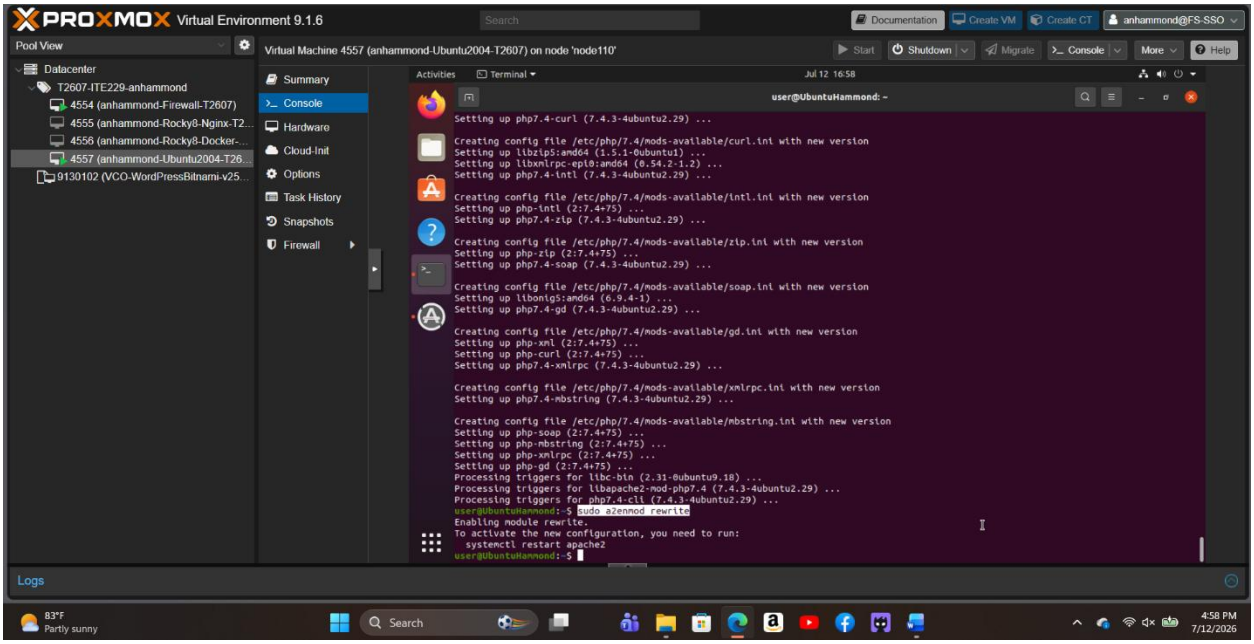
This step installs PHP and the additional libraries needed for Wordpress to function properly.

Install Required MySQL Libraries



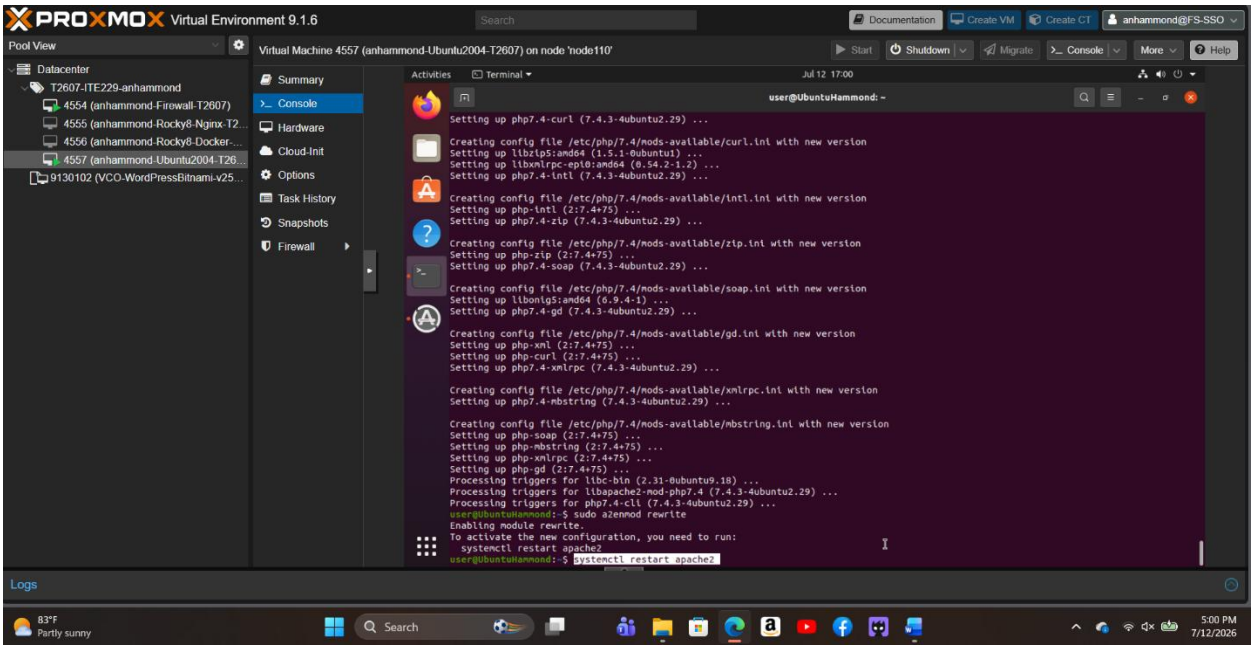
This step installs the MySQL libraries that allow PHP and WordPress to communicate with the database server.

Enable URL Rewrites (clean URLs)



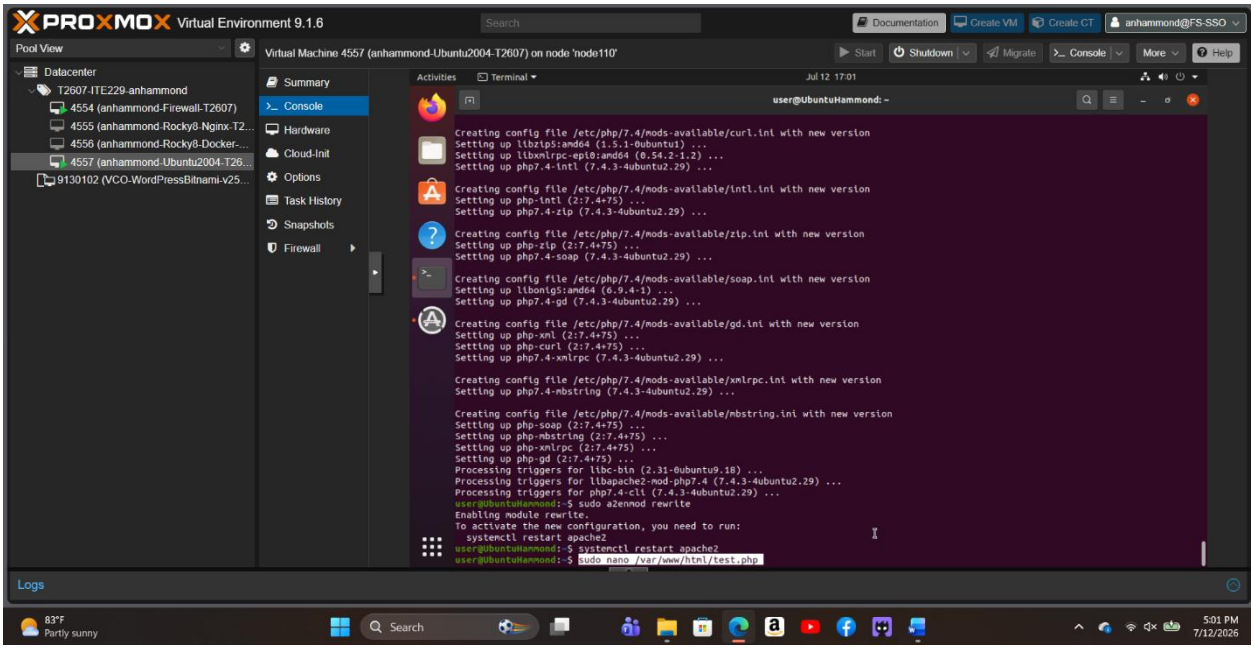
This step enables Apache's URL rewrite module, which allows WP to create clean and user-friendly URLs.

Restart Apache Service



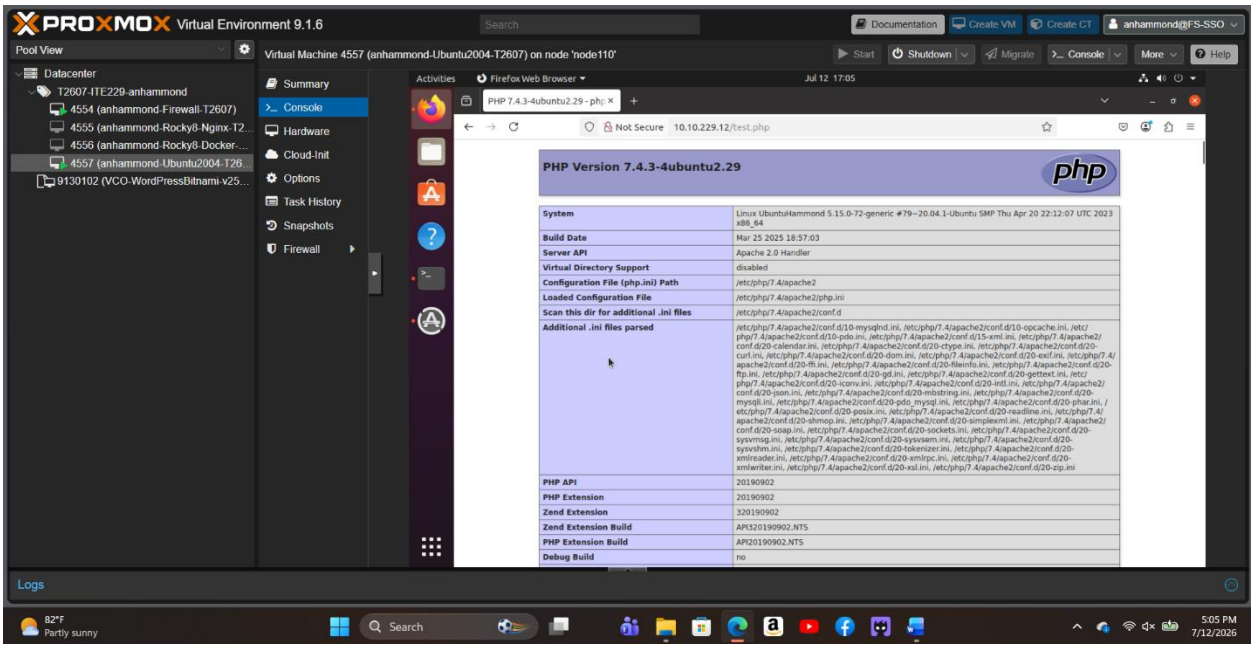
This step restarts the Apache web server to apply any configuration changes that were made during the installation process.

Create a test.php Web Page



This step creates a simple PHP test page to verify that PHP is installed and working correctly with the Apache web server.

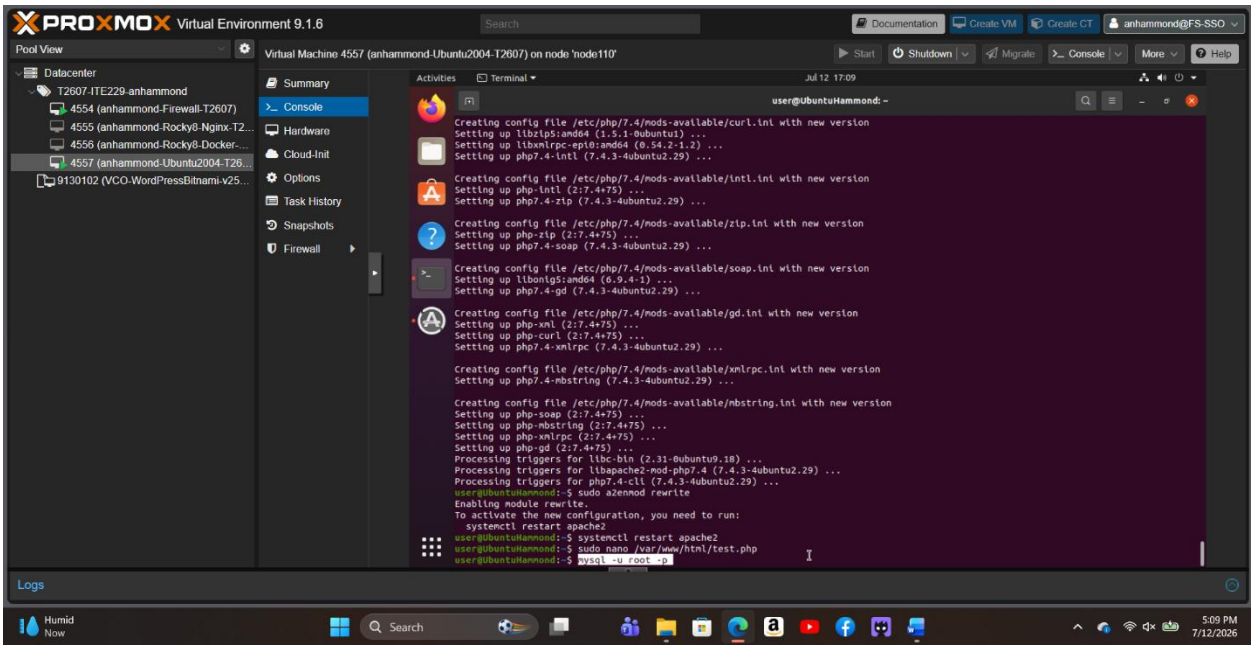
Test the test.php Web Page



This step verifies that the Apache web server and PHP are working correctly together.

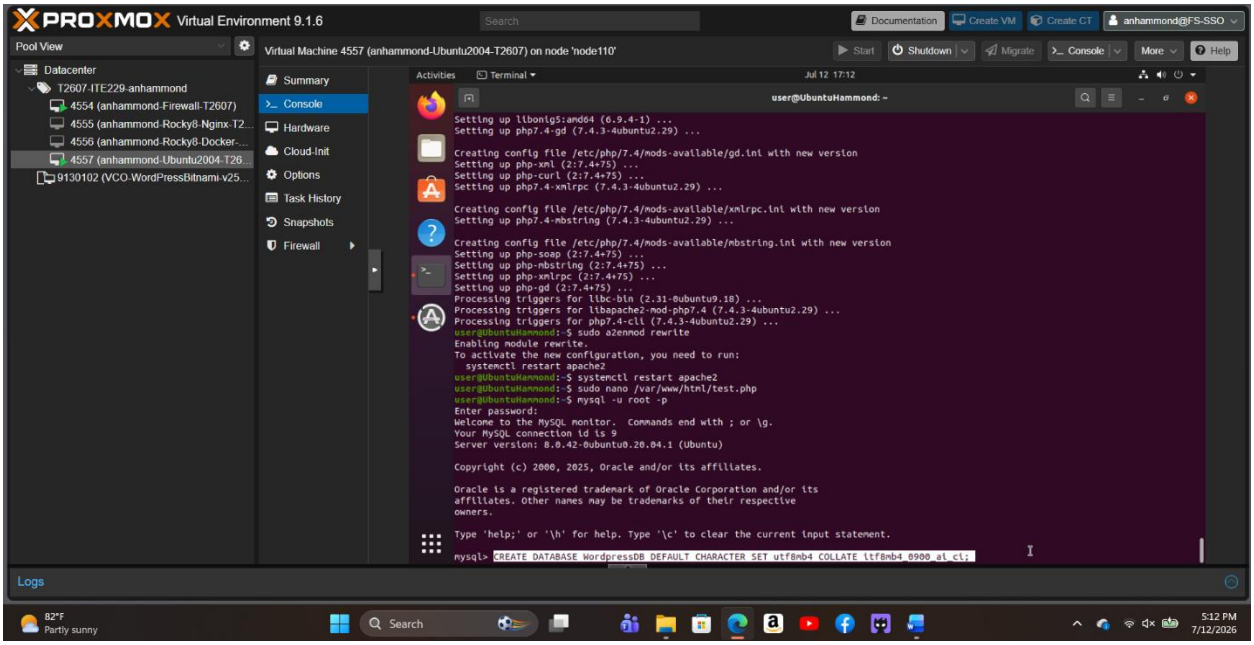
Database Configuration in MySQL

Log into MySQL



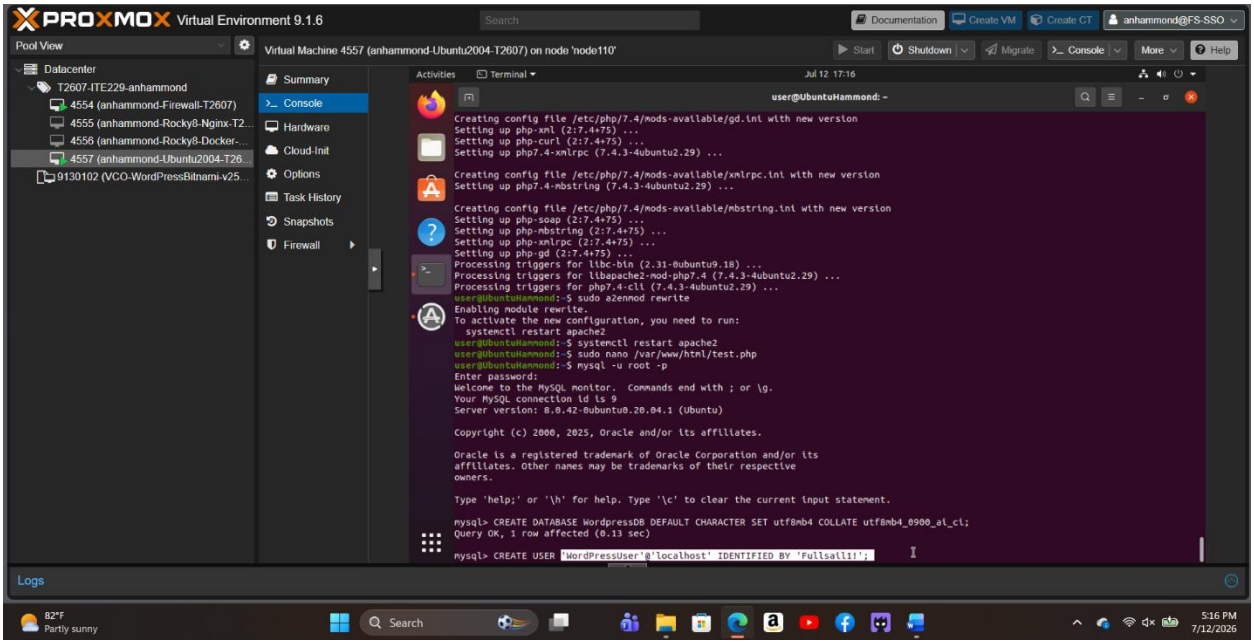
This step connects to the MySQL database server.

Create MySQL WordPress Database (WordPressDB)



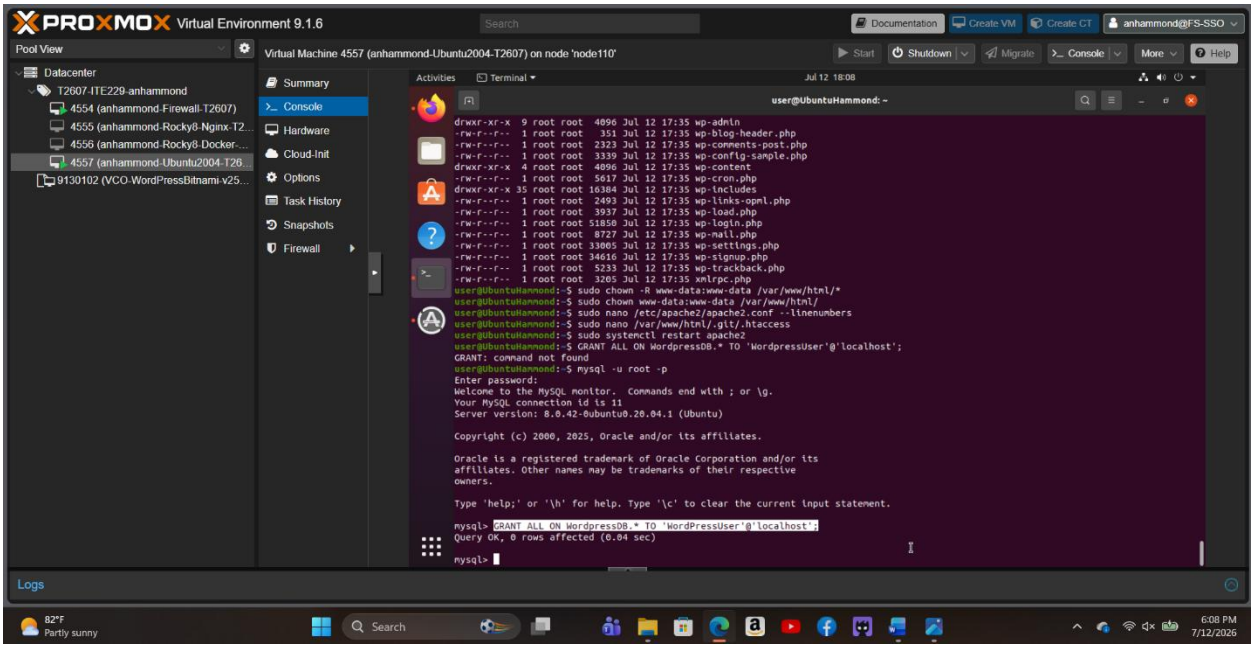
This step creates a dedicated database for the WP website.

Create MySQL WordPress User (WordPressUser)



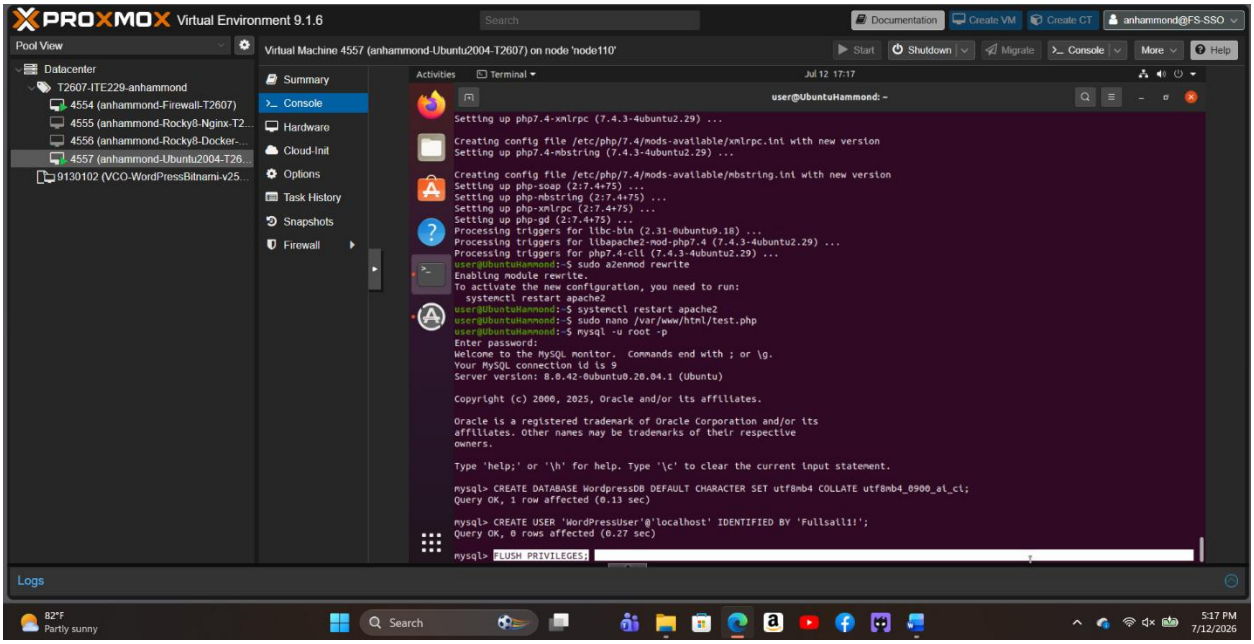
This step creates a MySQL user account for WP to access the WP database.

Grant Privileges to the WordPress Database (WordPressDB) to WordPress User (WordPressUser)



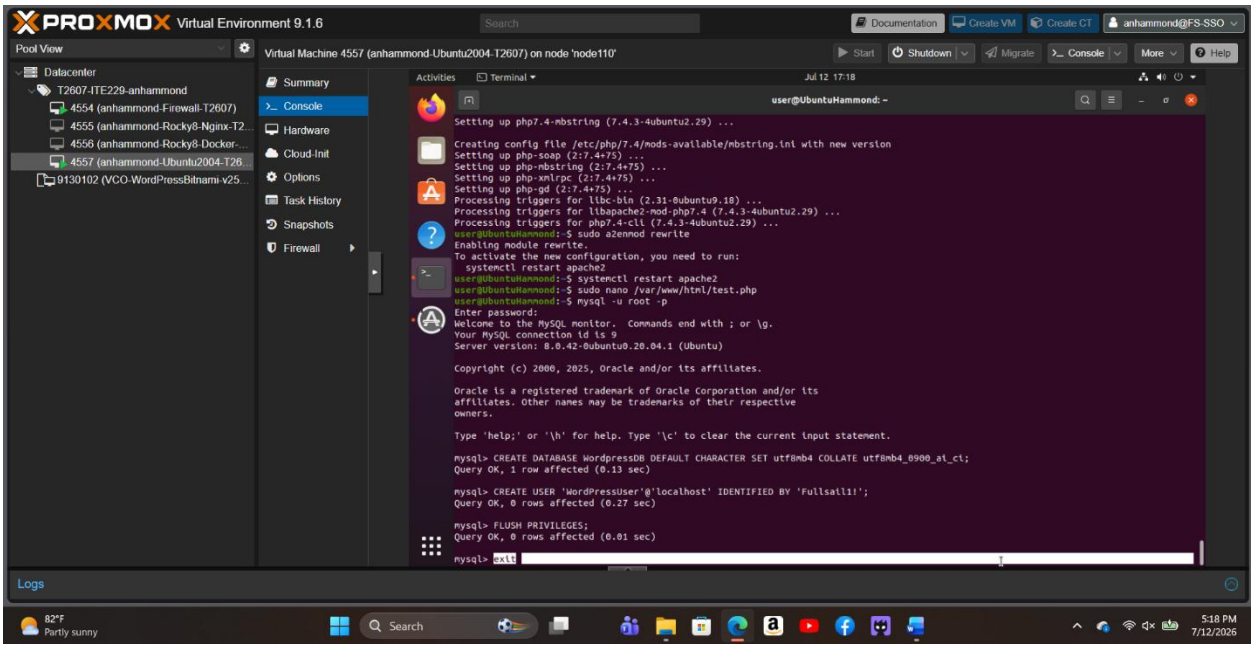
This step grants the WP user account permission to access and manage the WP database.

Flush Privileges



This step reloads MySQL's privilege tables so that any newly created users, passwords, and permission changes take effect immediately.

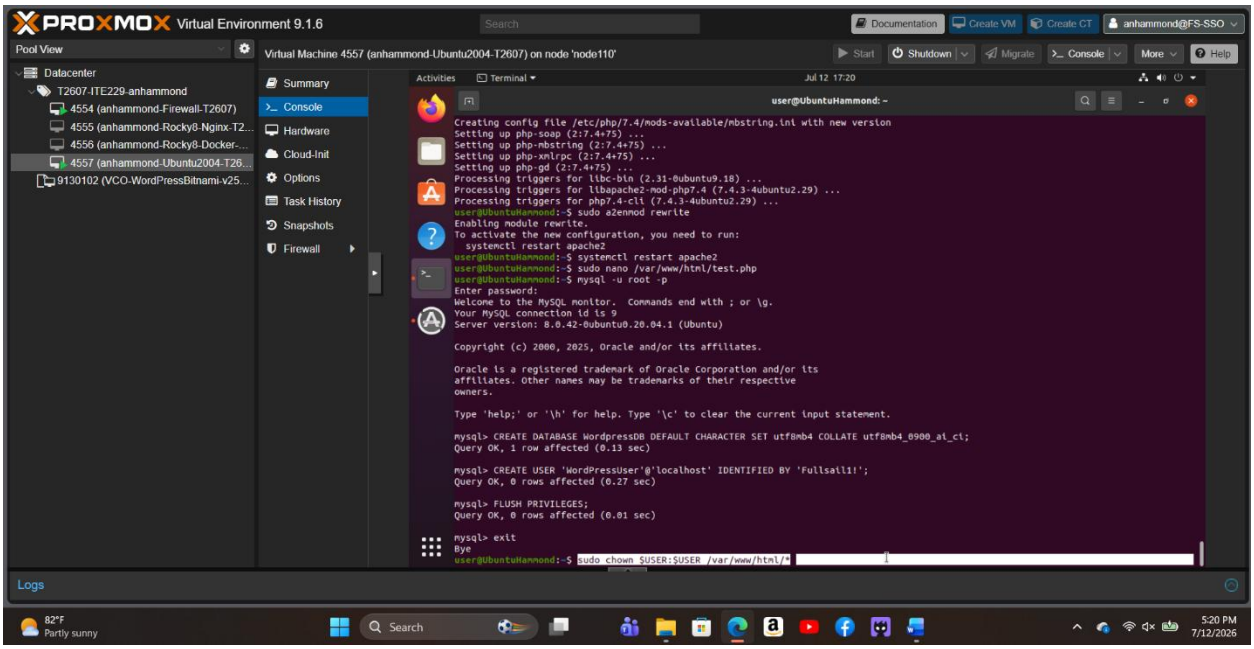
Exit MySQL



This step closes the MySQL command-line interface and returns the user to the Ubuntu terminal.

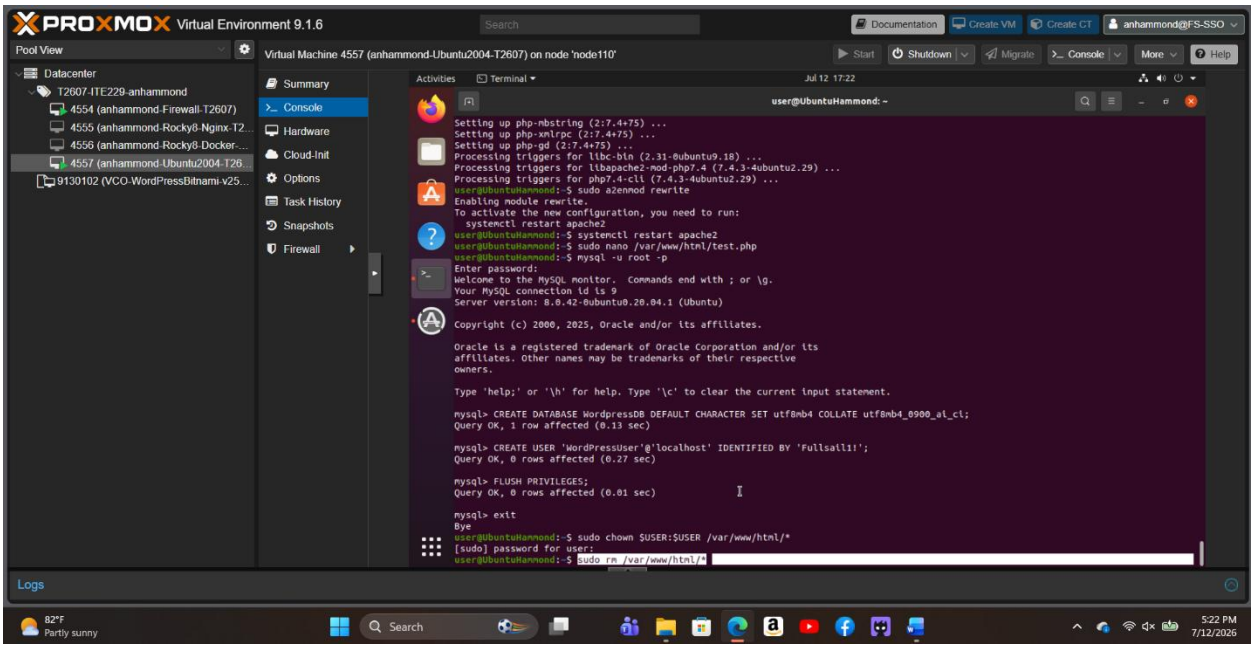
Install WordPress

Grant Permission to /var/www/html/ Directory to WordPress User



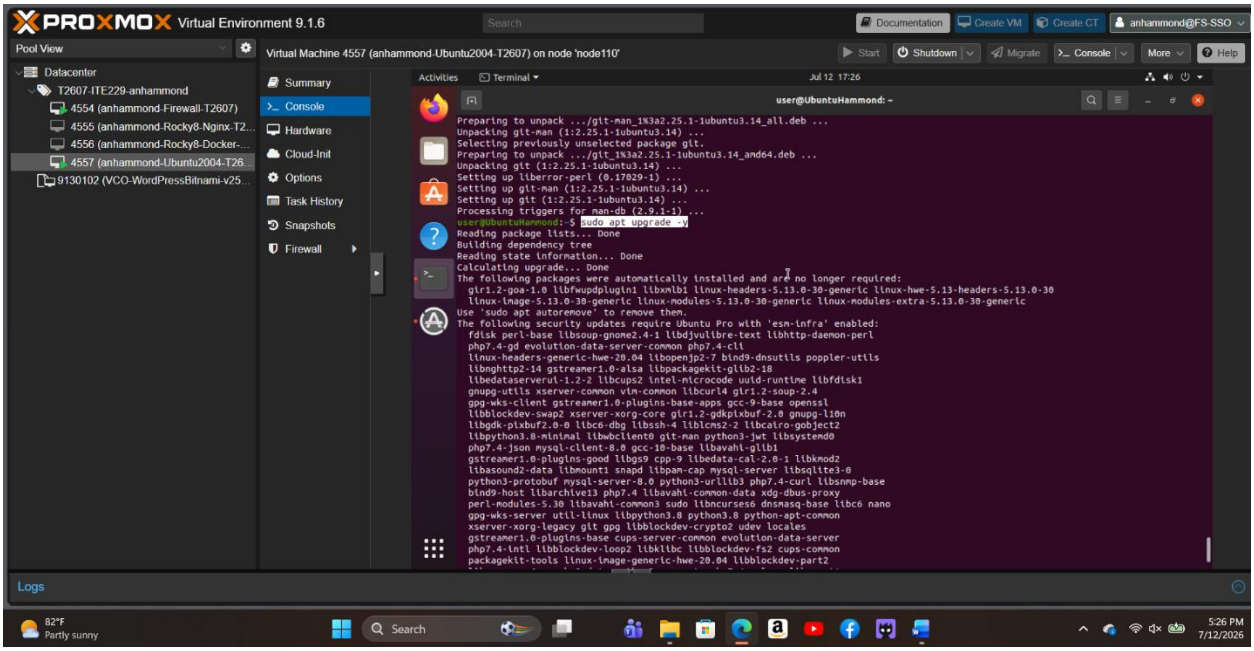
This step assigns the appropriate ownership and permissions to the web server directory where WP files are stored.

Delete Files from /var/www/html/ Directory



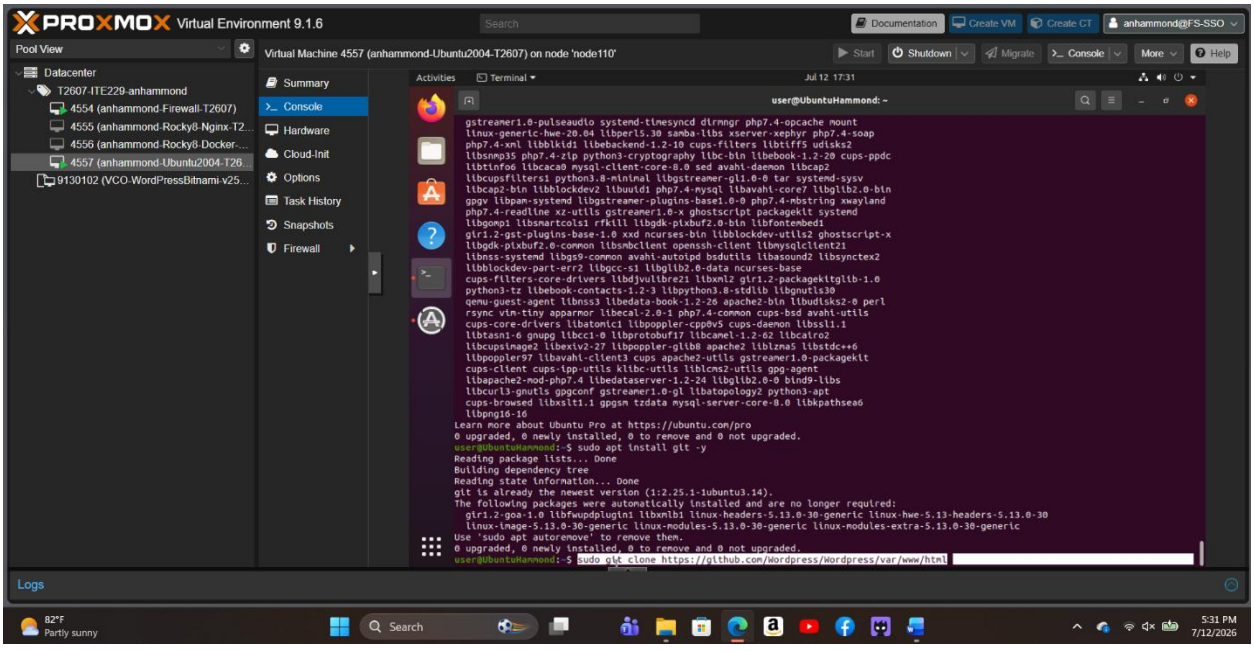
This step removes the default files from the Apache web server's document root directory to prepare it for the WP installation.

Verify /var/www/html/ Directory is Empty



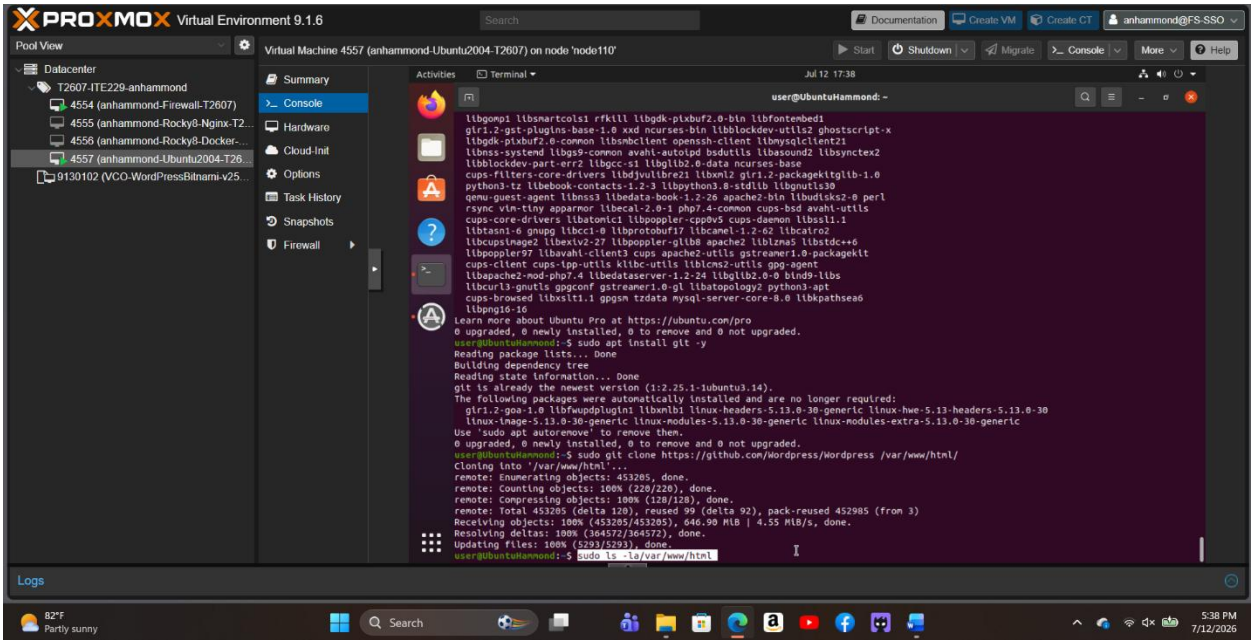
This step confirms that all existing files have been successfully removed from the directory before installing WP.

Clone WordPress to /var/www/html/ Directory



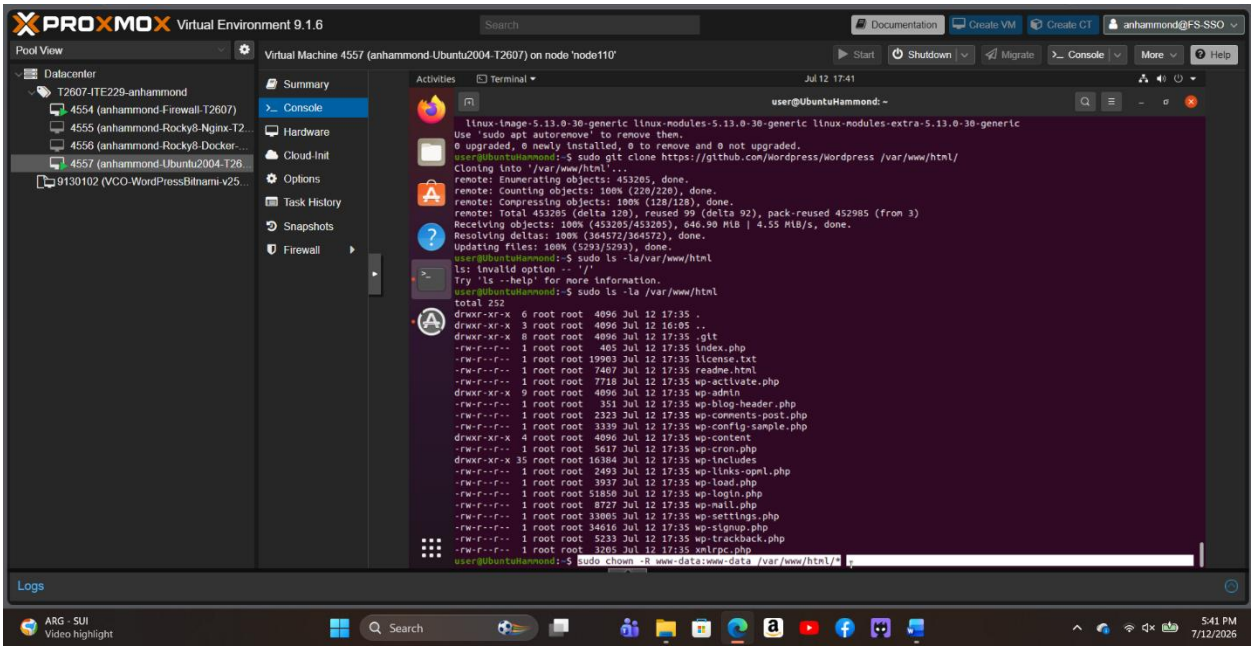
This step downloads and copies the WP installation files into the Apache web server's document root directory.

Verify /var/www/html/ Directory Contains WordPress Files



This step confirms that the WP files were successfully copied into the Apache web server.

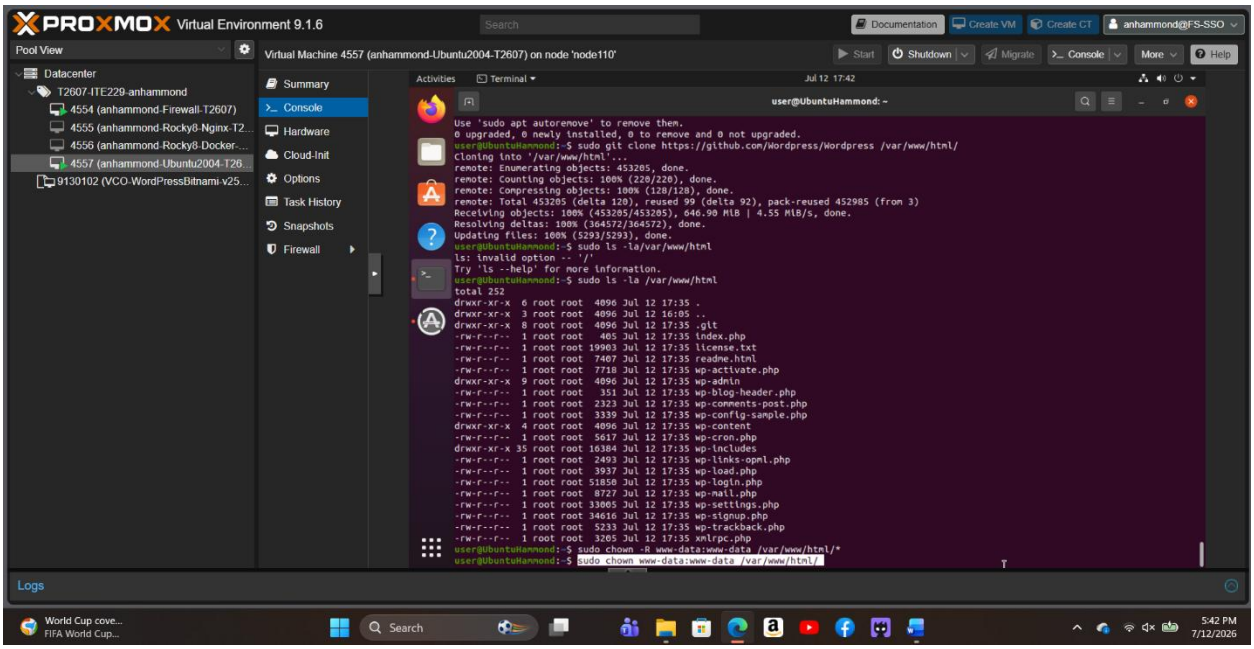
Verify Permissions on /var/www/html/ Directory



This step confirms that the WP files have the correct ownership and permissions.

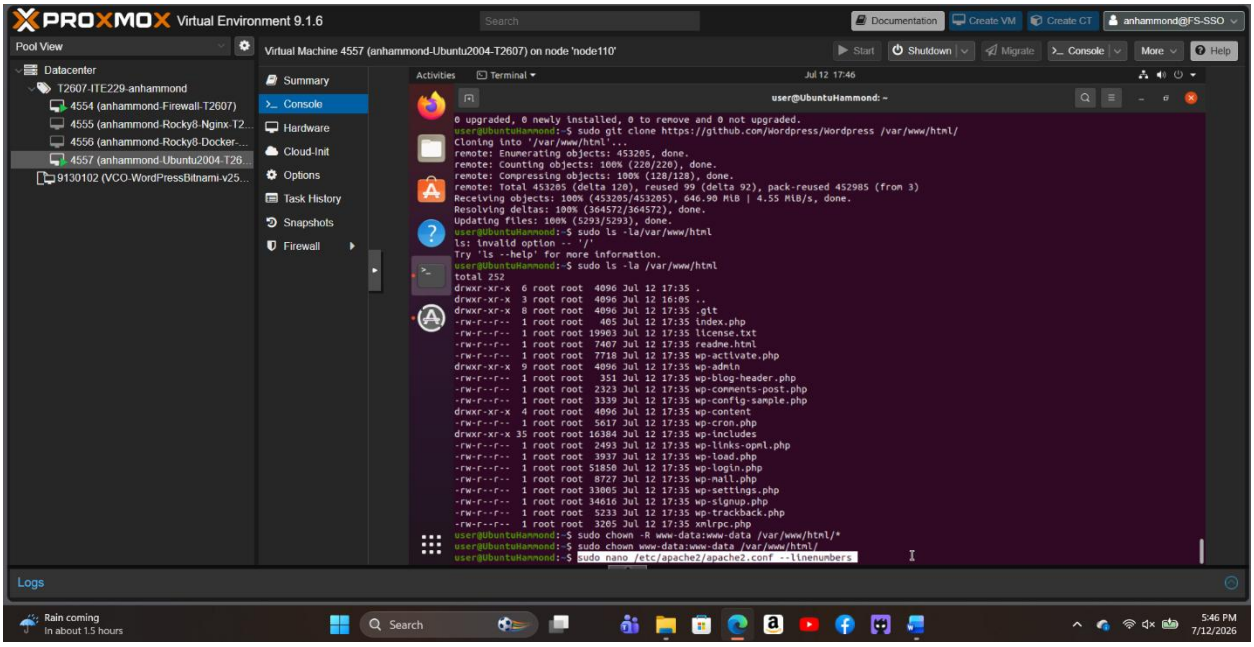
Edit Ownership

Edit Ownership of and the contents of /var/www/html/ Directory



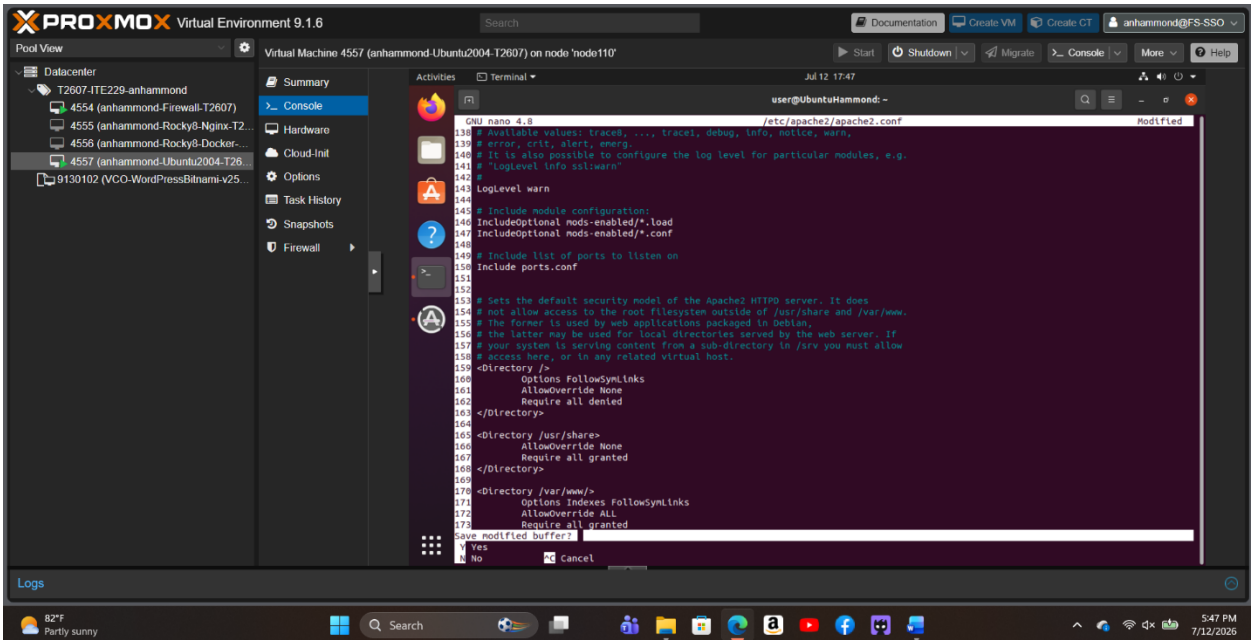
This step changes the ownership of all WP files using the “chown” command.

Edit the apache2.conf File



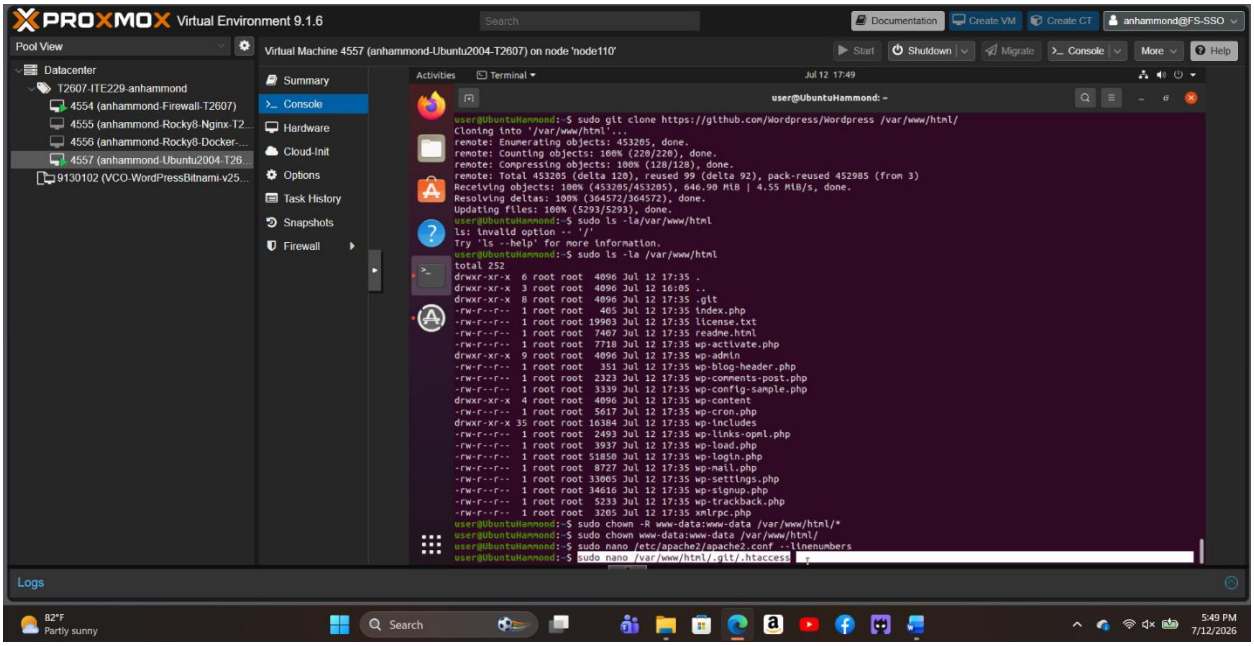
This step modifies the Apache configuration files to allow WP to use URL rewriting.

Override All Default Apache Directives



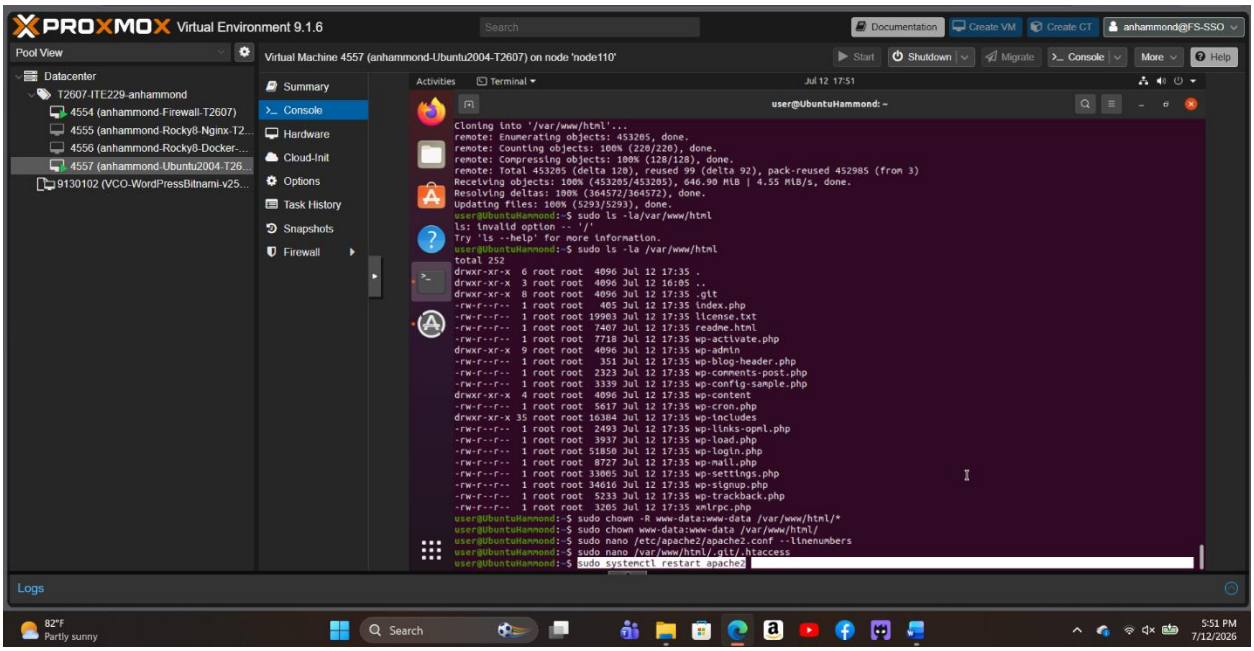
This step updates the Apache configuration to allow WP to use custom settings.

Create a .htaccess File in the /var/www/html/.git/ Directory



This step creates a .htaccess file to control access and support URL rewriting for the WP website.

Restart the Apache Service

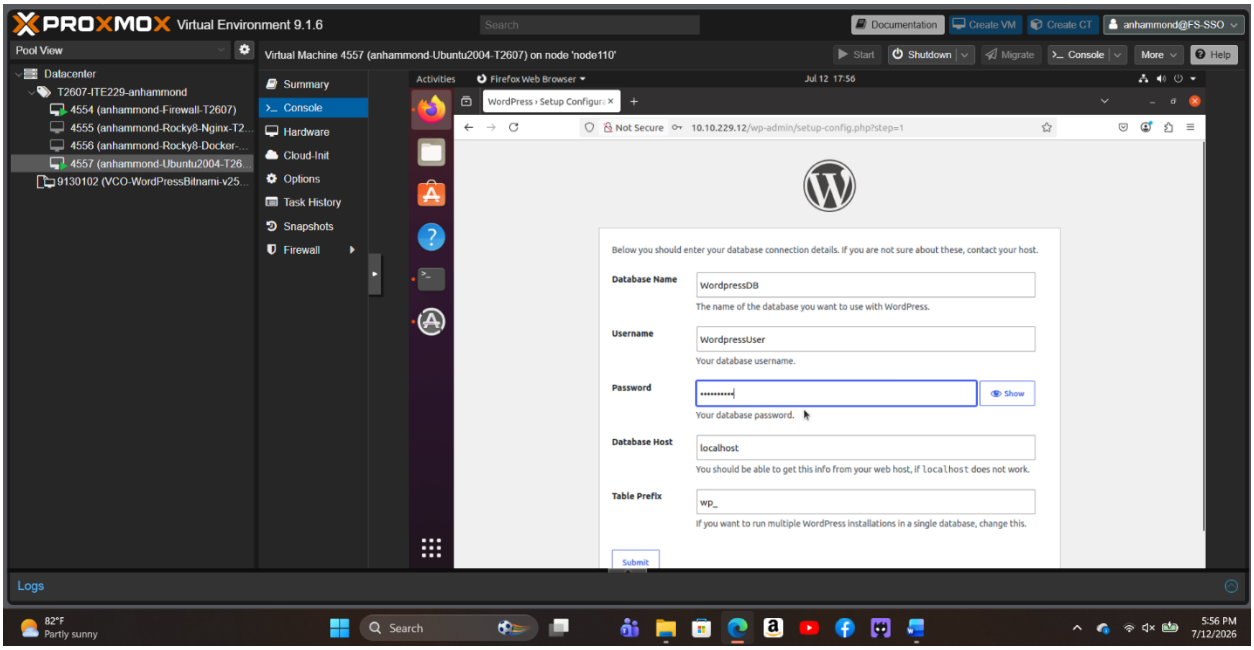


This step restarts the Apache web server to apply all config changes made during the WP installation process.

WordPress Configuration

Configure WordPress

WordPress Configuration Selections



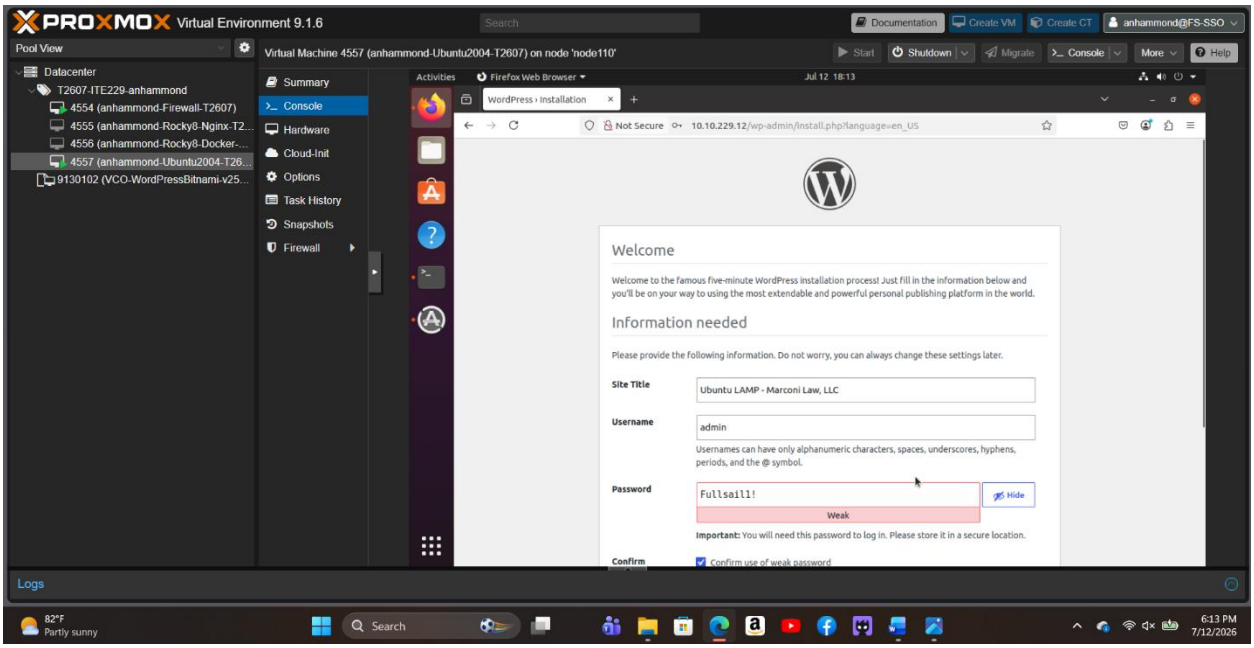
This step completes the WP setup by entering the database connection information in the WP configuration page.

- **Database Name:** WordPressDB
- **Username:** WordPressUser
- **Password:** *this is your WordPressUser password*
- **Database Host:** localhost
- **Table Prefix:** wp_

Run Installation

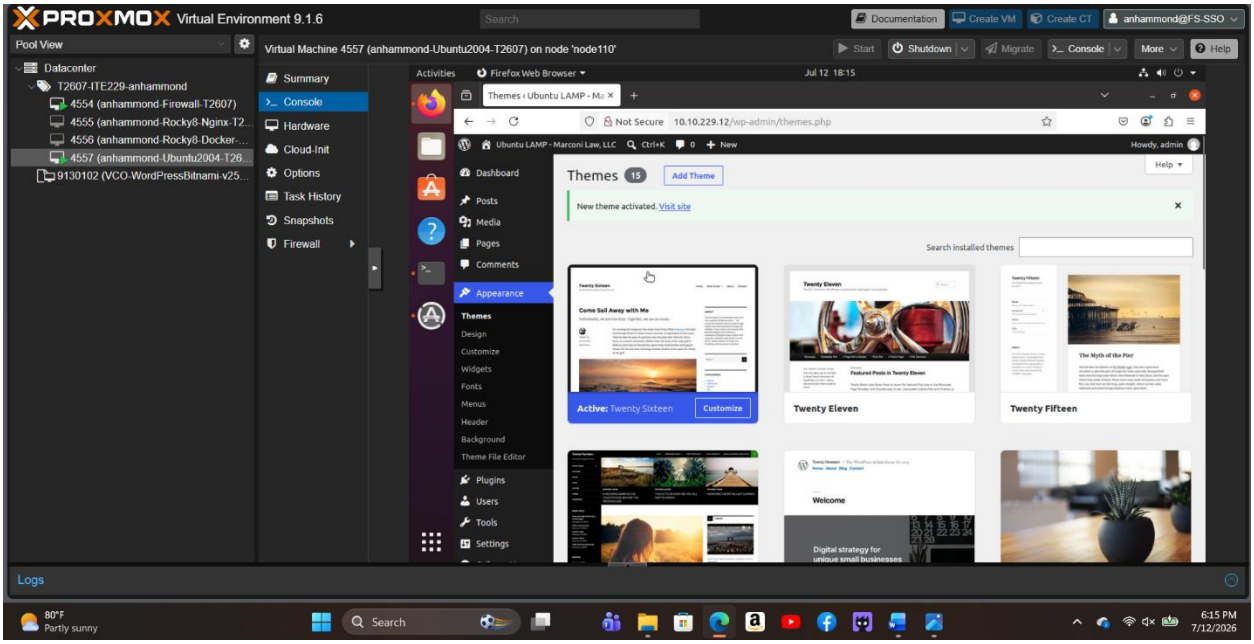
Legible, annotated screenshots AND written instructions/commands required

Create an Admin WordPress User



This step creates the administrator account that will be used to manage the WP website.

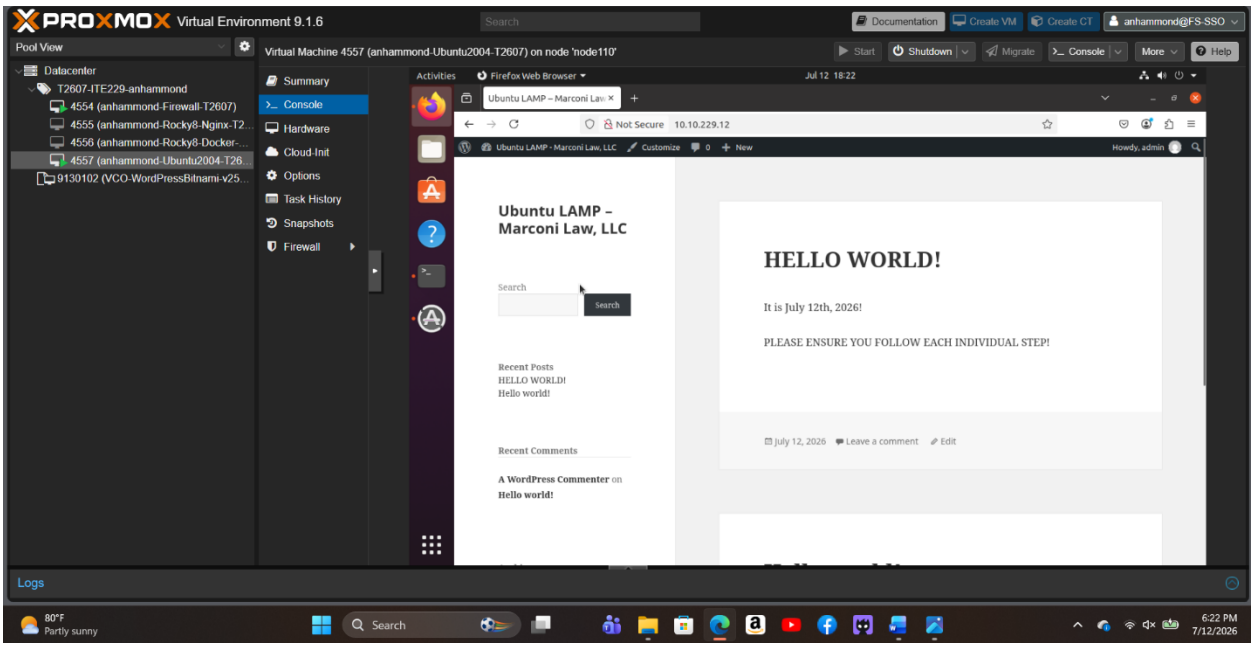
WordPress Site Selections



This step allows me to make changes to the theme.

- **Site Title:** Ubuntu LAMP
- **Username**: admin
- **Password:** *You will create this*
- **Your email:** root@localhost.local
- **Search Engine Visibility:** leave unchecked

Test WordPress Website



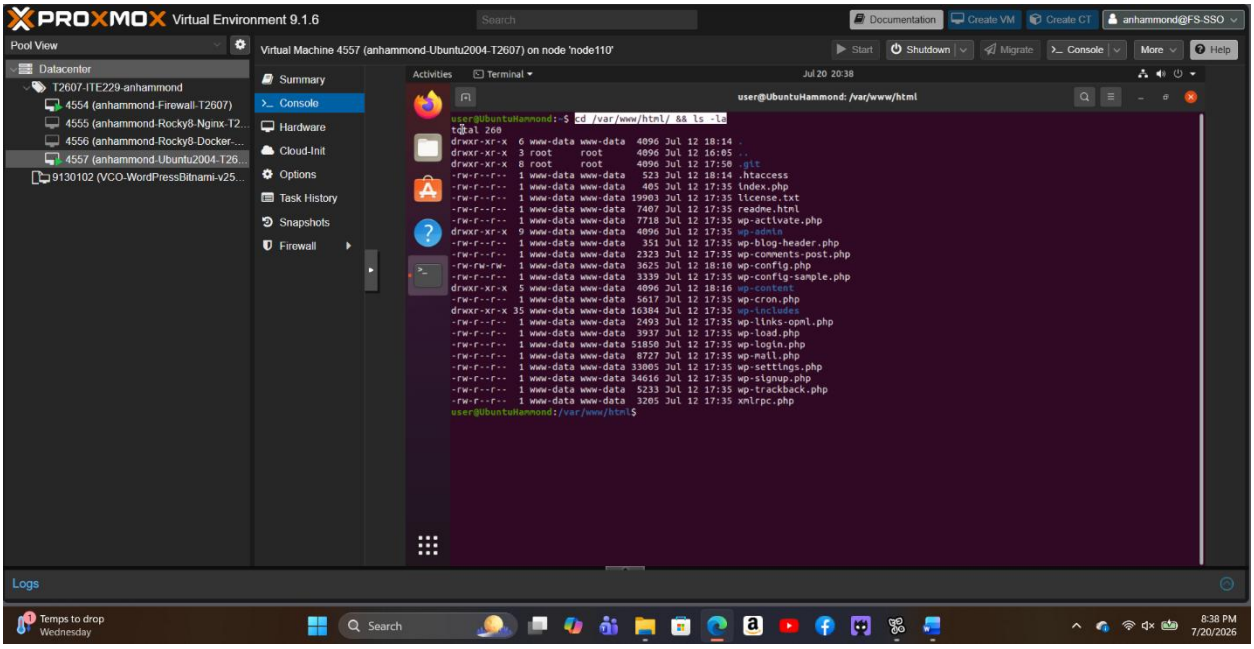
This step allows me to preview a custom post on the WP site.

END OF MILESTONE 2

WordPress Security Settings and Configurations

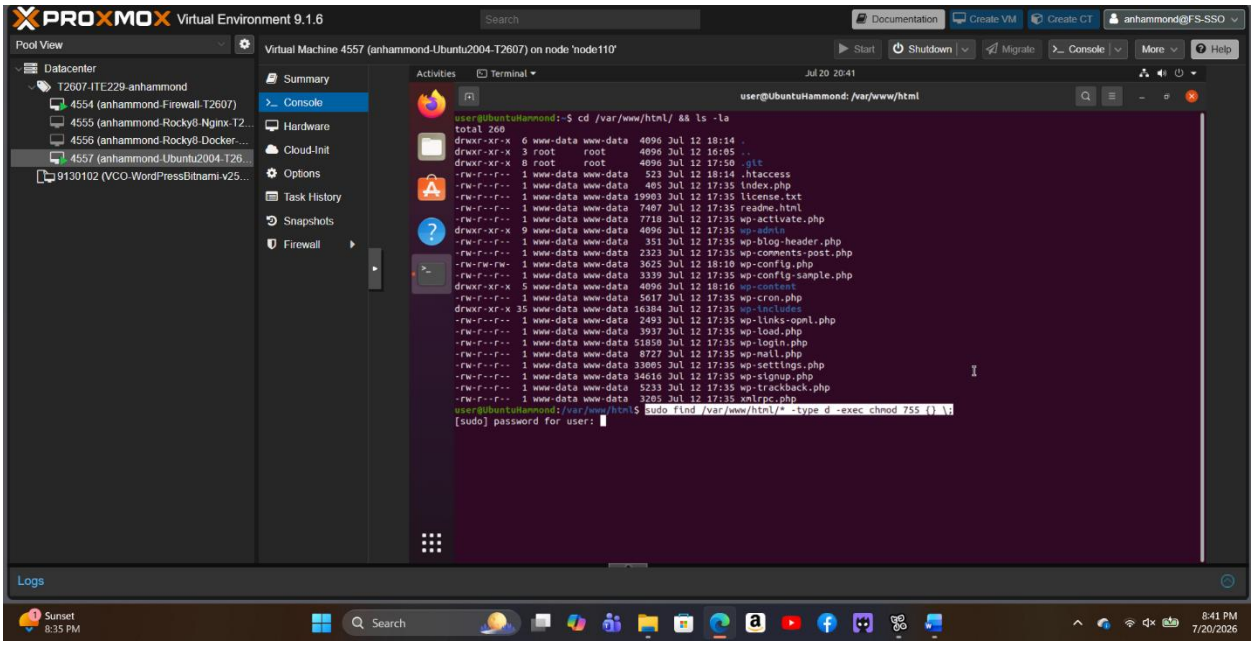
File Permissions

Vulnerability



This command opened up the directory to /var/www/html

Configuration



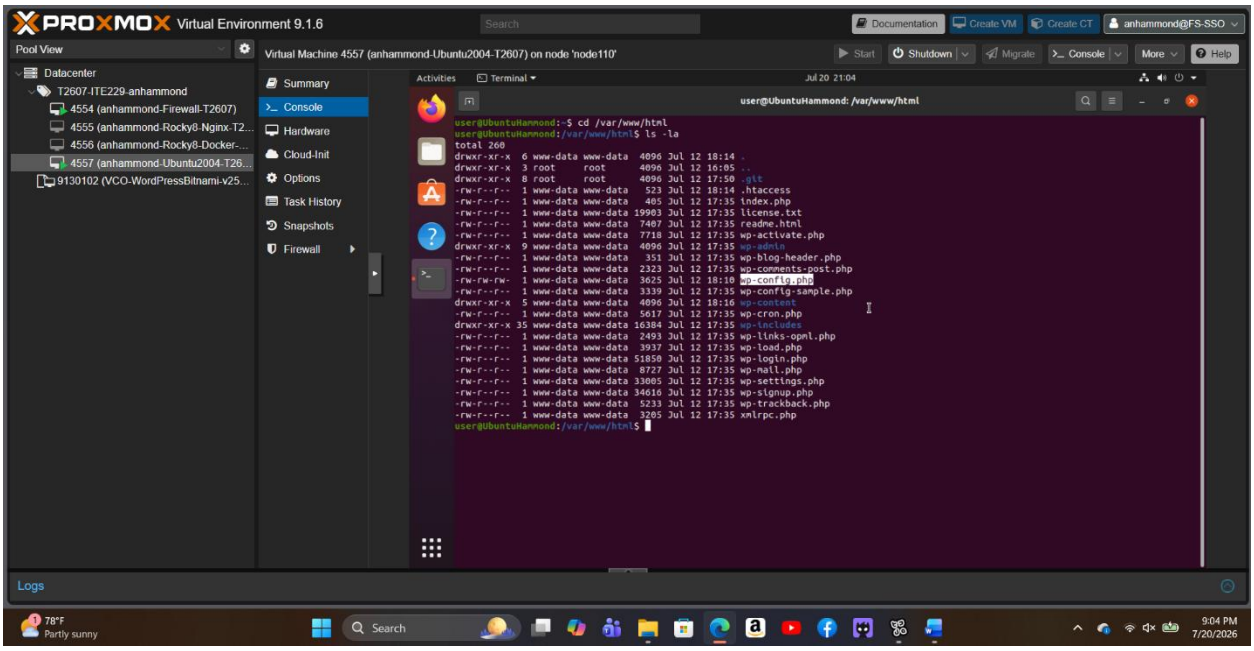
This command recursively changes the permissions

Validation

Finally, you will have a screenshot showing the vulnerability has been closed, or the steps taken to reduce the impact of said vulnerability. Include a brief description of what is occuring in the screenshot.

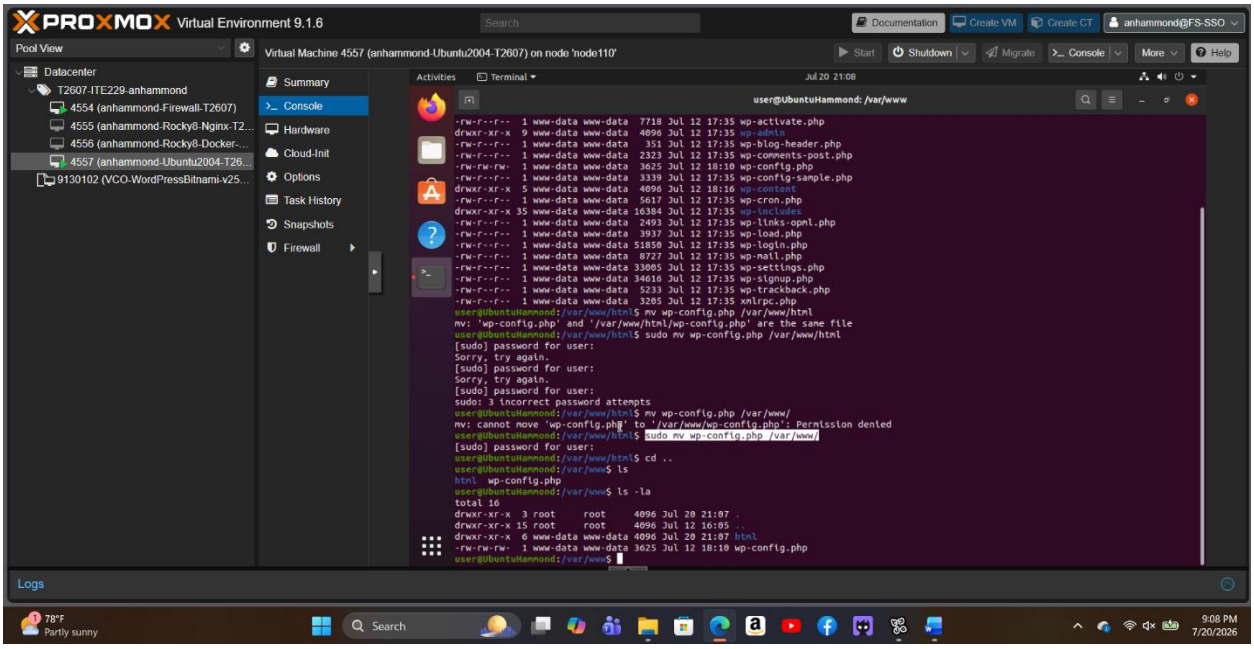
Securing wp-config.php

Vulnerability



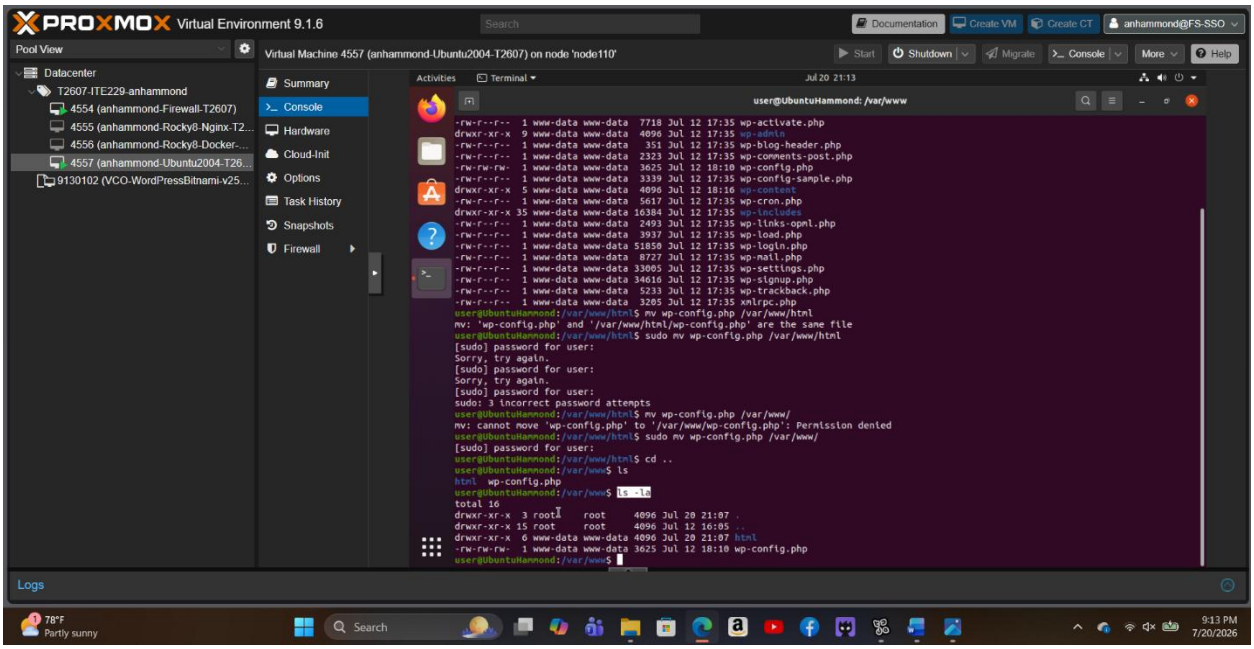
This command lists all files and folders in the WP web directory.

Configuration



The “mv” (move) command renames the file

Validation

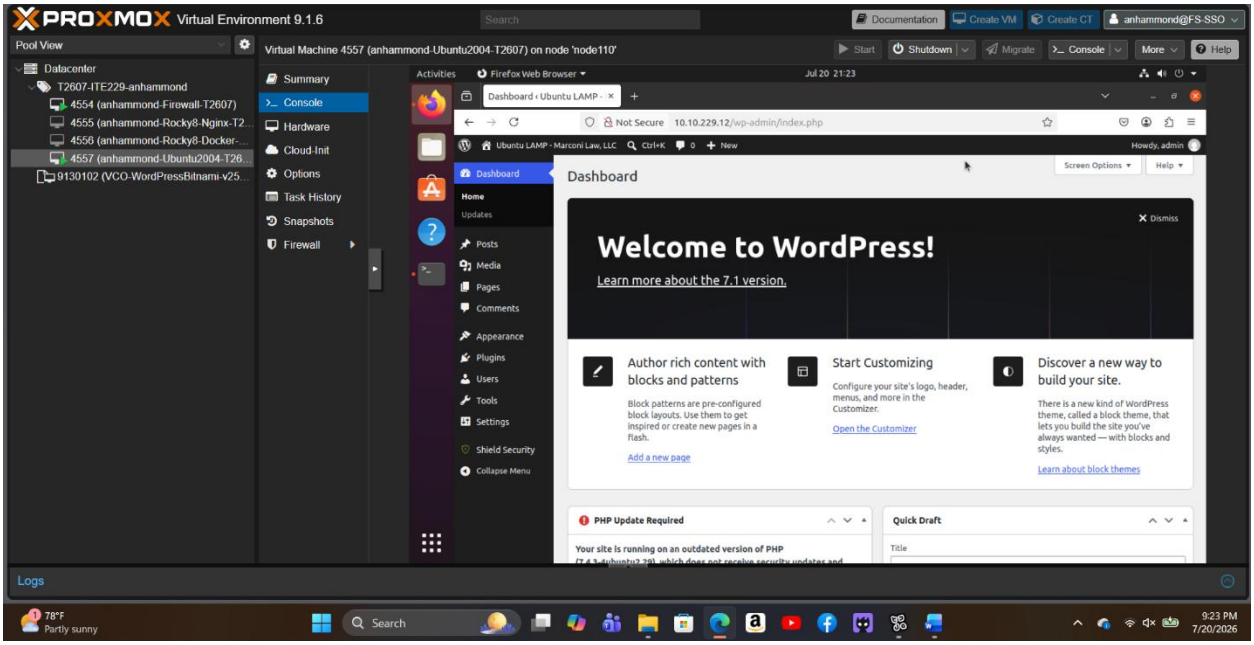


The “ls -la” command is used to verify that the file was successfully renamed.

Firewall (Shield)

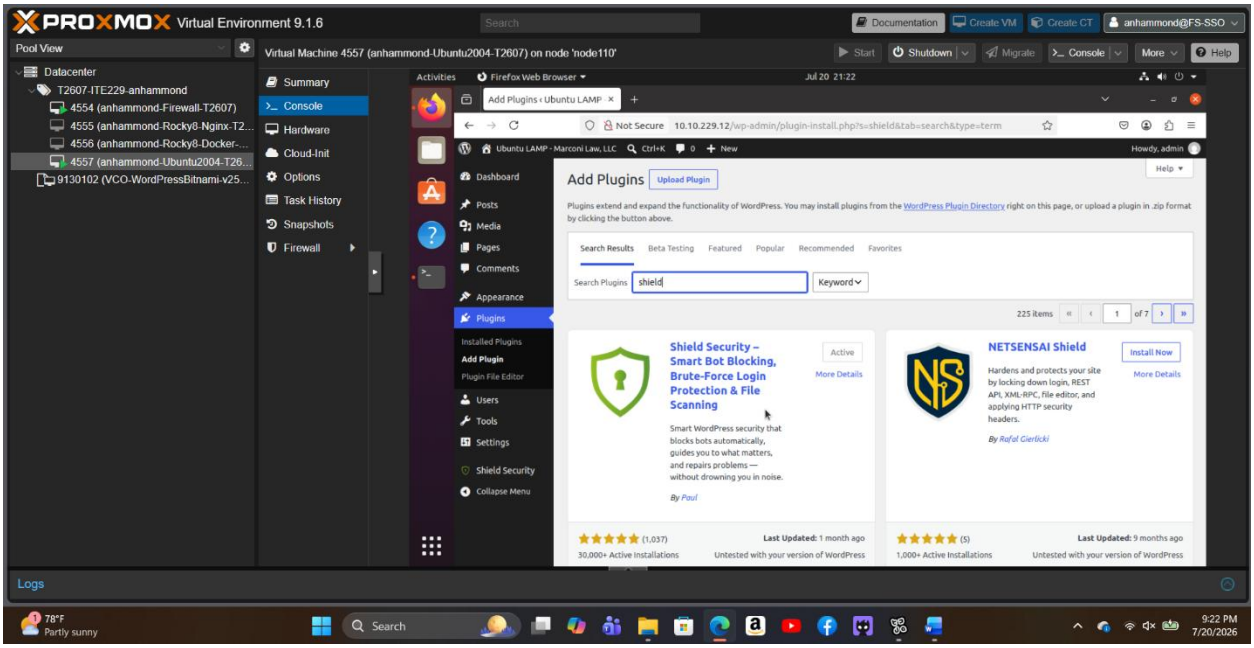
Before and After Legible, annotated screenshots AND written instructions/commands required

Vulnerability



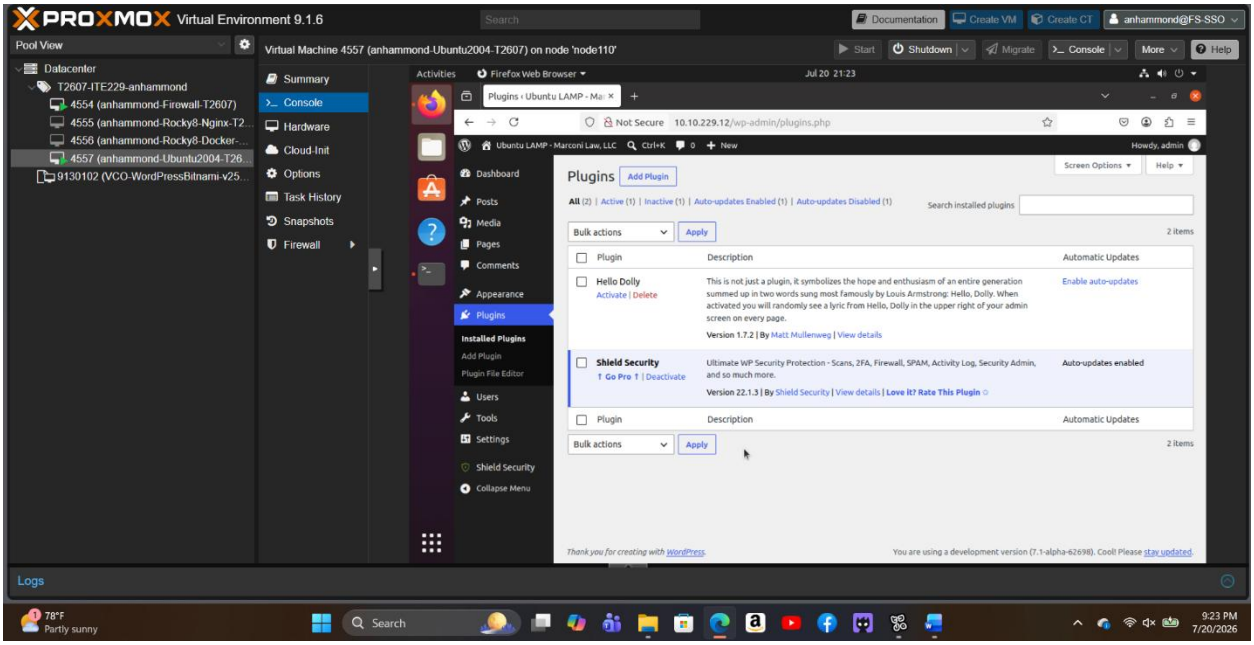
This is before the firewall was applied.

Configuration



I then proceeded to download Shield Pro for my Firewall.

Validation



This shows that the firewall is active and running.

Conclusion

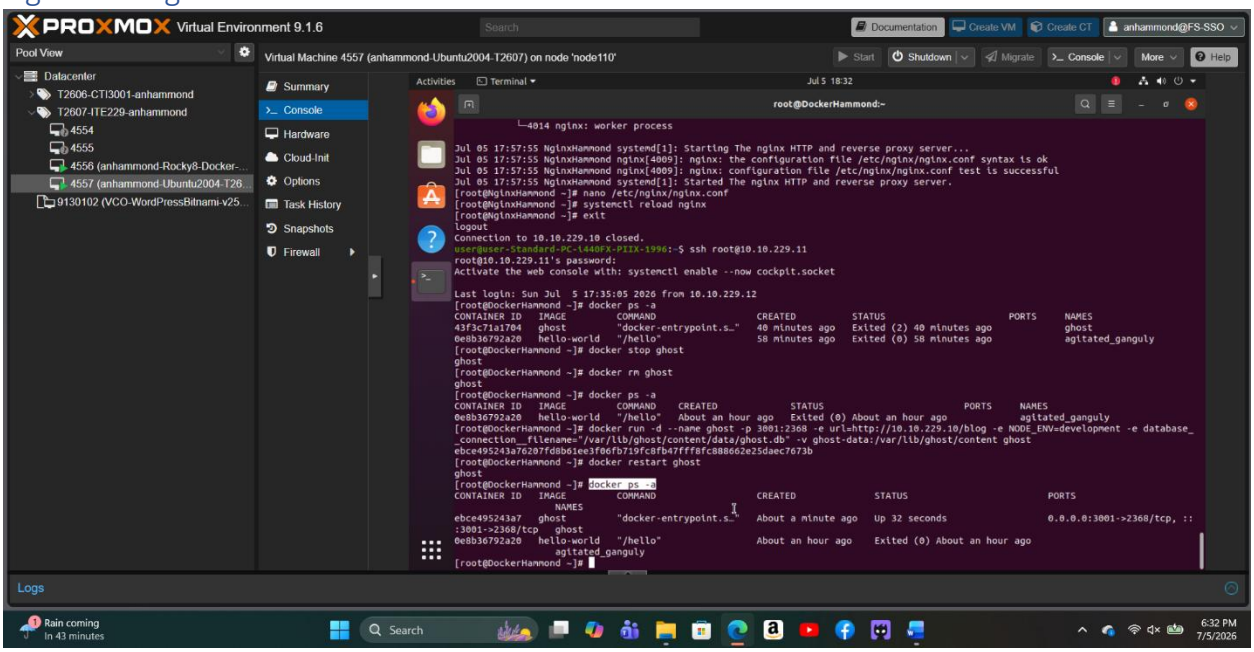
This project was completed for Marconi LLC to evaluate the security of a WordPress web server. The environment included Ubuntu Server, Apache, MySQL, PHP, WordPress, Rocky Linux virtual machines, Docker, and an internal virtual network. The objective was to demonstrate how common configuration mistakes can create security vulnerabilities and how they can be identified.

During testing, WordPress was installed and configured while several intentional vulnerabilities were introduced, including insecure file permissions, modified ownership settings, Apache configuration changes, and WordPress configuration file modifications. These changes were verified using Linux command-line tools to confirm that the vulnerabilities were successfully created.

The results demonstrate that improper file permissions and server misconfigurations can expose a web application to unauthorized access and other security risks. Marconi LLC can use the findings from this project to strengthen its WordPress security practices by implementing secure file permissions, protecting configuration files, and following industry best practices for web server management.

Appendix A

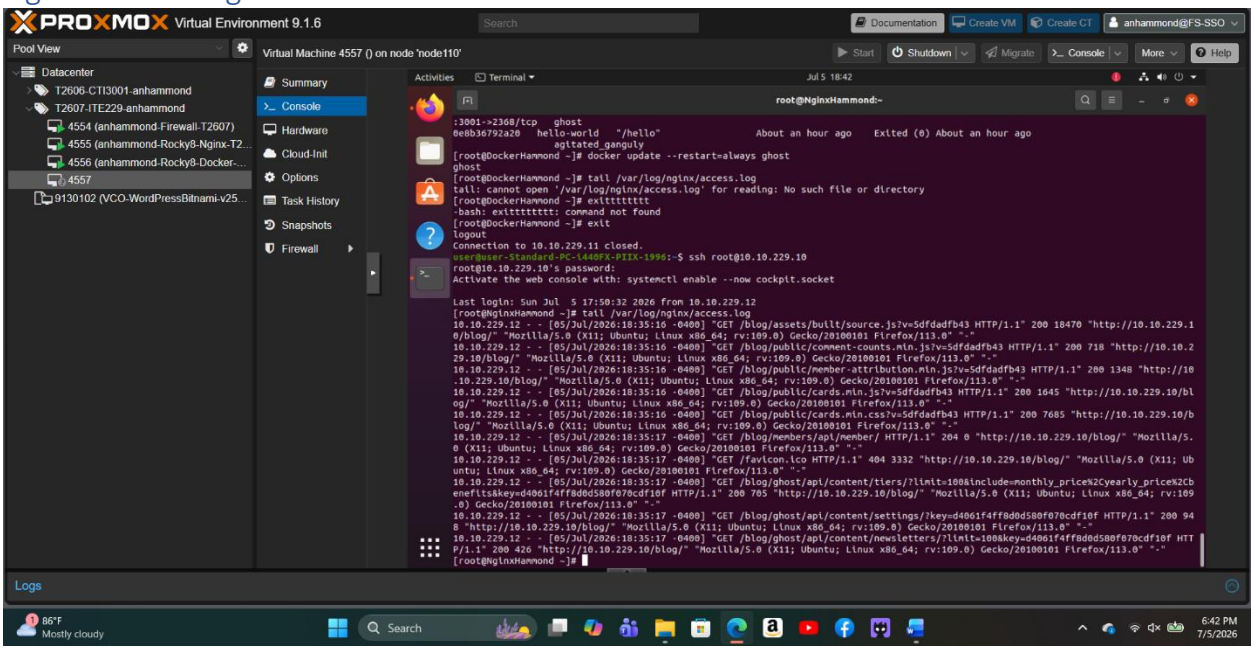
NginX Config File



[Both Appendix A & B are required for Milestone 1. You may delete this after completing the Appendices.]

Appendix B

NginX Access Log File



NginX Error Log File

